Министерство науки и высшего образования Российской Федерации
[Полное наименование университета]
[Институт / факультет]
[Кафедра]

КУРСОВАЯ РАБОТА

по дисциплине «Методы и технологии программирования»

на тему: «Сервис сокращения ссылок с аналитикой»
(вариант 29)

Выполнил: студент группы [группа]

[Фамилия И. О.]

Руководитель: [должность, Фамилия И. О.]

Оценка: _____

[Город] 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 5

1 АНАЛИТИЧЕСКАЯ ЧАСТЬ 9

1.1 Предметная область 9

1.2 Обзор аналогов 13

1.3 Способы генерации коротких кодов 15

1.4 Требования к системе 18

1.5 Выводы по главе 1 23

2 ПРОЕКТНАЯ ЧАСТЬ 25

2.1 Архитектура системы 25

2.2 Диаграмма вариантов использования 26

2.3 Диаграммы последовательности 28

2.4 Модель данных 31

2.5 Проектирование REST API 35

2.6 Диаграмма классов и интерфейсов 37

2.7 Стратегия кэширования 38

2.8 Развёртывание 42

2.9 Выводы по главе 2 45

3 ТЕХНОЛОГИЧЕСКАЯ ЧАСТЬ 46

3.1 Язык программирования Go 46

3.2 PostgreSQL 48

3.3 Redis 50

3.4 Аутентификация: bcrypt и HMAC-токены 50

3.5 Контейнеризация и непрерывная интеграция 52

3.6 Инструменты контроля качества и безопасности 53

3.7 Использование AI-ассистентов в разработке 54
3.8 Выводы по главе 3 55
4 РЕАЛИЗАЦИЯ 56
4.1 Структура репозитория 56
4.2 Генератор коротких кодов 57
4.3 Разрешение ссылки: LinkService.Resolve 58
4.4 Кэш на Redis: счётчик попаданий 59
4.5 Сборщик аналитики 60
4.6 QR-коды 62
4.7 Разбор User-Agent 66
4.8 Аутентификация 67
4.9 HTTP-слой: маршрутизация и middleware 68
4.10 Ограничение частоты запросов 69
4.11 Служебные конечные точки 71
4.12 Миграции и пакетная запись в PostgreSQL 71
4.13 Сохранение данных демо-режима 73
4.14 Запросы статистики 77
4.15 Запуск и корректная остановка 78
4.16 Дашборд 79
4.17 Контейнеризация 81
4.18 Непрерывная интеграция 82
4.19 Пользовательский интерфейс 83
4.20 Выводы по главе 4 87
5 ТЕСТИРОВАНИЕ 89
5.1 Стратегия тестирования 89

5.2 Результаты модульного тестирования и покрытие 90
5.3 Ключевые тест-кейсы 92
5.4 Дефекты, найденные тестами 95
5.5 Статический анализ и безопасность 97
5.6 Производительность 102
5.7 Выводы по главе 5 106
ЗАКЛЮЧЕНИЕ 108
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 112
ПРИЛОЖЕНИЯ 114
Приложение А. Инструкция по запуску 114
Приложение Б. Спецификация REST API 116
Приложение В. SQL-миграция `001_init.sql` 120
Приложение Г. История коммитов 120

ВВЕДЕНИЕ

Рост объёма цифрового контента и каналов его распространения привёл к тому, что ссылка стала базовой единицей маркетинговой и информационной коммуникации. Рекламные кампании, рассылки, публикации в социальных сетях и мессенджерах используют адреса веб-страниц, которые зачастую содержат десятки параметров отслеживания (UTM-метки, идентификаторы кампаний, сессий и партнёров) и достигают нескольких сотен символов. Такие адреса неудобны для восприятия, плохо читаются в печатных материалах, легко искажаются при копировании и переносе строк в сообщениях электронной почты.

Дополнительные ограничения накладывают сами каналы доставки. Длина SMS-сообщения ограничена, и каждый лишний символ в адресе сокращает полезный текст или увеличивает число сегментов и, как следствие, стоимость рассылки. В социальных сетях и мессенджерах длинные ссылки ухудшают внешний вид публикации. Отдельное направление составляют QR-коды: плотность кодируемого изображения растёт с длиной закодированной строки, поэтому короткий адрес даёт более крупный и надёжно считываемый код, что существенно для печатной продукции и наружной рекламы.

Вторым фактором актуальности является потребность в аналитике переходов. Для оценки эффективности канала привлечения необходимо знать, сколько раз и когда была открыта ссылка, из каких стран и с каких устройств пришли посетители, с каких ресурсов осуществлён переход. Прямая ссылка на целевую страницу этой информации не даёт, тогда как сервис-посредник, через который проходит каждый переход, получает её естественным образом. Наконец, использование промежуточного адреса позволяет изменять или отключать ссылку после распространения, задавать ей срок жизни и защищать бренд от использования устаревших или скомпрометированных адресов.

Существующие коммерческие сервисы (Bitly, TinyURL, Rebrandly и др.) решают эту задачу, однако предполагают передачу данных о переходах третьей стороне, накладывают ограничения на бесплатных тарифах и не

позволяют гибко интегрировать сервис во внутреннюю инфраструктуру организации. Для ряда организаций существенным является также размещение данных на собственных серверах с учётом требований Федерального закона № 152-ФЗ «О персональных данных» [19]. Это обусловливает практический интерес к разработке самостоятельного, развёртываемого на собственной инфраструктуре сервиса сокращения ссылок с аналитикой.

Цель работы — разработка и программная реализация веб-сервиса сокращения ссылок с редиректами, сбором статистики переходов (количество, география, источники трафика, устройства), кэшированием популярных ссылок и дашбордом для владельцев ссылок на основе стека Go, PostgreSQL и Redis.

Для достижения поставленной цели необходимо решить следующие **задачи**:

1. Исследовать предметную область, принципы работы сервисов сокращения ссылок и способы учёта переходов; выполнить обзор существующих аналогов и выявить требования к разрабатываемой системе (глава 1).

2. Проанализировать способы генерации коротких кодов, оценить ёмкость пространства кодов и вероятность коллизий и выбрать способ генерации, обеспечивающий непредсказуемость и масштабируемость (раздел 1.3).

3. Сформулировать функциональные и нефункциональные требования, включая требования к производительности редиректа, отказоустойчивости и безопасности (раздел 1.4).

4. Спроектировать архитектуру сервиса, модель данных, программный интерфейс (REST API) и стратегию кэширования; построить UML-диаграммы и диаграммы последовательности (глава 2).

5. Разработать асинхронный конвейер сбора аналитики, не замедляющий обработку редиректа, и обеспечить обезличивание данных о посетителях (разделы 2.3, 2.7).

6. Выбрать технологии и инструменты реализации, организовать контейнерное развёртывание и конвейер непрерывной интеграции с автоматической проверкой качества и безопасности кода (глава 3).

7. Реализовать сервис, покрыть его модульными и интеграционными тестами и подтвердить соответствие требованиям.

Объект исследования — процессы создания, хранения и разрешения коротких ссылок в веб-приложениях, а также сбора и анализа статистики переходов.

Предмет исследования — архитектурные и алгоритмические решения, обеспечивающие быстрый редирект, надёжный учёт переходов и эффективное кэширование популярных ссылок в высоконагруженном веб-сервисе.

Практическая значимость работы заключается в получении готового к развёртыванию сервиса, который запускается одной командой с помощью Docker Compose, не зависит от сторонних облачных платформ и хранит данные на инфраструктуре владельца. Сервис может использоваться маркетинговыми и информационными подразделениями организаций для формирования коротких адресов, отслеживания эффективности кампаний и получения QR-кодов коротких ссылок для печатной продукции. Принятые архитектурные решения (слоистая структура, интерфейсы хранилища и кэша, асинхронная запись событий) могут служить основой для масштабирования и развития системы, а также учебным примером построения сервисов на языке Go.

Пояснительная записка состоит из введения, пяти глав, заключения, списка литературы и приложений. В первой главе рассмотрены предметная область и аналоги, сформулированы требования. Во второй главе приведено проектирование системы. В третьей главе обоснован выбор технологий и описаны применяемые инструменты разработки и контроля качества. В

четвёртой главе описана реализация ключевых компонентов, контейнеризация, непрерывная интеграция и пользовательский интерфейс. В пятой главе изложены стратегия и результаты тестирования, анализ безопасности и производительности. В приложениях приведены инструкция по запуску, спецификация REST API, SQL-миграция и история коммитов.

1 АНАЛИТИЧЕСКАЯ ЧАСТЬ

1.1 Предметная область

1.1.1 Принцип работы сервиса сокращения ссылок

Сервис сокращения ссылок (URL shortener) устанавливает соответствие между коротким идентификатором (кодом) и исходным адресом ресурса. Пользователь передаёт сервису длинный адрес, получает короткий вида `https://example.com/aZ3k9Qp` и распространяет его. Когда посетитель открывает короткий адрес, браузер отправляет запрос на сервис, тот по коду находит исходный адрес и возвращает HTTP-ответ с перенаправлением; браузер автоматически переходит на целевую страницу. Принцип работы показан на рисунке 1.1.

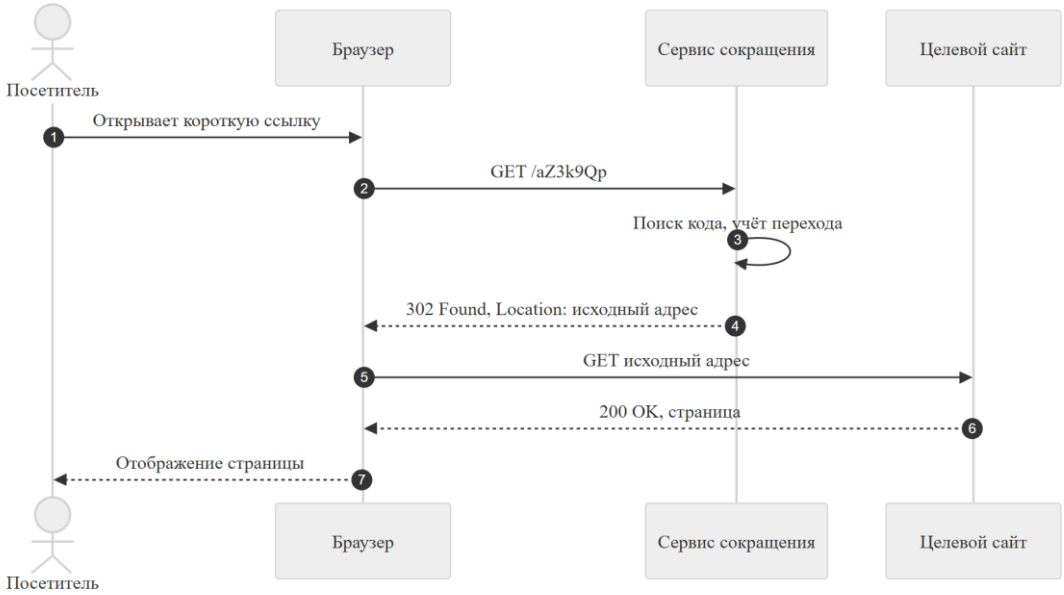


Рисунок 1.1 — Принцип работы сервиса сокращения ссылок

Основными операциями являются создание ссылки (запись пары «код — адрес» в хранилище) и разрешение ссылки (поиск по коду). Соотношение этих операций в реальных системах сильно смещено в сторону чтения: каждая ссылка создаётся один раз, а открывается многократно, нередко тысячи и миллионы раз. Данное свойство определяет архитектуру: путь разрешения

ссылки должен быть максимально быстрым и использовать кэширование, а побочные действия (запись статистики) не должны удлинять ответ [2].

1.1.2 Коды HTTP-перенаправлений

Семантика перенаправлений определена в спецификации RFC 9110 [8]. Для сокращателя значимы четыре кода класса 3xx, отличающиеся двумя свойствами: допускается ли кэширование перенаправления и сохраняется ли метод запроса (таблица 1.1).

Таблица 1.1 — Коды HTTP-перенаправлений

Код	Название	Постоянство	Метод запроса	Замечания
301	Moved Permanently	Постоянное	Клиенты могут заменить POST на GET	Может кэшироваться по умолчанию; повторные переходы браузер может не отправлять на сервис
302	Found	Временное	Клиенты могут заменить POST на GET	Не кэшируется без явных заголовков; каждый переход проходит через сервис
307	Temporary Redirect	Временное	Метод сохраняется	Аналог 302 со строгой семантикой метода
308	Permanent Redirect	Постоянное	Метод сохраняется	Аналог 301 со строгой семантикой метода

Постоянные перенаправления (301, 308) допускают кэширование ответа браузерами и промежуточными узлами. Если браузер запомнил такое перенаправление, при повторном обращении он перейдёт на целевой адрес без запроса к сервису, и этот переход не будет учтён. Кроме того, исправить или отключить ссылку после её кэширования клиентами невозможно. Поскольку основной функцией разрабатываемой системы является учёт каждого перехода, а также возможность деактивации ссылки и ограничения срока её жизни, выбрано временное перенаправление **302 Found**. Метод запроса для навигационных переходов браузера всегда GET, поэтому различие между 302 и 307 несущественно; выбор 302 обусловлен его распространённостью и совместимостью. Для исключения кэширования ответ дополнительно содержит заголовок `Cache-Control: private, max-age=0`. Недостатком

временного перенаправления является несколько меньшая «передача ссылочного веса» поисковыми системами по сравнению с постоянным, однако для рассматриваемой задачи (маркетинг, рассылки, QR-коды) этот фактор не является определяющим.

1.1.3 Источники аналитики

При обработке перехода сервис получает от клиента набор данных, из которых формируется статистика.

1. **Заголовок Referer** содержит адрес страницы, с которой выполнен переход. По нему определяется источник трафика (социальная сеть, поисковая система, внешний сайт). Заголовок является необязательным: он отсутствует при переходе из мессенджера, почтового клиента, при вводе адреса вручную, а также может быть урезан политикой `Referrer-Policy` источника. Пустое значение трактуется как «прямой» переход (`direct`). Для агрегации хранится не полный адрес, а только домен, что уменьшает объём данных и риск сохранения персональной информации, содержащейся в параметрах запроса.

2. **Заголовок User-Agent** описывает клиентское приложение. Из него эвристическим путём извлекаются браузер, операционная система и тип устройства (настольный компьютер, мобильное устройство, планшет), а также выделяются запросы поисковых роботов и программ предварительного просмотра ссылок, которые не должны искажать статистику живых посетителей.

3. **IP-адрес** клиента используется для определения географии (страна) по базе данных соответствия сетей и территорий, например GeoLite2 [20]. Если сервис размещён за обратным прокси или CDN, реальный адрес берётся из доверенного заголовка `X-Forwarded-For`; доверять этому заголовку можно только при явно включённой настройке, иначе клиент сможет подделать адрес. Многие CDN дополнительно передают код страны готовым заголовком (например, `CF-IPCountry`), что позволяет обойтись без локальной базы. Разрабатываемый сервис учитывает этот заголовок так же, как `X-Forwarded-`

`For`, — только при включённой настройке `TRUST_PROXY=true`; в противном случае страна определяется по локальной базе или принимает значение `ZZ` (неизвестно).

4. **Время запроса** используется для построения динамики переходов по дням.

1.1.4 Приватность и обезличивание

IP-адрес в совокупности с иными данными позволяет косвенно идентифицировать физическое лицо и в российском правовом поле может рассматриваться как персональные данные в смысле Федерального закона № 152-ФЗ [19]. Обработка таких данных предполагает наличие правового основания, определённых целей, уведомления субъекта и обязанности по защите. Для минимизации рисков при проектировании принят принцип минимизации данных:

– IP-адрес не сохраняется в базе данных и журналах приложения; он используется только в оперативной памяти для определения страны и вычисления идентификатора посетителя;

– идентификатор посетителя вычисляется как усечённый криптографический хэш SHA-256 от конкатенации секретной соли, IP-адреса и User-Agent и необратим при неизвестной соли; он позволяет подсчитывать уникальных посетителей, но не позволяет восстановить адрес;

– вместо полного Referer хранится только домен, вместо полного User-Agent — классифицированные значения браузера, ОС и устройства;

– гео-информация хранится с точностью до страны.

Следует отметить, что обезличивание уменьшает, но не исключает регуляторные риски полностью: окончательная юридическая квалификация зависит от условий эксплуатации и находится вне рамок настоящей работы.

1.2 Обзор аналогов

Для определения функциональных требований рассмотрены наиболее известные сервисы сокращения ссылок: коммерческие Bitly, TinyURL, Rebrandly, Short.io, бесплатный публичный сервис clck.ru (Яндекс) и два проекта с открытым исходным кодом — YOURLS и Kutt.

Bitly — один из первых и наиболее известных коммерческих сервисов; предоставляет сокращение ссылок, QR-коды, страницы «ссылка в профиле», собственные домены и расширенную аналитику (география, источники, устройства). Основные возможности настройки и глубина истории статистики зависят от тарифа. Имеет публичный API. Исходный код закрыт.

TinyURL — один из старейших публичных сервисов. Базовое сокращение доступно без регистрации; пользовательские алиасы, аналитика и собственные домены связаны с зарегистрированным аккаунтом и платными тарифами. Имеет API.

Rebrandly — сервис, ориентированный на брендированные ссылки с использованием собственного домена компании; предоставляет аналитику, группировку ссылок, API. Часть функций доступна на платных тарифах, существует ограниченный бесплатный тариф.

Short.io — коммерческий сервис с акцентом на собственные домены, аналитику, API и интеграции; имеется ограниченный бесплатный тариф.

clck.ru — публичный сервис компании «Яндекс». Отличается простотой: сокращение выполняется без развёртывания собственной инфраструктуры и без сложной настройки; возможности управления ссылкой (алиасы, срок жизни, личный кабинет с аналитикой) ограничены по сравнению с коммерческими платформами. Предоставляет простой программный интерфейс. Исходный код закрыт.

YOURLS (Your Own URL Shortener) — свободное решение на PHP, разворачиваемое на собственном сервере. Поддерживает пользовательские ключевые слова, базовую статистику переходов, API и расширение плагинами (в частности, для гео-определения, ограничения срока жизни и т. п.). Требует

web-сервера и MySQL-совместимой СУБД; архитектура ориентирована на классические хостинги, высокая нагрузка требует дополнительной настройки кэширования.

Kutt — свободное решение на Node.js для самостоятельного размещения. Поддерживает собственные домены, пользовательские алиасы, защиту ссылки паролем, срок жизни, статистику и API.

Результаты сравнения приведены в таблице 1.2. Оценки носят обобщённый характер: наборы функций и условия тарифов у коммерческих сервисов меняются, поэтому точные ограничения не приводятся и должны уточняться на сайтах сервисов.

Таблица 1.2 — Сравнение сервисов сокращения ссылок

Критерий	**Bitly**	**TinyURL**	**Rebrandly**	**Short.io**	**clck.ru**	**Y**
Собственный алиас	да (в т. ч. на платных тарифах)	да (с аккаунтом)	да	да	ограниченно	да
Срок жизни ссылки	на платных тарифах	на платных тарифах	да	да	нет данных / не основная функция	через
Аналитика: география	да	на платных тарифах	да	да	ограниченно	базов
Аналитика: рефереры	да	на платных тарифах	да	да	ограниченно	да
Аналитика: устройства	да	на платных тарифах	да	да	ограниченно	базов
API	да	да	да	да	простой публичный	да
Открытый код, self-hosted	нет	нет	нет	нет	нет	да
Бесплатный тариф	да, ограниченный	да, ограниченный	да, ограниченный	да, ограниченный	бесплатный сервис	беспл
Основные ограничения	лимиты и функции зависят от тарифа	детальная аналитика платная	лимиты бесплатного тарифа	лимиты бесплатного тарифа	минимум функций, данные у третьей стороны	PHP-с масш вручн

Анализ показывает, что коммерческие сервисы обладают наиболее полным набором функций, но предполагают хранение статистики и ссылок у третьей стороны, зависят от тарифной политики и закрыты для модификации. Открытые решения YOURLS и Kutt закрывают потребность в самостоятельном размещении, но используют иные технологические стеки и не ориентированы на конкретную связку Go, PostgreSQL и Redis. Разрабатываемый сервис ориентирован на минимально достаточный набор функций (алиасы, срок жизни, QR-коды, аналитика по странам, источникам и устройствам, REST API, дашборд) при приоритете производительности редиректа и приватности данных посетителей.

1.3 Способы генерации коротких кодов

1.3.1 Сравнение подходов

Код ссылки должен быть коротким, уникальным и, желательно, непредсказуемым. Рассмотрены три подхода (таблица 1.3).

Таблица 1.3 — Способы генерации коротких кодов

Способ	Принцип	Достоинства	Недостатки
Счётчик + base62	Порядковый номер записи кодируется в системе счисления по основанию 62	Нет коллизий по построению; коды самые короткие	Коды предсказуемы, возможен перебор всех ссылок подряд; требуется глобальный счётчик — узкое место и точка отказа при масштабировании
Хэш URL	Берётся усечённый префикс хэша (MD5, SHA-256) от исходного адреса и кодируется в base62	Одинаковый адрес даёт одинаковый код; не нужен счётчик	Коллизии при усечении, требующие обработки; один код на адрес мешает отдельному учёту для разных владельцев и кампаний; при добавлении «соли» для устранения этого недостатка преимущество теряется

Способ	Принцип	Достоинства	Недостатки
Случайный код	Код формируется из криптографически стойкого генератора псевдослучайных чисел	Непредсказуемость; независимость узлов (нет общего счётчика); простая реализация	Возможны коллизии, поэтому требуются проверка уникальности и повтор

Для сервиса с аналитикой, где каждый владелец должен иметь собственную ссылку на один и тот же адрес (разные кампании — разная статистика), хэширование адреса неприменимо. Счётчик создаёт предсказуемость: злоумышленник может перебрать пространство и получить доступ к «скрытым» ссылкам, не предназначенным для публичного распространения. Поэтому выбран **случайный код** фиксированной длины (по умолчанию 7 символов алфавита `0-9A-Za-z`), формируемый генератором `crypto/rand` с отсечением (rejection sampling) значений, исключающим смещение распределения. Уникальность гарантируется ограничением `UNIQUE` в базе данных; при нарушении выполняется повторная генерация (не более пяти попыток).

1.3.2 Ёмкость пространства кодов

Алфавит base62 содержит 62 символа (10 цифр, 26 строчных и 26 прописных латинских букв). Число различных кодов длиной L равно 62^L. Значения для разных длин приведены в таблице 1.4.

Таблица 1.4 — Ёмкость пространства кодов

Длина кода L	Число кодов 62^L	Порядок величины
5	916 132 832	$9{,}2 \cdot 10^8$
6	56 800 235 584	$5{,}7 \cdot 10^{10}$
7	3 521 614 606 208	$3{,}5 \cdot 10^{12}$
8	218 340 105 584 896	$2{,}2 \cdot 10^{14}$

При $L = 7$ доступно $N = 62^7 \approx 3{,}52 \cdot 10^{12}$ кодов, что многократно превышает потребности даже крупных сервисов.

1.3.3 Вероятность коллизий

Вероятность того, что среди n независимо случайно выбранных кодов из N возможных найдётся хотя бы одна пара совпадающих, описывается задачей о днях рождения (парадоксом дней рождения) [2]:

$$P(n) = 1 - \prod_{k=0}^{n-1} (1 - k/N) \approx 1 - \exp(-n(n-1) / (2N)).$$

Для $n = 10^7$ и $N = 3{,}52 \cdot 10^{12}$ показатель экспоненты равен

$$n(n-1)/(2N) \approx 10^{14} / (2 \cdot 3{,}52 \cdot 10^{12}) \approx 14{,}2,$$

откуда $P \approx 1 - e^{-14{,}2} \approx 1 - 6{,}8 \cdot 10^{-7} \approx 0{,}9999993$. Иными словами, среди десяти миллионов случайных 7-символьных кодов совпадение хотя бы одной пары практически неизбежно, а математическое ожидание числа совпадающих пар равно примерно 14. Это не означает проблемы для системы: коллизия возникает только тогда, когда сгенерированный код **уже занят**, и она обнаруживается при вставке благодаря ограничению уникальности. Практический интерес представляет вероятность коллизии при вставке очередной записи в таблицу, содержащую k кодов: $p = k/N$. Для $k = 10^7$ имеем $p \approx 10^7 / 3{,}52 \cdot 10^{12} \approx 2{,}8 \cdot 10^{-6}$, то есть примерно один случай на 350 тысяч создаваемых ссылок. Вероятность того, что все пять последовательных попыток окажутся неудачными, составляет $p^5 \approx 1{,}8 \cdot 10^{-28}$ и пренебрежимо мала. Суммарное ожидаемое число повторных генераций при создании 10^7 ссылок равно сумме k/N по k от 0 до $n-1$, то есть $\approx n^2/(2N) \approx 14$. Таким образом, при длине 7 накладные расходы на обработку коллизий ничтожны (около 14 дополнительных обращений на 10^7 вставок).

Если бы уникальность не проверялась (например, при хэшировании без контроля), вероятность ошибки была бы неприемлемой уже при существенно меньшем числе записей: значение n, при котором $P = 0{,}5$, составляет приблизительно $1{,}18 \cdot \sqrt{N} \approx 2{,}2 \cdot 10^6$ ссылок. Это обосновывает обязательность проверки уникальности на уровне СУБД, а не на уровне приложения.

Непредсказуемость кодов оценивается вероятностью угадывания существующей ссылки одной попыткой: при 10^7 активных ссылок она составляет около $2{,}8 \cdot 10^{-6}$. Для перебора, существенно затрагивающего

пространство, требуются миллиарды запросов, что делает перебор экономически бессмысленным и легко обнаруживаемым при мониторинге.

1.3.4 Пользовательские алиасы

Помимо случайных кодов, владелец ссылки может задать собственный алиас (3–32 символа `A-Za-z0-9_-`, не начинающийся с `-` или `_`). Алиас проверяется на допустимые символы и на отсутствие в списке зарезервированных путей (`api`, `dashboard`, `static`, `healthz`, `metrics`, `login`, `register`, `admin`, `favicon.ico`, `robots.txt`), чтобы он не перекрывал служебные адреса сервиса. Занятость алиаса определяется тем же ограничением уникальности; при конфликте возвращается ошибка `409 Conflict`.

1.4 Требования к системе

1.4.1 Функциональные требования

ФТ-1. Регистрация и аутентификация. Система должна позволять пользователю зарегистрироваться по адресу электронной почты и паролю (от 8 до 72 байт — предел алгоритма bcrypt) и войти в систему, получив токен доступа ограниченного срока действия.

ФТ-2. Создание короткой ссылки. Аутентифицированный пользователь должен иметь возможность создать короткую ссылку для адреса с протоколом `http` или `https`, указав при необходимости название, пользовательский алиас и срок действия.

ФТ-3. Валидация адреса. Система должна отклонять пустые, чрезмерно длинные (более 2048 символов), синтаксически некорректные адреса, адреса с иными схемами (`javascript:`, `data:`, `file:` и др.), а также адреса, ведущие на сам сервис.

ФТ-4. Перенаправление. Любой посетитель должен при открытии короткого адреса `/{code}` быть перенаправлен на исходный адрес ответом `302`

`Found`; для несуществующего кода возвращается `404`, для истёкшей или отключённой ссылки — `410 Gone`.

ФТ-5. Срок жизни и деактивация. Система должна прекращать перенаправление по истечении срока действия ссылки и позволять владельцу отключить ссылку.

ФТ-6. Управление ссылками. Владелец должен получать список своих ссылок (с постраничной выдачей), просматривать отдельную ссылку и удалять (деактивировать) её. Доступ к чужим ссылкам запрещён.

ФТ-7. Сбор статистики. Система должна регистрировать каждый переход с указанием времени, реферера (домена), страны, браузера, ОС и типа устройства и вычислять обезличенный идентификатор посетителя.

ФТ-8. Выдача статистики. Владелец должен получать агрегированную статистику по ссылке за выбранный период (по умолчанию 30 дней, максимум 366 дней): общее число переходов, число уникальных посетителей, распределение по дням, странам, источникам, браузерам, ОС и типам устройств.

ФТ-9. Дашборд. Система должна предоставлять веб-интерфейс, в котором владелец создаёт ссылки, видит их список и графики статистики.

ФТ-10. Популярные ссылки. Система должна публично выдавать список наиболее популярных кодов.

ФТ-11. Служебные функции. Система должна предоставлять проверку состояния (`/healthz`) и метрики для мониторинга (`/metrics`).

ФТ-12. QR-код ссылки. Система должна выдавать QR-код короткой ссылки в виде PNG-изображения запрошенного размера (от 64 до 1024 пикселей, по умолчанию 256) по адресу `/qr/{code}.png`; для несуществующего кода возвращается `404`, для отключённой или истёкшей ссылки — `410 Gone`, для недопустимого размера — `400`. Выдача QR-кода не считается переходом и не учитывается в статистике; дашборд предоставляет просмотр и скачивание кода.

1.4.2 Нефункциональные требования

НФТ-1. Производительность редиректа. Разрешение популярной ссылки должно выполняться без обращения к основной СУБД (через кэш); задержка обработки на стороне сервиса при попадании в кэш должна измеряться единицами миллисекунд. Запись статистики не должна выполняться в критическом пути запроса. Производительность проверяется микробенчмарками и нагрузочным тестом редиректа (генератор `cmd/loadgen`; подготовлен также сценарий k6).

НФТ-2. Доступность при отказе кэша. Недоступность Redis не должна приводить к отказу редиректов: сервис обязан деградировать до чтения из PostgreSQL, фиксируя предупреждения в журнале.

НФТ-3. Устойчивость к перегрузке аналитики. При переполнении очереди событий сервис должен отбрасывать события с учётом счётчика потерь, но не блокировать обработку переходов.

НФТ-4. Надёжность данных. Данные ссылок, пользователей и переходов хранятся в реляционной СУБД с транзакционными гарантиями; схема изменяется только через версионируемые миграции. При остановке сервиса необработанные события аналитики должны быть сброшены в хранилище (graceful shutdown). В демонстрационном режиме без СУБД данные не должны теряться при закрытии программы: состояние сохраняется в файл атомарно (сбой посреди записи не повреждает прежнюю версию), при принудительном завершении допускается потеря лишь изменений последних долей секунды, а повреждённый файл данных не должен молча перезаписываться.

НФТ-5. Безопасность. Требования рассмотрены в разделе 1.4.3.

НФТ-6. Масштабируемость. Приложение не должно хранить пользовательское состояние в памяти процесса (токены самодостаточны, кэш — во внешнем Redis), чтобы допускалось горизонтальное масштабирование за балансировщиком.

НФТ-7. Конфигурируемость и переносимость. Все параметры задаются переменными окружения; система запускается одной командой Docker Compose; без внешних сервисов возможен запуск в демонстрационном режиме с хранилищем в памяти, при желании со снимком данных в файле (`DATA_FILE`), чтобы данные и вход пользователя переживали перезапуск.

НФТ-8. Качество кода. Все пакеты покрываются модульными тестами; в конвейере непрерывной интеграции выполняются статический анализ, проверка уязвимостей и сборка образа.

1.4.3 Требования безопасности

Анализ выполнен по классификации OWASP Top 10 [17]. Применимость и меры приведены в таблице 1.5.

Таблица 1.5 — Угрозы безопасности и меры защиты

Угроза	Применимость	Мера защиты
Открытое перенаправление (Unvalidated Redirects) [18]	Сервис по определению выполняет перенаправление на адрес, заданный пользователем, и может использоваться для маскировки вредоносных адресов	Допускаются только схемы `http`/`https`; запрещены ссылки на сам сервис (исключаются цепочки редиректов); возможность деактивации ссылки; регистрация владельца каждой ссылки; ограничение частоты создания
SSRF (Server-Side Request Forgery)	Не применима: сервис не выполняет запросов по пользовательским адресам, не загружает страницы и не получает превью	Архитектурное решение: адрес только сохраняется и возвращается клиенту в заголовке `Location`
Нарушение контроля доступа (Broken Access Control)	Применима: владелец не должен видеть и менять чужие ссылки и статистику	Все операции выполняются с проверкой `owner_id`; чужая ссылка неотличима от отсутствующей (`404`)

Угроза	Применимость	Мера защиты
Подбор паролей	Применима к `/auth/login`	Rate limiting по IP на регистрацию и вход (`/auth/register`, `/auth/login`); хэширование bcrypt с параметром стоимости 12; единое сообщение об ошибке для неверного email и неверного пароля; при неизвестном email выполняется сравнение с фиктивным хэшем, чтобы время ответа не выдавало существование учётной записи
Хранение паролей, криптографические сбои	Применимо	Пароли хранятся только в виде bcrypt-хэшей; токены подписываются HMAC-SHA256 с секретом из окружения; при использовании PostgreSQL секрет обязателен
Внедрение (SQL-инъекции)	Применимо	Параметризованные запросы драйвера pgx; отсутствие конкатенации SQL
XSS в дашборде	Применима: в интерфейсе выводятся названия и адреса, введённые пользователями	Весь пользовательский текст выводится средствами DOM (`textContent`, `createElement`); `innerHTML` не применяется; заголовки безопасности (`Content-Security-Policy` с `script-src 'self'`, `X-Content-Type-Options`, `X-Frame-Options`, `Referrer-Policy`)
Отказ в обслуживании	Применим	Rate limiting на создание ссылок, регистрацию и вход (token bucket по IP; по умолчанию 5 запросов/с, всплеск 20); ограничение размера тела запроса 64 КБ; ограниченная очередь аналитики; для лимитирования самого редиректа на уровне сети предполагается обратный прокси

Угроза	Применимость	Мера защиты
Подделка IP через `X-Forwarded-For`	Применима к аналитике и rate limiting	Заголовки `X-Forwarded-For` и `CF-IPCountry` учитываются только при `TRUST_PROXY=true`
Уязвимые зависимости	Применима	`govulncheck`, `gosec`, Dependabot в конвейере CI
Раскрытие персональных данных	Применимо	Обезличивание: IP не хранится (см. 1.1.4)

Следует отметить, что выбранные меры не обеспечивают защиту от размещения ссылок на фишинговые ресурсы; для промышленной эксплуатации необходимо интегрировать проверку адресов по базам вредоносных ресурсов, что выходит за пределы настоящей работы.

1.5 Выводы по главе 1

1. Проанализирован принцип работы сервиса сокращения ссылок. Показано, что для учёта каждого перехода и возможности отключения ссылки следует применять временное перенаправление 302 с запретом кэширования ответа клиентом.

2. Определены источники аналитики (Referer, User-Agent, IP-адрес, время) и принципы обезличивания: IP-адрес не сохраняется, уникальные посетители определяются по солёному хэшу, реферер сводится к домену.

3. Выполнен обзор семи аналогов; установлено, что коммерческие сервисы функциональны, но закрыты и хранят данные у третьей стороны, а открытые решения используют иные стеки; обоснована целесообразность собственной реализации на Go, PostgreSQL и Redis.

4. Для генерации кодов выбран случайный base62-код длиной 7 символов (ёмкость $\approx 3{,}5 \cdot 10^{12}$); показано, что вероятность коллизии при вставке в таблицу из 10^7 записей составляет около $2{,}8 \cdot 10^{-6}$, что решается ограничением уникальности и повторной генерацией.

5. Сформулированы 12 функциональных и 8 нефункциональных требований, проведён анализ угроз по OWASP; эти требования являются исходными данными для проектирования во второй главе.

2 ПРОЕКТНАЯ ЧАСТЬ

2.1 Архитектура системы

Система построена по слоистой архитектуре с однонаправленными зависимостями [4, 6]: транспортный слой (HTTP) обращается к слою бизнес-логики (сервису), а тот — к абстракциям хранилища, кэша и сборщика аналитики. Слои взаимодействуют через интерфейсы, определённые в потребляющих пакетах, поэтому конкретные реализации (PostgreSQL, Redis или реализации в памяти) подменяются без изменения бизнес-логики. Для хранилища предусмотрены три режима работы: PostgreSQL (промышленная эксплуатация), память (тесты и быстрый запуск) и память со снимком состояния в файле (демонстрация); они описаны в разделе 2.8. Структура пакетов приведена в таблице 2.1.

Таблица 2.1 — Пакеты системы

Слой	Пакет	Назначение
Транспорт	`internal/httpapi`	HTTP-обработчики, middleware (request-id, recover, access-log, ограничение тела, rate limiting, заголовки безопасности), раздача дашборда
Бизнес-логика	`internal/service`	`LinkService`: создание, разрешение, удаление, статистика
Бизнес-логика	`internal/auth`	регистрация, вход, bcrypt, HMAC-токены
Бизнес-логика	`internal/qrcode`	формирование PNG с QR-кодом короткой ссылки (кодирование матрицы — `rsc.io/qr`, отрисовка в 1-битный PNG — собственная)
Бизнес-логика	`internal/shortcode`	генерация кодов base62, валидация алиасов
Данные	`internal/storage`, `storage/postgres`, `storage/memory`	интерфейсы хранилища и реализации; `memory.Persistent` добавляет к хранилищу в памяти сохранение снимка в файл
Данные	`internal/cache` (`RedisCache`, `MemoryCache`)	кэш горячих ссылок и счётчики популярности
Аналитика	`internal/analytics`	`Collector`: очередь, воркеры, обогащение, пакетная запись; парсинг User-Agent, Referer, гео

Слой	Пакет	Назначение
Модель	`internal/model`	доменные сущности
Инфраструктура	`internal/config`, `cmd/shortener`	конфигурация из окружения, сборка зависимостей, graceful shutdown
Представление	`web/`	дашборд (HTML и JavaScript, встраивается в бинарный файл через `embed`)

Ключевая особенность — **асинхронный конвейер аналитики**: обработчик редиректа не ждёт записи статистики, а помещает событие в буферизованный канал, из которого воркеры пакетно записывают данные в PostgreSQL. Благодаря этому задержка редиректа не зависит от скорости записи в базу.

На рисунке 2.1 показана диаграмма контейнеров (C4, уровень Container).

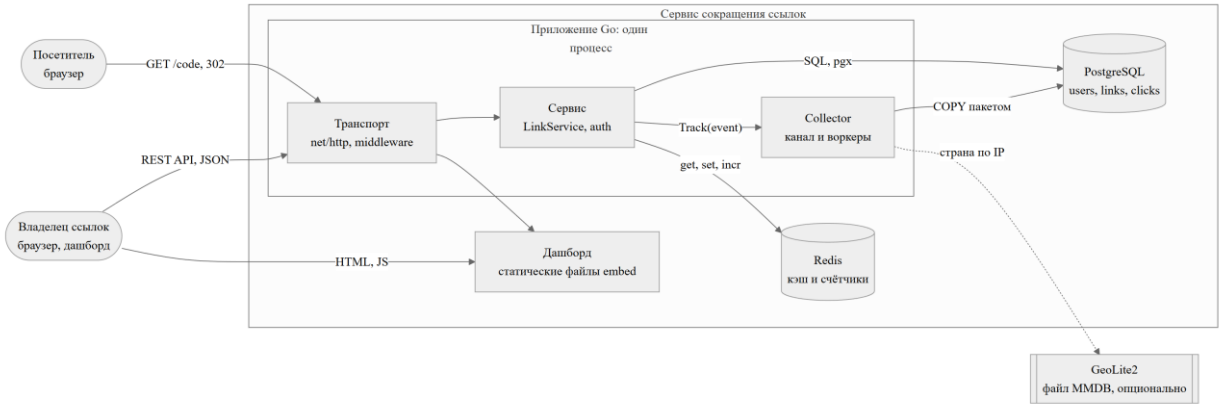


Рисунок 2.1 — Диаграмма контейнеров системы (C4, уровень Container)

2.2 Диаграмма вариантов использования

Выделены три роли: **Посетитель** (любой, кто открывает короткую ссылку), **Владелец ссылок** (зарегистрированный пользователь дашборда) и **Администратор / DevOps** (развёртывание и сопровождение). Вариант использования «Получить QR-код ссылки» (UC14) доступен без входа в систему, так как QR-код содержит только саму короткую ссылку, которая и так публична; на практике им пользуется владелец ссылки из дашборда. Диаграмма вариантов использования показана на рисунке 2.2.

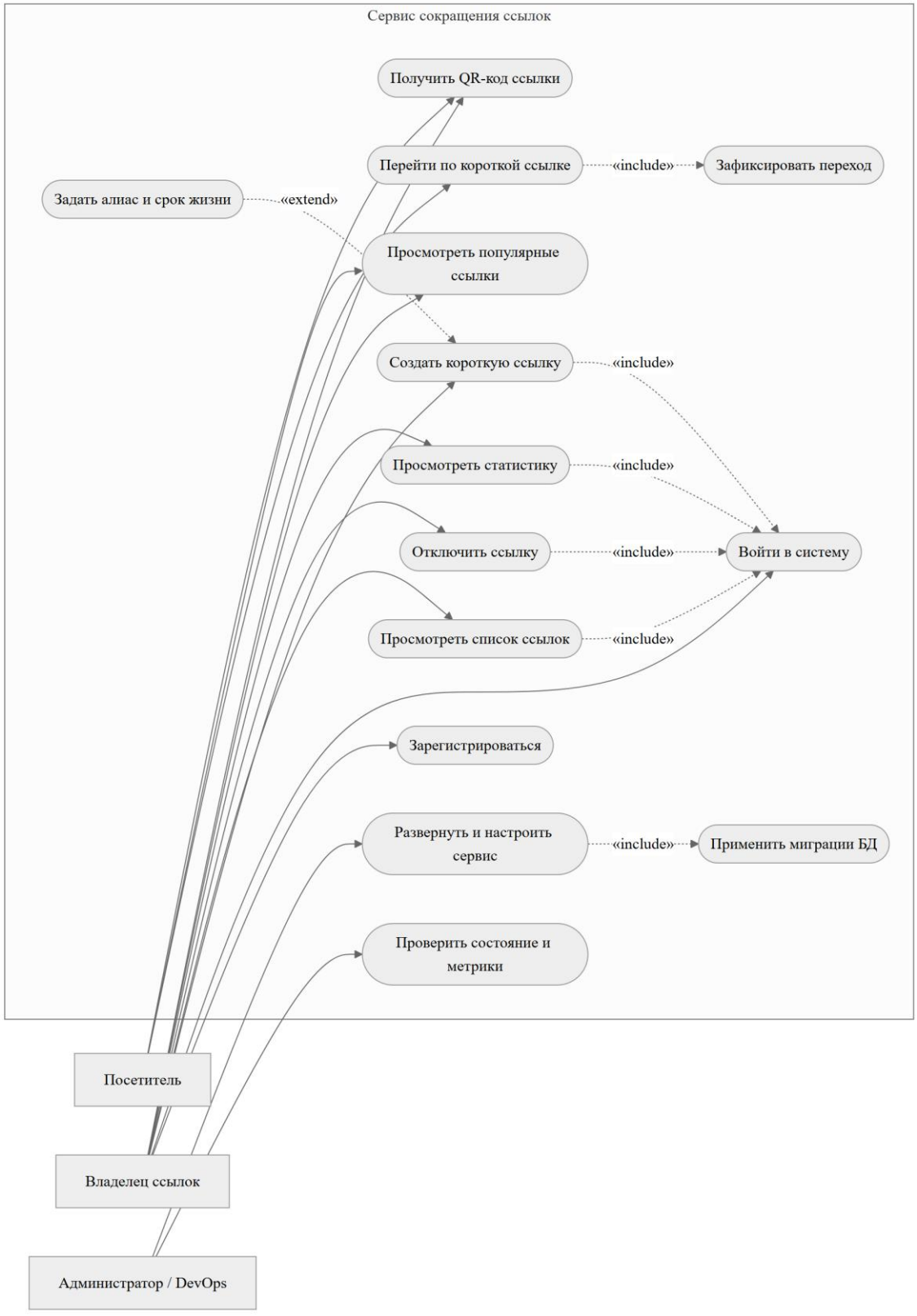


Рисунок 2.2 — Диаграмма вариантов использования

2.3 Диаграммы последовательности

2.3.1 Создание ссылки

Запрос создания ссылки (`POST /api/v1/links`) проходит цепочку middleware (rate limiting, ограничение тела), проверку токена, нормализацию адреса и генерацию кода. При использовании случайного кода уникальность проверяется ограничением БД, а при конфликте выполняется повторная попытка (рисунок 2.3).

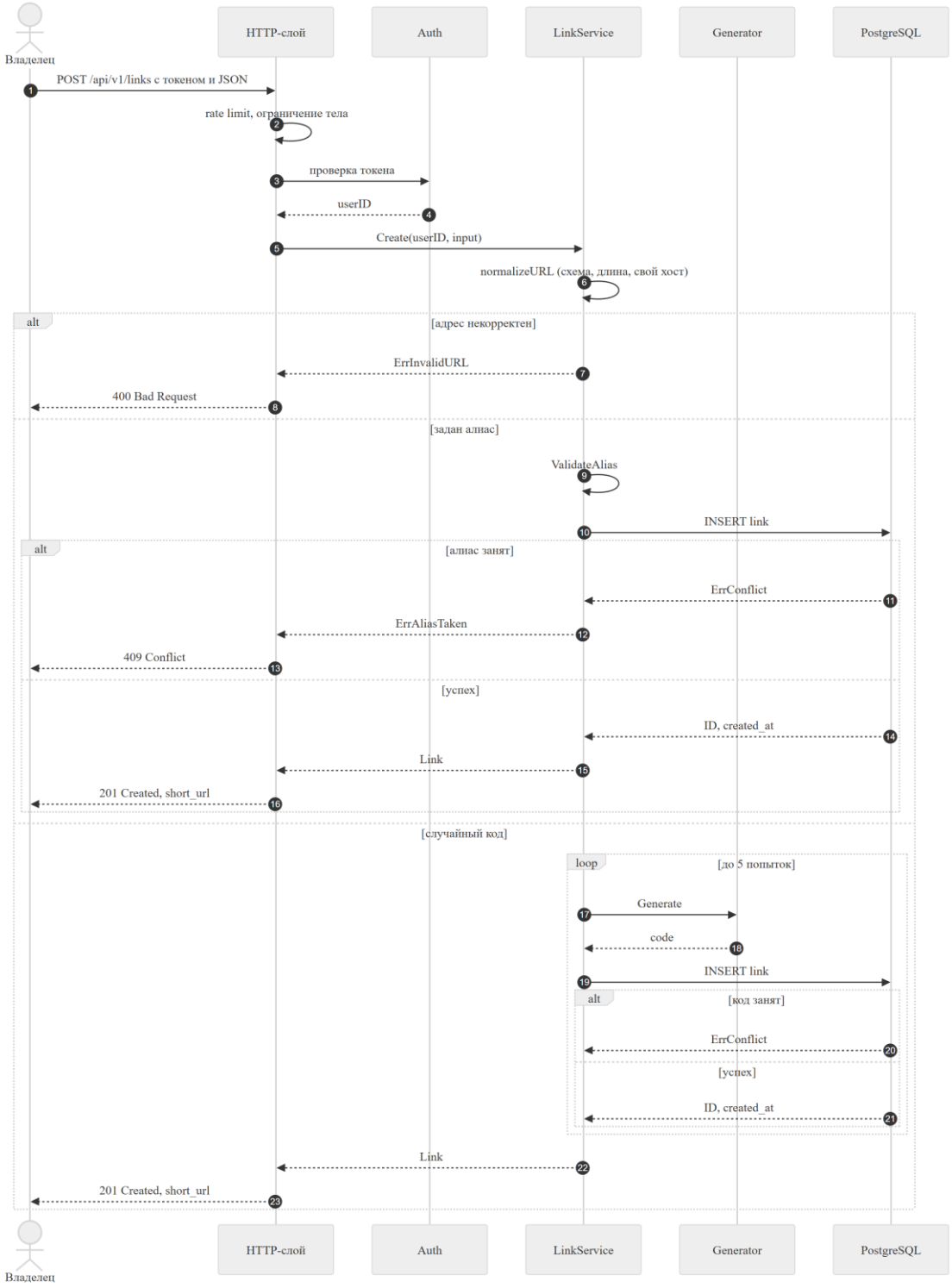


Рисунок 2.3 — Последовательность создания короткой ссылки

2.3.2 Редирект с кэшем и постановкой события в очередь

Основной сценарий системы — разрешение ссылки (рисунок 2.4). Сначала выполняется поиск в Redis. При попадании запрос к БД не требуется. При промахе (или недоступности Redis) ссылка читается из PostgreSQL,

увеличивается счётчик обращений в окне, и при достижении порога популярности ссылка помещается в кэш. В любом случае событие перехода неблокирующим образом ставится в очередь аналитики.

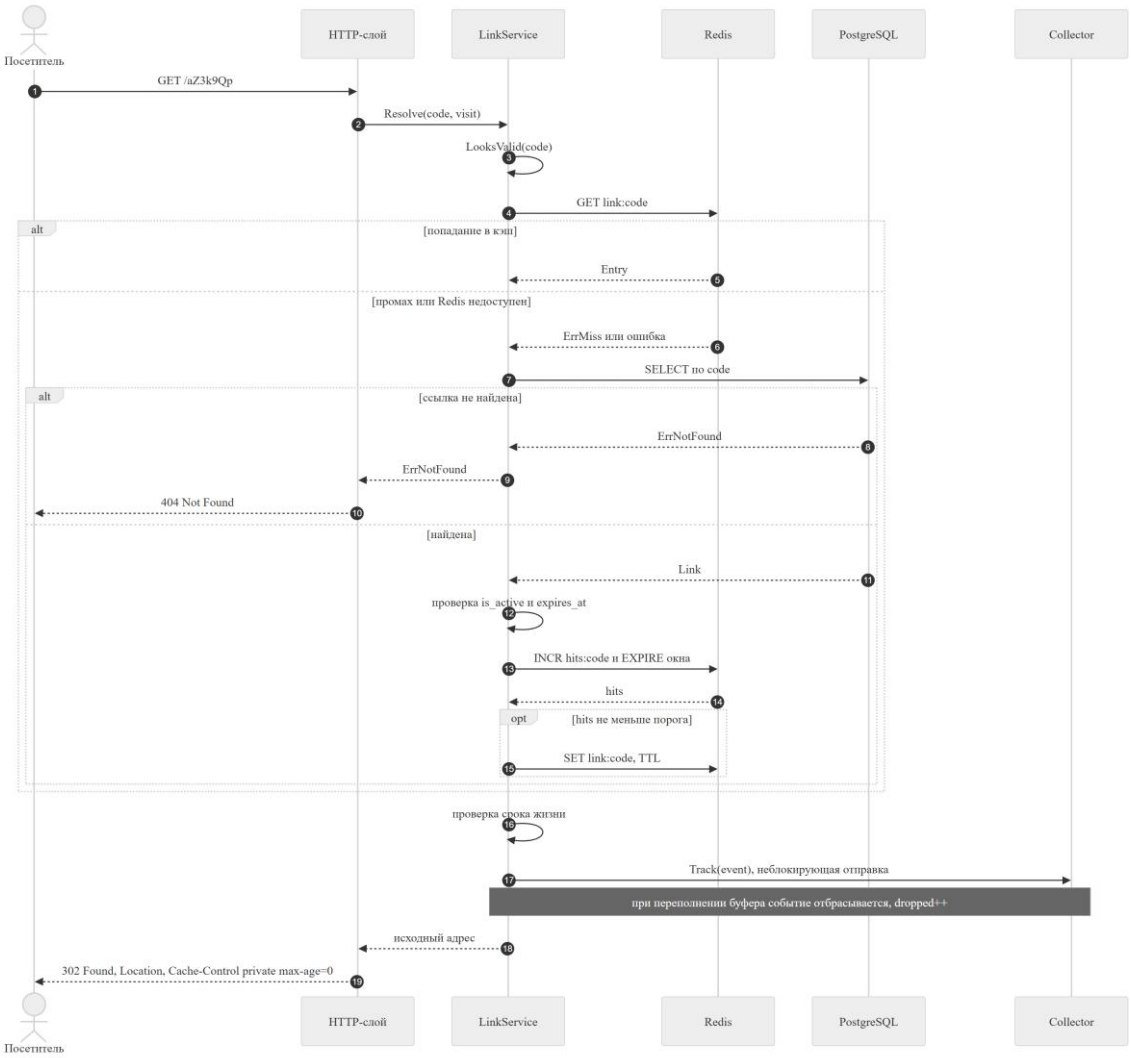


Рисунок 2.4 — Последовательность редиректа с кэшированием и постановкой события в очередь

2.3.3 Сбор аналитики

Сбор статистики вынесен в фоновый конвейер (рисунок 2.5). N воркеров читают события из общего канала, обогащают их и накапливают пакет. Пакет записывается при достижении размера `ANALYTICS_BATCH` либо по таймеру `ANALYTICS_FLUSH`. При остановке приложения канал дочитывается, и накопленные данные сбрасываются.

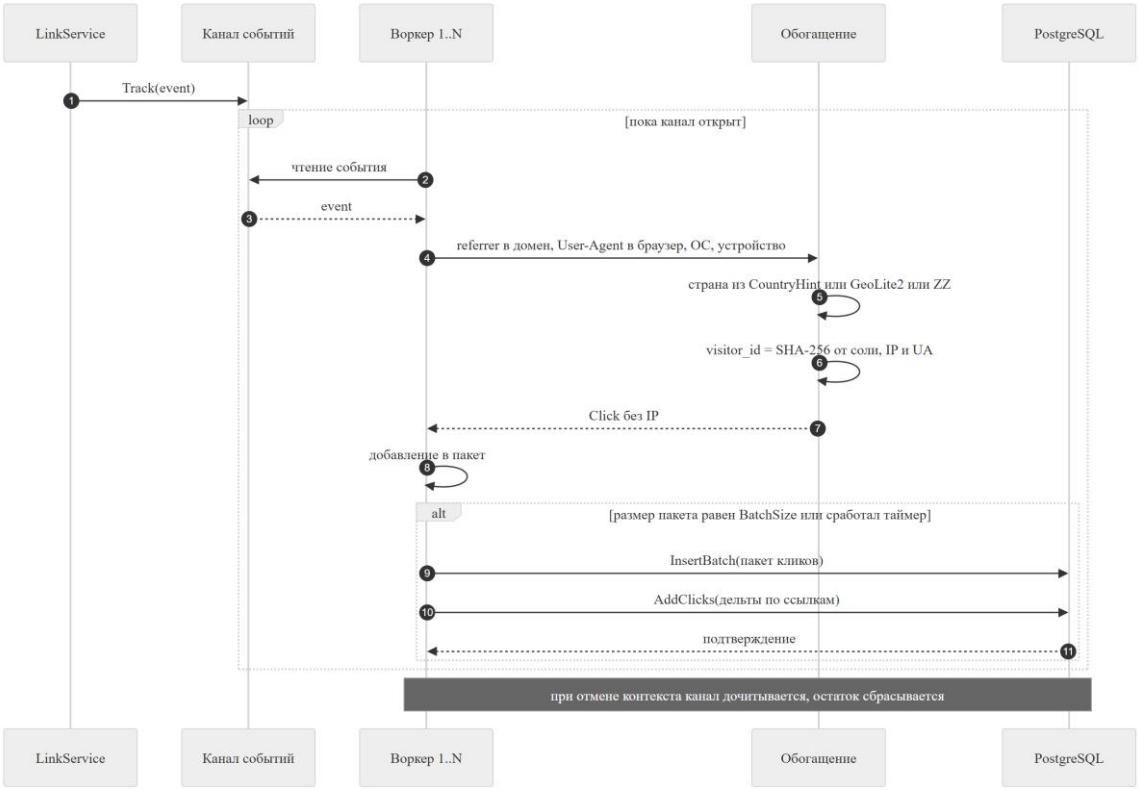


Рисунок 2.5 — Последовательность сбора и пакетной записи аналитики

2.4 Модель данных

Данные хранятся в PostgreSQL в четырёх таблицах (рисунок 2.6): `users`, `links`, `clicks` и служебной `schema_migrations`, в которой фиксируются применённые миграции. Таблицы `users`, `links` и `clicks` создаёт SQL-файл `001_init.sql`, а `schema_migrations` — код применения миграций (`Migrate`) перед выполнением файлов, поэтому её описания в SQL-миграции нет.

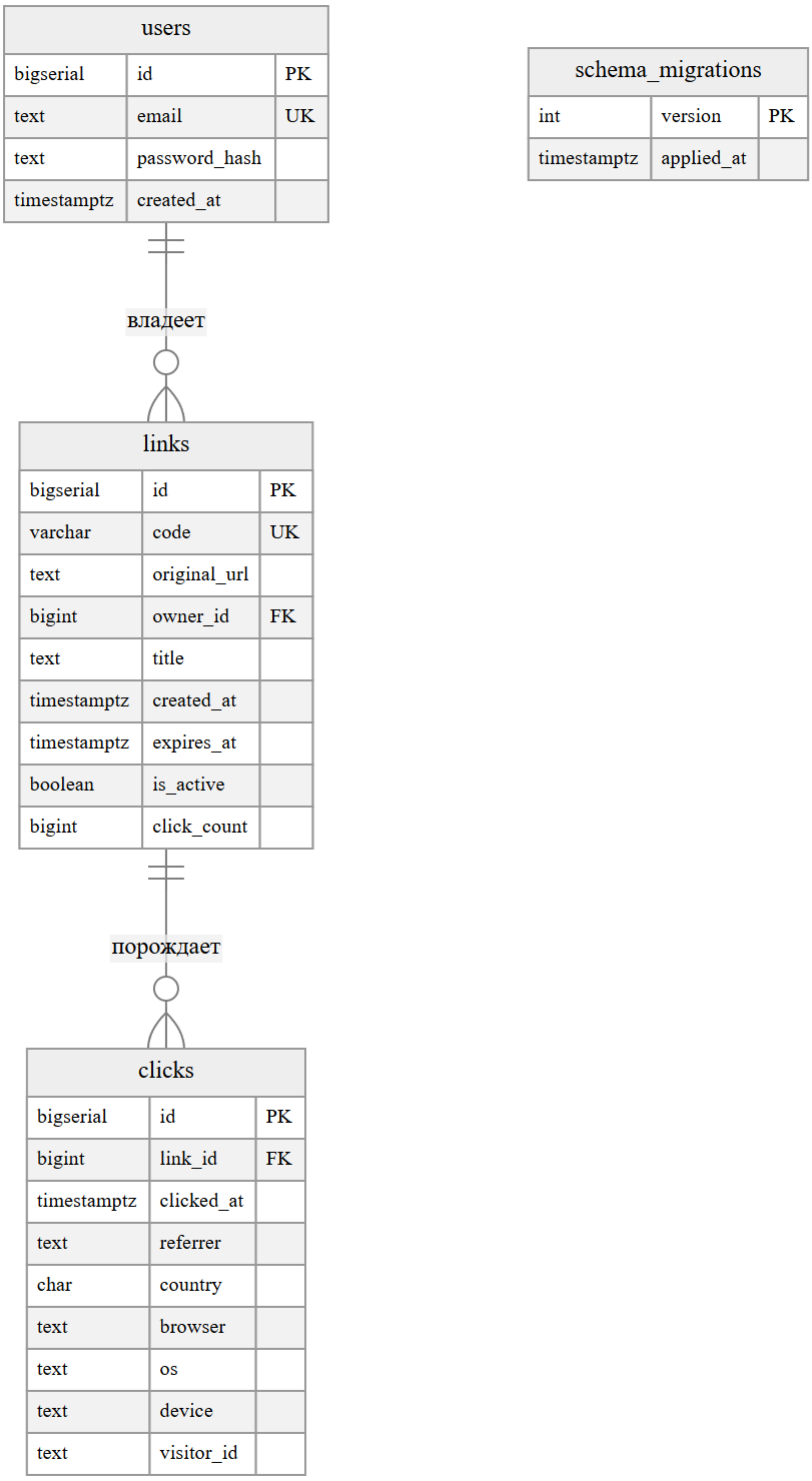


Рисунок 2.6 — ER-диаграмма базы данных

Описание таблиц приведено в таблице 2.2.

Таблица 2.2 — Структура таблиц базы данных

Таблица	Поле	Тип	Ограничения / назначение
users	id	BIGSERIAL	первичный ключ
	email	TEXT	NOT NULL, UNIQUE; идентификатор для входа
	password_hash	TEXT	NOT NULL; bcrypt-хэш
	created_at	TIMESTAMPTZ	NOT NULL, по умолчанию now()
links	id	BIGSERIAL	первичный ключ
	code	VARCHAR(32)	NOT NULL, UNIQUE; короткий код или алиас
	original_url	TEXT	NOT NULL; адрес назначения
	owner_id	BIGINT	внешний ключ на users(id), ON DELETE CASCADE
	title	TEXT	NOT NULL, по умолчанию пустая строка
	created_at	TIMESTAMPTZ	NOT NULL, по умолчанию now()
	expires_at	TIMESTAMPTZ	NULL — бессрочная ссылка
	is_active	BOOLEAN	NOT NULL, по умолчанию true; признак «мягкого» удаления
	click_count	BIGINT	NOT NULL, по умолчанию 0; денормализованный счётчик
clicks	id	BIGSERIAL	первичный ключ
	link_id	BIGINT	NOT NULL, внешний ключ на links(id), ON DELETE CASCADE
	clicked_at	TIMESTAMPTZ	NOT NULL; время перехода
	referrer	TEXT	домен источника или direct
	country	CHAR(2)	код ISO 3166-1 alpha-2 или ZZ
	browser, os, device	TEXT	классифицированные значения User-Agent
	visitor_id	TEXT	обезличенный хэш для подсчёта уникальных посетителей
schema_migrations	version	INT	первичный ключ; номер применённой миграции
	applied_at	TIMESTAMPTZ	время применения

Индексы и их обоснование.

– `links.code` — уникальный индекс, создаваемый ограничением `UNIQUE`. Обслуживает самую частую операцию (поиск по коду при редиректе) и одновременно гарантирует уникальность кодов, необходимую для обработки коллизий (раздел 1.3.3).

– `idx_links_owner_created (owner_id, created_at DESC)` — составной индекс для выдачи списка ссылок владельца в порядке «новые первыми» без отдельной сортировки.

– `idx_clicks_link_time (link_id, clicked_at)` — основной индекс аналитических запросов: статистика всегда считается по одной ссылке за временной интервал, поэтому условие `link_id = ? AND clicked_at >= ? AND clicked_at < ?` выполняется диапазонным сканированием индекса.

– `users.email` — уникальный индекс для входа по email и контроля уникальности.

Денормализация `click_count`. Счётчик переходов хранится в таблице `links` для быстрого отображения в списке без агрегирования миллионов строк `clicks`. Он увеличивается агрегированными дельтами: после вставки пакета выполняется один запрос `UPDATE ... FROM unnest(...)`, поэтому число обновлений строк `links` пропорционально числу пакетов и числу затронутых ссылок, а не числу переходов. Вставка пакета и обновление счётчиков выполняются отдельными запросами без общей транзакции; при сбое второго запроса ошибка фиксируется в журнале, а значение `click_count` может отставать от числа строк `clicks`.

Каскадное удаление. Удаление пользователя удаляет его ссылки, а удаление ссылки — её переходы, что исключает «осиротевшие» записи. Операция удаления в API реализована как деактивация (`is_active = false`), история статистики при этом сохраняется.

Типы данных. Время хранится с часовым поясом (`TIMESTAMPTZ`), что исключает ошибки при сравнении моментов. Страна хранится как `CHAR(2)` с кодом `ZZ` для неизвестного значения. Идентификатор посетителя представлен

хэшем, а не IP-адресом, что соответствует принципу минимизации данных (раздел 1.1.4).

Масштабирование таблицы `clicks`, растущей быстрее остальных, возможно секционированием по времени (`PARTITION BY RANGE (clicked_at)`) и/или предварительной агрегацией по дням; в текущей версии эти меры не реализованы и рассматриваются как направление развития.

2.5 Проектирование REST API

Программный интерфейс построен по принципам REST поверх HTTP с обменом в формате JSON. Версия API включена в путь (`/api/v1`). Аутентификация — токен в заголовке `Authorization: Bearer <token>`. Перечень конечных точек приведён в таблице 2.3.

Таблица 2.3 — Конечные точки REST API

Метод	Путь	Описание	Аутентификация	Усп с
POST	`/api/v1/auth/register`	Регистрация: `{email, password}`	нет	201 {to use
POST	`/api/v1/auth/login`	Вход: `{email, password}`	нет	200 {to use
GET	`/api/v1/me`	Идентификатор текущего пользователя	да	200
POST	`/api/v1/links`	Создание ссылки (`url`, `alias`, `title`, `expires_at`)	да	201 + sho заго Loc
GET	`/api/v1/links?limit&offset`	Список ссылок владельца	да	200 {li [li
GET	`/api/v1/links/{code}`	Одна ссылка	да	200
DELETE	`/api/v1/links/{code}`	Деактивация ссылки	да	204
GET	`/api/v1/links/{code}/stats?from&to`	Статистика за период (по умолчанию 30 дней; `to` включительно)	да	200 Lin

Метод	Путь	Описание	Аутентификация	Усп с
GET	`/api/v1/top?n=10`	Популярные коды	нет	200 {co [co
GET	`/qr/{code}.png?size=N`	QR-код короткой ссылки (`size` 64–1024, по умолчанию 256)	нет	200 ima Cac Con pub max age
GET	`/{code}`	Редирект	нет	302 Loc
GET	`/healthz`	Проверка БД и кэша: `{status, checks}`	нет	200
GET	`/metrics`	Метрики в текстовом формате Prometheus	нет	200
GET	`/dashboard/`	Дашборд	нет	200
GET	`/`	Перенаправление на дашборд	нет	302

Ошибки возвращаются в едином формате `{"error": "описание"}`. Соответствие внутренних ошибок сервиса кодам ответа приведено в таблице 2.4.

Таблица 2.4 — Соответствие ошибок и кодов HTTP

Ошибка сервиса	Код HTTP	Смысл
`ErrInvalidURL`, `ErrBadAlias`, `ErrBadPeriod`, параметр `size` вне диапазона 64–1024	400 Bad Request	некорректные входные данные
отсутствует или недействителен токен	401 Unauthorized	требуется аутентификация
`ErrNotFound`	404 Not Found	ссылка не найдена или принадлежит другому пользователю
`ErrAliasTaken`, занятый email	409 Conflict	нарушение уникальности
`ErrGone`	410 Gone	срок действия истёк или ссылка отключена
превышен лимит запросов	429 Too Many Requests	сработал rate limiting

Ошибка сервиса	Код HTTP	Смысл
тело запроса больше 64 КБ	413 Request Entity Too Large	превышен предел размера тела
прочие ошибки	500 Internal Server Error	внутренняя ошибка

Решение возвращать `404` на запрос чужой ссылки (вместо `403`) сделано намеренно: сервис не раскрывает факт существования ссылки другого пользователя. Конечная точка `/qr/{code}.png` публична: в QR-код записывается только короткая ссылка, раскрывать которую и так предполагается; запрос не считается переходом и не попадает в статистику. Для публичных конечных точек (`/top`) не раскрываются данные, позволяющие идентифицировать владельцев.

2.6 Диаграмма классов и интерфейсов

Доменная логика опирается на небольшие интерфейсы (рисунок 2.7). Интерфейс `Store` объединяет три репозитория; бизнес-логика получает его целиком, а `Collector` — только `LinkRepository` и `ClickRepository`, что соответствует принципу разделения интерфейсов. `LinkService` зависит исключительно от абстракций; реализации подставляются в `cmd/shortener`.

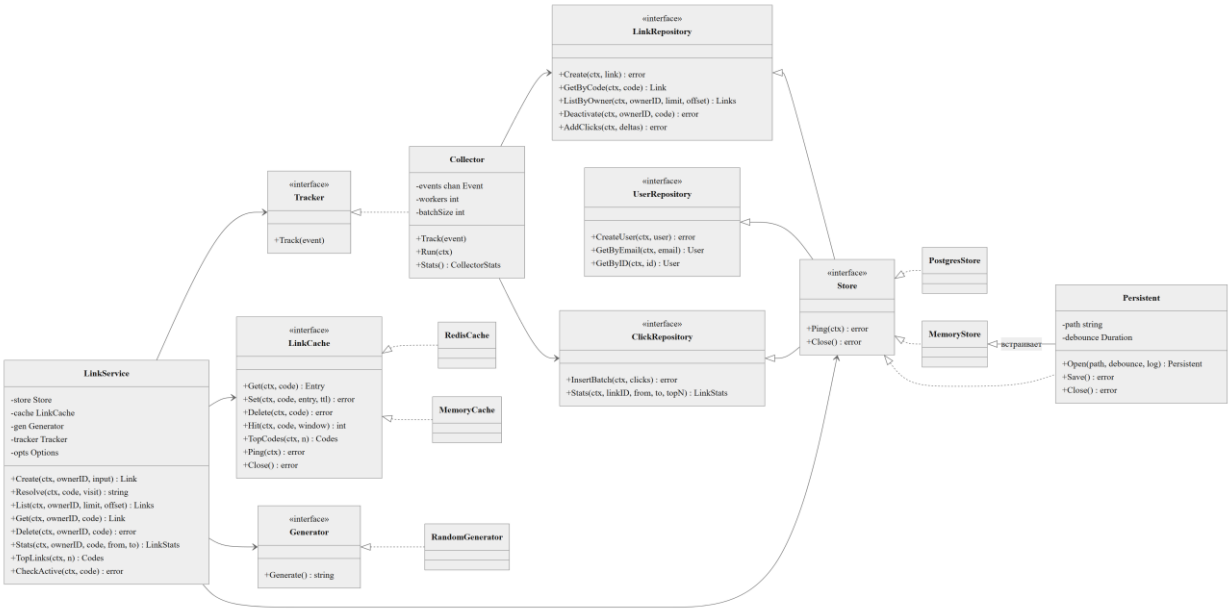


Рисунок 2.7 — Диаграмма интерфейсов и классов основных пакетов

Схема встраивания интерфейсов в `Store` отражает идиому Go: интерфейс определяется набором методов, а реализация удовлетворяет ему неявно, без объявления «реализует». Класс `Persistent` встраивает `MemoryStore`, наследуя все методы репозиториев, и добавляет только жизненный цикл — загрузку снимка при открытии (`Open`), запись (`Save`) и итоговое сохранение при закрытии (`Close`). Благодаря этому тесты `LinkService` выполняются с реализациями `MemoryStore` и `MemoryCache` без внешних сервисов, а `Collector` проверяется с подставным репозиторием [1, 10].

2.7 Стратегия кэширования

2.7.1 Что и зачем кэшируется

Кэшируются данные, необходимые для редиректа без обращения к БД: идентификатор ссылки, адрес назначения и срок окончания (структура `Entry`). Ключ — `link:{code}`. Данные в Redis хранятся в сериализованном виде, а срок жизни записи ограничивается как настроенным временем жизни, так и оставшимся временем жизни самой ссылки, чтобы просроченная ссылка не отдавалась из кэша.

2.7.2 Политика попадания в кэш (порог популярности)

Кэширование всех подряд ссылок неэффективно: большинство ссылок открывается единожды или редко и только вытесняло бы действительно популярные. Поэтому применяется политика **«кэшировать после порога»**. При каждом промахе увеличивается счётчик `hits:{code}` (команда `INCR`, при создании ключа устанавливается `EXPIRE` на время окна). Окно фиксированное, а не скользящее: оно открывается первым обращением (`INCR` вернул 1 — выставляется `EXPIRE`) и не продлевается последующими. Если значение счётчика достигло порога `PopularThreshold` в течение окна `HitWindow`, запись помещается в кэш на время `CacheTTL`. Значения по умолчанию приведены в таблице 2.5. Отсчёт окна начинается с первого обращения; по

истечении окна счётчик исчезает, поэтому редкие обращения никогда не накапливают значение, достаточное для кэширования.

Таблица 2.5 — Параметры кэширования по умолчанию

Параметр	Значение по умолчанию	Назначение
`PopularThreshold`	3 обращения	порог, после которого ссылка считается популярной
`HitWindow`	10 минут	окно подсчёта обращений
`CacheTTL`	1 час	максимальное время жизни записи в кэше
`StatsTopN`	10	число элементов в рейтингах статистики

Решение о кэшировании принимается на пути промаха, поэтому для уже закэшированных ссылок дополнительная нагрузка отсутствует. Недостаток подхода: до достижения порога первые обращения обслуживаются из БД; для редиректов, идущих из первых секунд массовой рассылки, это приводит к кратковременному всплеску запросов к БД, который ограничивается малым значением порога. Решение принятия кэш-решения показано на рисунке 2.8.

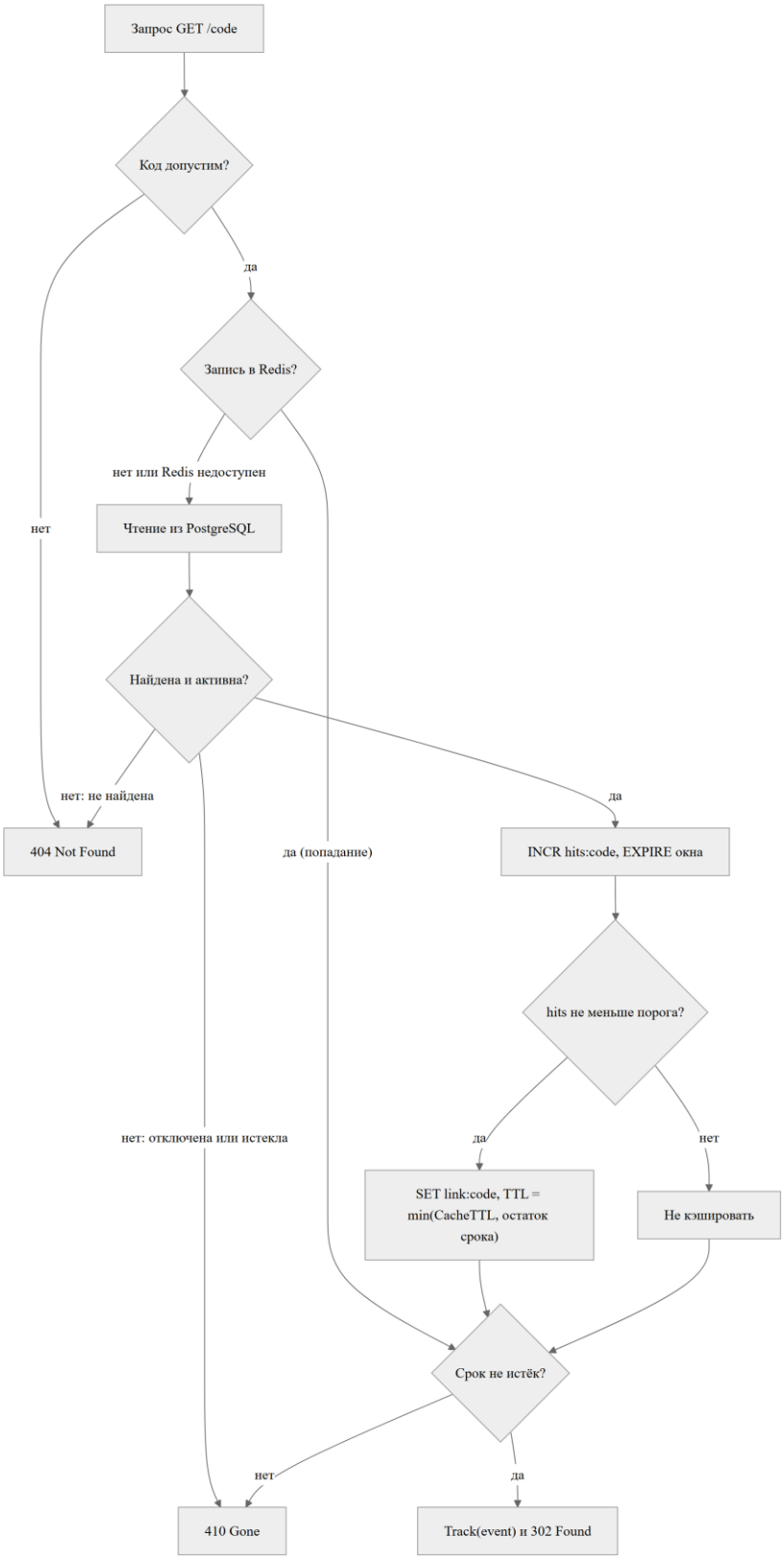


Рисунок 2.8 — Алгоритм принятия решения о кэшировании

2.7.3 Деградация при отказе Redis

Redis рассматривается как ускоритель, а не как источник истины. Любая ошибка при обращении к кэшу (кроме обычного промаха `ErrMiss`) логируется на уровне предупреждения и не прерывает обработку запроса: сервис читает ссылку из PostgreSQL. Ошибки записи в кэш и счётчика популярности также не влияют на ответ. Эндпоинт `/healthz` отражает состояние обоих компонентов, что позволяет оркестратору и мониторингу обнаружить деградацию. Платой за доступность является увеличение нагрузки на БД и рост задержки при отказе кэша; индекс по `links.code` обеспечивает приемлемую скорость точечного чтения.

2.7.4 Вытеснение и инвалидация

Redis в конфигурации развёртывания запускается с ограничением памяти 128 МБ и политикой вытеснения `allkeys-lru`: при нехватке памяти удаляются наименее давно использованные ключи, независимо от наличия у них TTL. Это согласуется с политикой популярности: редко запрашиваемые записи вытесняются первыми, а ключи-счётчики также могут быть удалены без ущерба для корректности (счётчик начнётся заново). При деактивации ссылки запись немедленно удаляется из кэша командой `DEL` (инвалидация), чтобы отключённая ссылка перестала работать без ожидания окончания TTL.

2.7.5 Рейтинг популярных ссылок

Для публичного рейтинга (`/top`) используется отсортированное множество Redis (sorted set), в котором счёт кода увеличивается командой `ZINCRBY` в той же транзакции Redis, что и счётчик окна. Метод `TopCodes` возвращает до n кодов с наибольшим счётом [7, 12]. Поскольку счётчики обновляются только на пути промаха кэша (раздел 2.7.2), обращения к уже закэшированным ссылкам в рейтинге не учитываются, поэтому он отражает активность до попадания ссылки в кэш и после истечения записи, то есть

является приближённым: он не заменяет точную статистику из PostgreSQL, но вычисляется за время O(log N + n) без нагрузки на основную базу.

2.8 Развёртывание

Система поставляется в виде многоступенчатого Docker-образа и файла Docker Compose. **Образ** собирается в два этапа: на этапе сборки (`golang:1.27-alpine`) копируются манифесты модулей и скачиваются зависимости (слой кэшируется), затем выполняется сборка статического бинарного файла (`CGO_ENABLED=0`, `-trimpath`, `-ldflags="-s -w"`); на этапе исполнения используется минимальный образ `distroless/static` с непривилегированным пользователем `nonroot`. Результирующий образ содержит только исполняемый файл, без оболочки и пакетного менеджера, что уменьшает размер и поверхность атаки [15].

Файл Compose описывает три сервиса: `app` (приложение, порт 8080), `postgres` (PostgreSQL 17 Alpine с томом `pgdata`) и `redis` (Redis 7 Alpine, `maxmemory 128mb`, политика `allkeys-lru`). Для `app` задана политика перезапуска `unless-stopped`; для `postgres` и `redis` определены проверки работоспособности (`pg_isready`, `redis-cli ping`), а приложение запускается только после их успешного прохождения (`depends_on` с условием `service_healthy`). Секрет подписи токенов `AUTH_SECRET` не хранится в репозитории: он обязателен и передаётся из файла окружения `.env` (шаблон — `.env.example`). Схема развёртывания приведена на рисунке 2.9.

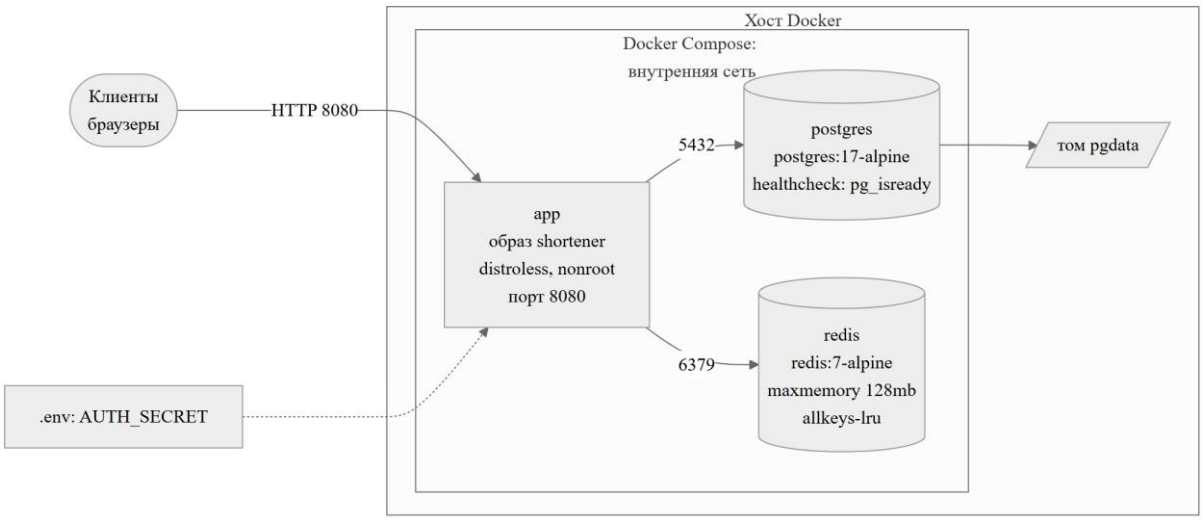


Рисунок 2.9 — Диаграмма развёртывания Docker Compose

Режимы хранения. Конфигурация приложения задаётся переменными окружения (таблица 2.6). Если `DATABASE_URL` или `REDIS_URL` не заданы, используются реализации в памяти, что упрощает локальный запуск и тестирование. Для хранилища определены три режима:

1. *PostgreSQL* (задан `DATABASE_URL`) — основной режим для промышленной эксплуатации: данные надёжно хранятся в базе, секрет `AUTH_SECRET` обязателен.

2. *Память* (не заданы ни `DATABASE_URL`, ни `DATA_FILE`) — реализация `memory.Store`; данные существуют только до остановки процесса. Режим нужен для модульных тестов и быстрого запуска.

3. *Память со снимком в файле* (задан `DATA_FILE`, `DATABASE_URL` пуст) — реализация `memory.Persistent`: при запуске состояние загружается из JSON-файла, а после изменений и при остановке записывается обратно. Секрет подписи токенов при пустом `AUTH_SECRET` хранится в файле `<DATA_FILE>.secret`, поэтому токены остаются действительными после перезапуска.

Одновременное задание `DATA_FILE` и `DATABASE_URL` считается ошибкой конфигурации: два источника истины для одних и тех же данных привели бы к расхождению. Третий режим появился потому, что чисто «временное»

хранение делает демонстрацию неудобной: после каждого закрытия окна пропадают учётные записи, ссылки и статистика, а регистрация заново обесценивает подготовленные примеры. Установка PostgreSQL ради демонстрации на ноутбуке нежелательна — это тяжёлая зависимость, а демонстрационный объём данных мал. Файл со снимком — компромисс, не усложняющий запуск: он не заменяет базу данных, не рассчитан на несколько процессов и большие объёмы и поэтому предназначен только для демонстрации. Реализация описана в разделе 4.13.

Таблица 2.6 — Основные параметры конфигурации

Переменная	Значение по умолчанию	Назначение
HTTP_ADDR	:8080	адрес прослушивания
BASE_URL	http://localhost:8080	базовый адрес для формирования коротких ссылок
DATABASE_URL	пусто (память)	строка подключения PostgreSQL
DATA_FILE	пусто	путь к JSON-файлу снимка данных демо-режима (только без DATABASE_URL)
REDIS_URL	пусто (память)	строка подключения Redis
AUTH_SECRET	— (обязателен с PostgreSQL)	секрет подписи токенов; с DATA_FILE берётся из файла <DATA_FILE>.secret либо создаётся в нём
TOKEN_TTL	72h	срок действия токена
GEOIP_DB	пусто	путь к базе GeoLite2
TRUST_PROXY	false	доверять заголовкам X-Forwarded-For и CF-IPCountry
CODE_LENGTH	7	длина генерируемого кода
ANALYTICS_WORKERS	4	число воркеров аналитики
ANALYTICS_BUFFER	10000	размер буфера канала событий
ANALYTICS_BATCH	500	размер пакета записи
ANALYTICS_FLUSH	1s	интервал принудительного сброса пакета
RATE_LIMIT_RPS	5	допустимая частота запросов с одного IP
RATE_LIMIT_BURST	20	размер «всплеска» token bucket
LOG_LEVEL	info	уровень журналирования

2.9 Выводы по главе 2

1. Спроектирована слоистая архитектура с изоляцией внешних зависимостей за интерфейсами и асинхронным конвейером аналитики, не включающим запись статистики в критический путь редиректа.

2. Построены диаграммы контейнеров, вариантов использования, последовательности для трёх ключевых сценариев, классов и развёртывания.

3. Спроектирована реляционная модель из трёх основных таблиц с индексами, обеспечивающими поиск по коду, выдачу списка ссылок владельца и диапазонные аналитические запросы.

4. Определён REST API (конечные точки, форматы ошибок, коды ответов) с проверкой владельца на каждой операции.

5. Разработана стратегия кэширования по порогу популярности с ограничением времени жизни, инвалидацией при отключении ссылки, вытеснением `allkeys-lru` и корректной деградацией до чтения из БД при отказе Redis.

6. Определены три режима хранения (PostgreSQL, память, память со снимком в файле); последний обеспечивает сохранность данных демонстрационного режима без внешней базы данных.

3 ТЕХНОЛОГИЧЕСКАЯ ЧАСТЬ

3.1 Язык программирования Go

Для реализации выбран язык Go. Решающими стали следующие свойства [1, 10].

1. **Горутины и каналы.** Конвейер аналитики естественно выражается в модели взаимодействующих последовательных процессов: обработчик HTTP помещает событие в буферизованный канал, а пул горутин-воркеров читает его, обогащает и пакетно записывает. Горутина занимает единицы килобайт памяти, планировщик среды выполнения мультиплексирует их на потоки ОС, поэтому тысячи одновременных соединений не требуют пула потоков и асинхронных обратных вызовов. Неблокирующая отправка (`select` с веткой `default`) позволяет отбрасывать события при переполнении очереди, не задерживая редирект.

2. **Стандартная библиотека.** Пакет `net/http` содержит промышленный HTTP-сервер. Начиная с Go 1.22 мультиплексор поддерживает маршрутизацию по методу и шаблону пути (`GET /api/v1/links/{code}`) с извлечением параметров, что позволило обойтись без сторонних маршрутизаторов. Пакеты `log/slog` (структурированные журналы), `crypto/rand`, `crypto/hmac`, `crypto/sha256`, `embed` (встраивание дашборда в бинарный файл), `context` (отмена и сроки) также входят в стандартную поставку. Проект использует версию Go 1.27 (в `go.mod` указана 1.27.1); минимально необходимой для шаблонной маршрутизации является версия 1.22.

3. **Статическая сборка.** Результатом сборки является один исполняемый файл без зависимостей от среды выполнения, что позволяет использовать минимальный образ `distroless` и упрощает развёртывание.

4. **Производительность.** Компилируемый код с автоматическим управлением памятью и сравнительно короткими паузами сборщика мусора

обеспечивает высокую пропускную способность для задач ввода-вывода, характерных для редиректа.

5. **Инструментарий.** Встроенные средства тестирования (`go test`, детектор гонок `-race`, замеры покрытия), форматирования (`gofmt`), статического анализа (`go vet`) и управления зависимостями (модули) снижают затраты на настройку проекта.

Для сравнения рассмотрены альтернативные платформы (таблица 3.1). Оценки качественные и отражают типичные свойства технологий; точные значения зависят от конкретной реализации и нагрузки.

Таблица 3.1 — Сравнение платформ реализации

Критерий	**Go**	**Python / FastAPI**	**Rust / Actix**	**Node.js**
Модель параллелизма	горутины и каналы, многопоточность из коробки	асинхронность `asyncio`, ограничение GIL для CPU-задач	потоки и async/await (tokio), безопасность памяти на этапе компиляции	однопоточный цикл событий, воркеры для CPU-задач
Пропускная способность на задачах ввода-вывода	высокая	средняя	очень высокая	высокая
Порог входа	низкий	очень низкий	высокий (владение, времена жизни)	низкий
Поставка	один статический файл	интерпретатор и зависимости	один файл	среда выполнения и `node_modules`
Типизация	статическая	динамическая с подсказками типов	статическая, строгая	динамическая (или TypeScript)
Потребление памяти	умеренное	выше	минимальное	умеренное
Скорость разработки	высокая	очень высокая	ниже	высокая
Экосистема для веб-сервисов	зрелая, богатая stdlib	очень обширная	развивающаяся	очень обширная

Python/FastAPI уступает по пропускной способности и требует развёртывания интерпретатора и зависимостей; Rust/Actix даёт наилучшую производительность, но существенно повышает трудоёмкость разработки, что неоправданно для учебного проекта с нагрузкой, ограничиваемой скорее вводом-выводом, чем вычислениями; Node.js сопоставим по модели ввода-вывода, но уступает в простоте параллельной обработки CPU-операций (bcrypt, хэширование) и в статической типизации без TypeScript. Go представляет разумный компромисс между производительностью, простотой и скоростью разработки.

3.2 PostgreSQL

В качестве основного хранилища выбрана СУБД PostgreSQL [11] по следующим причинам.

– **ACID и целостность.** Транзакции, внешние ключи, ограничения `UNIQUE` и `NOT NULL` на уровне СУБД критичны для сервиса: уникальность кодов гарантируется базой данных и обеспечивает корректность при конкурентных запросах, а не логикой приложения.

– **Индексы.** Поддержка B-tree по умолчанию, составных и убывающих индексов, что используется в схеме (раздел 2.4).

– **Пакетная вставка.** Протокол `COPY` и многострочные вставки позволяют записывать сотни событий одной командой; в проекте пакет переходов записывается методом `CopyFrom` драйвера pgx (`COPY clicks ... FROM STDIN`), что снижает накладные расходы на сетевые обращения и фиксацию транзакций.

– **Агрегатные запросы.** `GROUP BY`, `date_trunc`, `count(DISTINCT ...)` позволяют вычислять статистику по дням, странам и источникам средствами СУБД.

– **Зрелость и открытость.** Свободная лицензия, развитая экосистема, поддержка секционирования таблиц для роста данных.

Реляционная модель соответствует связям «пользователь — ссылки — переходы» и потребности в агрегатных запросах; документоориентированные хранилища потребовали бы реализации агрегирования на стороне приложения.

3.2.1 Драйвер: pgx и database/sql

Для доступа к PostgreSQL применяется драйвер `pgx` версии 5 с пулом соединений `pgxpool` [13]. Сравнение с интерфейсом стандартной библиотеки `database/sql` приведено в таблице 3.2.

Таблица 3.2 — Сравнение pgx и database/sql

Критерий	pgx (native)	database/sql (+ драйвер)
Интерфейс	специфичный для PostgreSQL	универсальный
Пул соединений	`pgxpool` встроен	встроен в `database/sql`
Бинарный протокол и типы PostgreSQL	да, прямое сопоставление	через слой совместимости
`COPY FROM`	поддерживается	недоступен через универсальный интерфейс
Пакетная отправка запросов (batch)	да	нет
Переносимость на другую СУБД	нет	да

Поскольку проект ориентирован на PostgreSQL и использует пакетные операции, выбран нативный интерфейс `pgx`; переносимость между СУБД не требуется, а доступ к возможностям протокола полезен для записи аналитики. Схема изменяется через пронумерованные SQL-миграции, встроенные в бинарный файл (`embed`) и применяемые при запуске с учётом таблицы `schema_migrations` (её создаёт сам код применения миграций). Применение выполняется в одной транзакции под рекомендательной блокировкой `pg_advisory_xact_lock`, поэтому одновременный запуск нескольких экземпляров приложения не приводит к двойному применению.

3.3 Redis

Redis [7, 12] — in-memory хранилище структур данных; в системе он выполняет три роли.

– **Кэш ссылок.** Хранение в оперативной памяти даёт задержки порядка долей миллисекунды; команда `SET` с `EX` задаёт время жизни, а политика `allkeys-lru` автоматически вытесняет редкие ключи.

– **Счётчики популярности.** Команда `INCR` атомарна, поэтому конкурентные обращения не приводят к потерянным обновлениям; вместе с `EXPIRE` она реализует окно подсчёта.

– **Рейтинг.** Отсортированные множества (sorted sets) позволяют поддерживать и получать топ кодов по убыванию счёта за логарифмическое время.

Для работы с Redis используется клиент `go-redis` версии 9 [14]: поддерживает пул соединений, контексты и конвейеризацию команд. Тестирование выполняется на встроенной имитации сервера `miniredis` (v2.40.0), что позволяет запускать модульные тесты без реального Redis. Альтернатива — Memcached — не поддерживает богатых структур данных и атомарных счётчиков с тем же набором возможностей, поэтому не обеспечила бы рейтинг и счётчики популярности.

3.4 Аутентификация: bcrypt и HMAC-токены

Хэширование паролей. Пароли хэшируются алгоритмом bcrypt (пакет `golang.org/x/crypto/bcrypt`) с параметром стоимости 12 в промышленной конфигурации и минимальной стоимостью в тестах. Bcrypt является адаптивной функцией с солью, замедляющей перебор; параметр стоимости позволяет повышать вычислительную сложность по мере роста мощности оборудования. Альтернативы (scrypt, Argon2id) обладают дополнительной стойкостью к атакам на специализированном оборудовании, однако bcrypt остаётся приемлемым и широко поддерживаемым вариантом.

Токены. Токен доступа имеет вид `base64url(userID.expUnix).base64url(HMAC-SHA256(secret, payload))` и подписывается секретом сервера. Проверка выполняется сравнением подписи за постоянное время (`hmac.Equal`) и контролем срока действия. Сравнение с JWT приведено в таблице 3.3.

Таблица 3.3 — Сравнение собственных HMAC-токенов и JWT

Критерий	HMAC-токен проекта	JWT
Зависимости	только стандартная библиотека	внешняя библиотека
Сложность формата	минимальная: идентификатор, срок, подпись	заголовок, полезная нагрузка, подпись; множество алгоритмов
Риск ошибок конфигурации	подпись всегда HMAC-SHA256, выбор алгоритма клиентом невозможен	известные классы уязвимостей: `alg=none`, подмена алгоритма (RS256/HS256) при некорректной проверке
Интероперабельность	только внутреннее использование	стандарт (RFC 7519), поддержка в разных системах
Отзыв токена	только по истечении срока (или смене секрета)	то же без дополнительного хранилища
Ключевая ротация, `kid`, стороннее подтверждение	не поддерживается	поддерживается

Поскольку единственным потребителем токена является само приложение, а набор нужных утверждений минимален, выбран самодостаточный HMAC-токен: он проще проверяется, не требует сторонних библиотек и исключает класс атак, связанных с выбором алгоритма в заголовке. Подпись по алгоритму HMAC [9] вычисляется из общего секрета. Ограничением является отсутствие стандартизованной интероперабельности и механизма отзыва; для распределённых систем с несколькими издателями токенов предпочтительнее JWT с жёсткой проверкой алгоритма.

3.5 Контейнеризация и непрерывная интеграция

Docker и Compose. Контейнеризация обеспечивает воспроизводимость окружения и позволяет запустить систему вместе с зависимостями командой `docker compose up`. Подробно архитектура развёртывания описана в разделе 2.8 [15].

GitHub Actions. Конвейер непрерывной интеграции [16] запускается при каждом `push` и `pull request` и состоит из независимых заданий (рисунок 3.1).

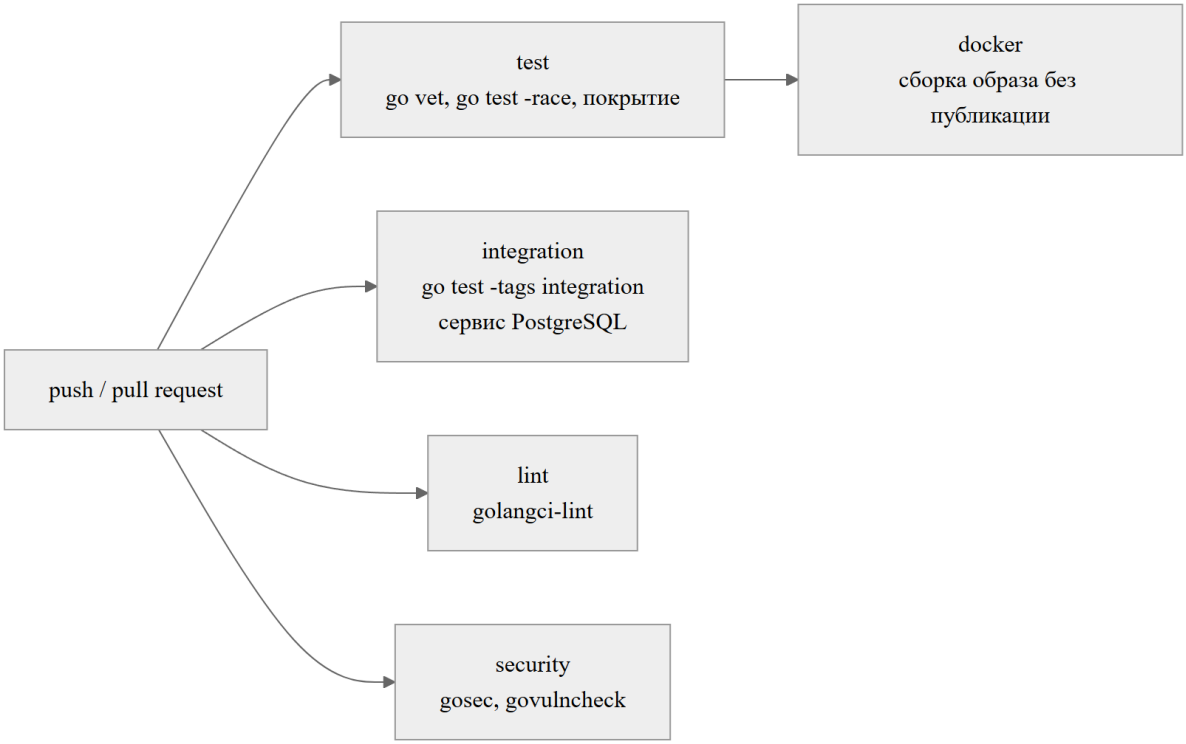


Рисунок 3.1 — Конвейер непрерывной интеграции

Задание `test` выполняет `go vet` и тесты с детектором гонок и сохраняет отчёт о покрытии как артефакт. Задание `integration` поднимает контейнер PostgreSQL в качестве сервиса GitHub Actions и выполняет тесты с тегом `integration`, проверяющие реализацию хранилища на реальной СУБД. Задание `lint` запускает `golangci-lint`, `security` — анализаторы безопасности, `docker` проверяет, что образ собирается. Права токена

конвейера ограничены чтением содержимого (`permissions: contents: read`) в соответствии с принципом минимальных привилегий.

3.6 Инструменты контроля качества и безопасности

Перечень применяемых инструментов приведён в таблице 3.4.

Таблица 3.4 — Инструменты качества и безопасности

Инструмент	Тип	Назначение
`gofmt`	форматирование	единый стиль кода
`go vet`	статический анализ	типовые ошибки: неверные форматные строки, копирование блокировок и др.
`golangci-lint`	агрегатор линтеров	`errcheck`, `staticcheck`, `govet`, `ineffassign`, `unused`, `gosec`, `bodyclose`, `misspell`, `revive`, `gocritic` (конфигурация `.golangci.yml`)
`gosec`	SAST	поиск небезопасных конструкций: слабая криптография, жёстко заданные секреты, небезопасные разрешения
`govulncheck`	SCA	поиск известных уязвимостей в зависимостях и стандартной библиотеке с учётом достижимости уязвимого кода
Dependabot	автоматическое обновление	еженедельные предложения обновления модулей Go, действий GitHub Actions и базовых образов Docker
`go test -race`	динамический анализ	обнаружение гонок данных

Сочетание SAST, SCA и динамического анализа покрывает основные классы дефектов, выявляемых автоматически. Необходимо отметить, что автоматические средства не заменяют ручного ревью и модели угроз; они уменьшают вероятность регрессий и известных уязвимостей [3, 17]. Тестирование организовано по уровням: модульные тесты каждого пакета без внешних сервисов (реализации хранилища и кэша в памяти, `miniredis`), интеграционные тесты хранилища PostgreSQL под тегом `integration` микробенчмарки, нагрузочный генератор `cmd/loadgen` и подготовленный сценарий k6 для проверки производительности редиректа.

Фактические версии языка и внешних зависимостей, зафиксированные в `go.mod`, приведены в таблице 3.5. Набор зависимостей намеренно невелик:

маршрутизация, журналирование, HMAC и встраивание статики используют стандартную библиотеку.

Таблица 3.5 — Версии языка и зависимостей

Компонент	Версия	Назначение
Go	1.27 (директива `go 1.27.1`)	язык и стандартная библиотека
`github.com/jackc/pgx/v5`	v5.11.0	драйвер и пул соединений PostgreSQL, `CopyFrom`
`github.com/redis/go-redis/v9`	v9.23.0	клиент Redis
`golang.org/x/crypto`	v0.58.0	bcrypt
`github.com/oschwald/maxminddb-golang`	v1.13.1	чтение базы GeoLite2
`github.com/alicebob/miniredis/v2`	v2.40.0	имитация Redis в тестах

3.7 Использование AI-ассистентов в разработке

В процессе разработки использовался ИИ-ассистент Claude Code. Его применение было ограничено вспомогательными задачами: генерацией каркаса проекта и типового кода (репозитории, middleware, конфигурация), написанием черновиков тестов, подготовкой черновиков документации и предварительным ревью. Для параллельной разработки применялась схема, при которой несколько экземпляров ассистента работали над отдельными пакетами, опираясь на общий контракт — документ `docs/DESIGN.md` и уже готовые интерфейсы (`model`, `storage`, `cache`, `analytics.Tracker`, `service`). Фиксация интерфейсов заранее позволила независимо реализовывать пакеты и затем собирать их без существенных конфликтов.

Ответственность за архитектурные решения (слоистая структура, асинхронный конвейер аналитики, политика кэширования, выбор технологий), за постановку требований и приёмку результата остаётся на разработчике. Сгенерированный код подвергался проверке: чтению, запуску тестов и линтеров, анализу граничных случаев и вопросов безопасности. Опыт показывает, что ассистент ускоряет выполнение рутинных операций, однако может допускать неточности и предлагать избыточные или устаревшие

решения, поэтому результаты не принимались без проверки, а текст настоящей записки сверялся с исходным кодом.

3.8 Выводы по главе 3

1. Go выбран за модель параллелизма, подходящую для конвейера аналитики, богатую стандартную библиотеку с маршрутизацией по шаблонам и статическую сборку; приведено сравнение с Python/FastAPI, Rust/Actix и Node.js.

2. PostgreSQL обеспечивает целостность и агрегатные запросы, Redis — быстрый кэш, атомарные счётчики и рейтинг; драйверы pgx и go-redis дают доступ к специфичным возможностям СУБД.

3. Аутентификация реализована на bcrypt и подписанных HMAC-токенах, что упрощает систему и исключает ряд уязвимостей, типичных для JWT, ценой отсутствия стандартной интероперабельности.

4. Контейнеризация, конвейер GitHub Actions и набор инструментов (golangci-lint, gosec, govulncheck, Dependabot) обеспечивают воспроизводимость сборки и автоматический контроль качества и безопасности.

5. Использование AI-ассистента ограничено вспомогательными задачами при сохранении ответственности разработчика за архитектуру и проверку результатов.

4 РЕАЛИЗАЦИЯ

4.1 Структура репозитория

Исходный код организован по стандартной для Go схеме: исполняемые команды находятся в `cmd/`, закрытые для импорта из других модулей пакеты — в `internal/`. Состав репозитория показан в листинге 4.1.

Листинг 4.1 — Структура каталогов репозитория

```
url-shortener/
├── cmd/
│   ├── shortener/main.go        точка входа сервиса, сборка зависимостей,
graceful shutdown
│   └── loadgen/main.go          генератор нагрузки с перцентилями задержки
├── internal/
│   ├── analytics/               Collector, разбор User-Agent, Referer, GeoIP
│   ├── auth/                    регистрация, вход, bcrypt, HMAC-токены
│   ├── cache/                   интерфейс LinkCache, RedisCache, MemoryCache
│   ├── config/                  конфигурация из переменных окружения
│   ├── httpapi/                 маршруты, обработчики, middleware, rate
limiting, метрики
│   ├── model/                   доменные сущности
│   ├── qrcode/                  PNG с QR-кодом ссылки: rsc.io/qr +
собственная отрисовка
│   ├── service/                 LinkService: создание, разрешение,
статистика
│   ├── shortcode/               генератор кодов и проверка алиасов
│   └── storage/
│       ├── storage.go           интерфейсы репозиториев
│       ├── memory/              реализация в памяти: memory.go, persist.go
(снимок в файл)
│       └── postgres/            реализация на pgx; migrations/001_init.sql
├── web/                         дашборд (index.html, app.js, style.css),
embed.go
├── docs/                        проектный документ, сценарий k6, скриншоты,
записка
├── .github/workflows/ci.yml     конвейер непрерывной интеграции
├── Dockerfile, docker-compose.yml, Makefile, .golangci.yml
└── go.mod, go.sum
```

Зависимости направлены строго от транспорта к данным: `httpapi` → `service` → (`storage`, `cache`, `analytics`). Пакеты `storage` и `cache` содержат интерфейсы и их реализации; сборка конкретных реализаций выполняется в `cmd/shortener/main.go`. Рабочий код на Go (включая генератор нагрузки и встраивание статики) занимает около 3,9 тыс. строк, тесты — около 3,4 тыс. строк.

4.2 Генератор коротких кодов

Генератор (`internal/shortcode/generator.go`) формирует код из алфавита base62 с помощью `crypto/rand`. Прямое взятие остатка от деления байта (0–255) на 62 приводило бы к смещению: 256 не кратно 62, и первые восемь символов алфавита выпадали бы чаще. Поэтому применено отсечение (rejection sampling): байты не меньше 248 (наибольшего кратного 62 числа, не превышающего 256) отбрасываются (листинг 4.2).

Листинг 4.2 — Генерация кода с отсечением смещённых значений

```
const maxByte = 256 - 256%len(alphabet) // 248

func (r *Random) Generate() (string, error) {
    out := make([]byte, 0, r.length)
    buf := make([]byte, r.length+r.length/2+4)
    for len(out) < r.length {
        if _, err := rand.Read(buf); err != nil {
            return "", fmt.Errorf("shortcode: crypto/rand: %w", err)
        }
        for _, b := range buf {
            if int(b) >= maxByte {
                continue // отбрасываем, иначе первые символы алфавита
выпадали бы чаще
            }
            out = append(out, alphabet[int(b)%len(alphabet)])
            if len(out) == r.length {
                break
            }
        }
    }
    return string(out), nil
}
```

Доля отбрасываемых байтов составляет 8/256 ≈ 3 %, поэтому буфер, прочитанный за одно обращение к `crypto/rand`, в подавляющем большинстве случаев содержит достаточно годных значений, и внешний цикл выполняется один раз. Равномерность распределения проверяется тестом `TestRandomUniform` (раздел 5.5). Уникальность кода обеспечивает не генератор, а ограничение `UNIQUE` в базе: `LinkService.Create` при `storage.ErrConflict` повторяет генерацию до пяти раз.

4.3 Разрешение ссылки: LinkService.Resolve

Метод `Resolve` реализует основной сценарий системы (рисунок 2.4). Он проверяет форму кода, читает запись из кэша, при промахе или ошибке кэша обращается к хранилищу, проверяет срок жизни и ставит событие в очередь аналитики (листинг 4.3).

Листинг 4.3 — Разрешение кода с деградацией при отказе кэша

```
func (s *LinkService) Resolve(ctx context.Context, code string, v Visit)
(string, error) {
    if !shortcode.LooksValid(code) {
        return "", ErrNotFound
    }
    now := s.now()

    entry, err := s.cache.Get(ctx, code)
    if err != nil {
        if !errors.Is(err, cache.ErrMiss) {
            // Кэш недоступен — деградируем до чтения из БД.
            s.log.Warn("cache get failed", "err", err)
        }
        entry, err = s.loadAndMaybeCache(ctx, code)
        if err != nil {
            return "", err
        }
    }
    if entry.ExpiresAt != nil && !now.Before(*entry.ExpiresAt) {
        return "", ErrGone
    }

    s.tracker.Track(analytics.Event{
        LinkID:      entry.LinkID,
        Time:        now,
        IP:          v.IP,
        UserAgent:   v.UserAgent,
        Referer:     v.Referer,
        CountryHint: v.CountryHint,
    })
    return entry.OriginalURL, nil
}
```

Любая ошибка кэша, кроме обычного промаха, лишь записывается в журнал: редирект продолжает работать за счёт PostgreSQL. Проверка `LooksValid` отсекает заведомо некорректные коды (слишком длинные, с недопустимыми символами) до обращения к кэшу и базе, что снижает нагрузку при переборе адресов. Срок жизни проверяется повторно и для записи из кэша: хотя TTL записи ограничен оставшимся временем жизни

ссылки, проверка исключает редирект по просроченной ссылке при любом расхождении часов.

Политика «кэшировать после порога» реализована в `loadAndMaybeCache` (листинг 4.4). Метод читает ссылку из хранилища, отклоняет отключённые и просроченные ссылки (`ErrGone`), увеличивает счётчик обращений и при достижении порога кладёт запись в кэш с временем жизни, не превышающим оставшегося срока ссылки.

Листинг 4.4 — Решение о кэшировании записи

```
func (s *LinkService) loadAndMaybeCache(ctx context.Context, code string)
(*cache.Entry, error) {
    link, err := s.store.GetByCode(ctx, code)
    if err != nil {
        if errors.Is(err, storage.ErrNotFound) {
            return nil, ErrNotFound
        }
        return nil, err
    }
    if !link.IsActive || link.Expired(s.now()) {
        return nil, ErrGone
    }
    entry := &cache.Entry{LinkID: link.ID, OriginalURL: link.OriginalURL,
ExpiresAt: link.ExpiresAt}

    hits, err := s.cache.Hit(ctx, code, s.opts.HitWindow)
    if err != nil {
        s.log.Warn("cache hit counter failed", "err", err)
        return entry, nil
    }
    if hits >= s.opts.PopularThreshold {
        ttl := s.opts.CacheTTL
        if link.ExpiresAt != nil {
            if left := link.ExpiresAt.Sub(s.now()); left < ttl {
                ttl = left
            }
        }
        if ttl > 0 {
            if err := s.cache.Set(ctx, code, *entry, ttl); err != nil {
                s.log.Warn("cache set failed", "err", err)
            }
        }
    }
    return entry, nil
}
```

4.4 Кэш на Redis: счётчик попаданий

Метод `RedisCache.Hit` увеличивает счётчик окна и счёт кода в рейтинге. Две команды отправляются одной транзакцией-конвейером

(MULTI/EXEC), что сокращает число сетевых обращений до одного; срок жизни счётчика выставляется отдельной командой только при первом обращении (листинг 4.5).

Листинг 4.5 — Счётчик обращений в Redis

```
func (r *RedisCache) Hit(ctx context.Context, code string, window
time.Duration) (int64, error) {
    var incr *redis.IntCmd
    _, err := r.c.TxPipelined(ctx, func(p redis.Pipeliner) error {
        incr = p.Incr(ctx, hitsKey(code))
        p.ZIncrBy(ctx, popularKey, 1, code)
        return nil
    })
    if err != nil {
        return 0, err
    }
    n := incr.Val()
    if n == 1 && window > 0 {
        if err := r.c.Expire(ctx, hitsKey(code), window).Err(); err != nil {
            return n, err
        }
    }
    return n, nil
}
```

Получаемое окно является фиксированным: оно открывается первым обращением и закрывается по истечении window, последующие обращения его не продлевают. Теоретически сбой между INCR и EXPIRE оставил бы счётчик без срока жизни; последствием был бы лишь более ранний перевод ссылки в кэш, а при нехватке памяти такой ключ вытесняется политикой allkeys-lru. Рейтинг для /api/v1/top читается командой ZREVRANGE (метод TopCodes).

4.5 Сборщик аналитики

Пакет analytics реализует асинхронный конвейер (рисунок 2.5). Метод Track вызывается из обработчика редиректа и не должен блокироваться, поэтому отправка в канал выполняется конструкцией select с веткой default (листинг 4.6). Читающая блокировка защищает флаг остановки: после закрытия приёма события отбрасываются, а не отправляются в канал, который уже дренируется.

Листинг 4.6 — Неблокирующая постановка события в очередь

```
func (c *Collector) Track(e Event) {
    c.mu.RLock()
    defer c.mu.RUnlock()
    if c.closed {
        c.dropped.Add(1)
        return
    }
    select {
    case c.ch <- e:
        c.accepted.Add(1)
    default:
        c.dropped.Add(1)
    }
}
```

Воркер (листинг 4.7) накапливает события в пакет и сбрасывает его при достижении `BatchSize` либо по таймеру `FlushInterval`. При отмене контекста воркер сначала закрывает приём новых событий (`stop`), затем дочитывает канал до опустошения и сбрасывает остаток: именно этот дренаж гарантирует, что события, принятые до остановки, не теряются (требование НФТ-4).

Листинг 4.7 — Воркер: пакетирование и дренаж при остановке

```
func (c *Collector) worker(ctx context.Context) {
    batch := make([]model.Click, 0, c.cfg.BatchSize)
    ticker := time.NewTicker(c.cfg.FlushInterval)
    defer ticker.Stop()
    flush := func() {
        if len(batch) > 0 {
            c.flush(batch)
            batch = batch[:0]
        }
    }
    for {
        select {
        case e := <-c.ch:
            batch = append(batch, c.enrich(e))
            if len(batch) >= c.cfg.BatchSize {
                flush()
            }
        case <-ticker.C:
            flush()
        case <-ctx.Done():
            c.stop()
            for { // дренаж: после stop новые события не поступают
                select {
                case e := <-c.ch:
                    batch = append(batch, c.enrich(e))
                    if len(batch) >= c.cfg.BatchSize {
                        flush()
                    }
```

```
                default:
                    flush()
                    return
                }
            }
        }
    }
}
```

Обогащение события (`enrich`) выполняется в воркере, а не в обработчике запроса, поэтому разбор User-Agent и поиск страны не увеличивают задержку редиректа. Идентификатор посетителя вычисляется как первые 16 шестнадцатеричных символов SHA-256 от строки «соль + IP + `|` + User-Agent»; в запись `Click` IP-адрес не попадает. Соль формируется из секрета `AUTH_SECRET` с префиксом `visitor:`. Страна берётся из подсказки CDN (только если она корректна: две латинские буквы, не `XX` и не `T1`), иначе определяется по базе GeoLite2, иначе принимает значение `ZZ`.

Запись пакета (`flush`) выполняется с таймаутом 5 с: сначала `InsertBatch` (в PostgreSQL — `COPY`), затем `AddClicks` с агрегированными по ссылкам приращениями. Ошибки записи увеличивают счётчик `Failed` и логируются; повторная отправка пакета не выполняется, то есть при сбое базы данные пакета теряются — это сознательный компромисс в пользу простоты.

4.6 QR-коды

Для удобного распространения коротких ссылок (печать, показ на экране, сканирование телефоном) сервис выдаёт QR-код ссылки. Реализация состоит из трёх частей: пакета `internal/qrcode`, метода `LinkService.CheckActive` и обработчика `GET /qr/{code}.png`.

Кодирование и отрисовка. Матрицу модулей QR-кода (выбор версии, коды Рида — Соломона, размещение модулей, маска) строит библиотека `rsc.io/qr` v0.2.0, единственная новая зависимость этой функции. Используется уровень коррекции ошибок M, позволяющий восстановить около 15 % повреждённых модулей: для коротких URL он даёт небольшую версию кода (крупные модули легче читаются камерой), сохраняя запас на

потёртость и блики. Растеризацию выполняет сам пакет: модуль (квадрат из целого числа пикселей) закрашивается в палитровом изображении `image.Paletted` из двух цветов, вокруг кода оставляется белая рамка шириной 4 модуля, которую требует стандарт ISO/IEC 18004 для надёжного распознавания. Масштаб модуля — целое число, поэтому сторона итоговой картинки не превышает запрошенную, но может быть немного меньше неё. Двухцветная палитра позволяет `image/png` записать файл с глубиной 1 бит на пиксель. Допустимый размер ограничен диапазоном 64–1024 пикселя, размер по умолчанию — 256 (листинг 4.8).

Листинг 4.8 — Формирование PNG-изображения QR-кода (`internal/qrcode/qrcode.go`)

```
// PNG кодирует text в QR-код с уровнем коррекции ошибок M (восстанавливается
// около 15 % повреждённых модулей) и возвращает PNG со стороной не больше
size
// пикселей. Размер приводится к диапазону [MinSize, MaxSize]. Модуль —
квадрат
// из целого числа пикселей, поэтому сторона может быть немного меньше
запрошенной.
func PNG(text string, size int) ([]byte, error) {
    size = min(max(size, MinSize), MaxSize)
    c, err := qr.Encode(text, qr.M)
    if err != nil {
        return nil, fmt.Errorf("qrcode: %w", err)
    }

    modules := c.Size + 2*quietZone
    scale := max(1, size/modules)
    side := modules * scale

    // Paletted по умолчанию заполнен индексом 0 — белым цветом.
    img := image.NewPaletted(image.Rect(0, 0, side, side), palette)
    for my := 0; my < c.Size; my++ {
        for mx := 0; mx < c.Size; mx++ {
            if !c.Black(mx, my) {
                continue
            }
            x0, y0 := (mx+quietZone)*scale, (my+quietZone)*scale
            for y := y0; y < y0+scale; y++ {
                row := img.Pix[y*img.Stride:]
                for x := x0; x < x0+scale; x++ {
                    row[x] = 1 // чёрный
                }
            }
        }
    }

    var buf bytes.Buffer
    if err := encoder.Encode(&buf, img); err != nil {
        return nil, fmt.Errorf("qrcode: %w", err)
```

```
    }
    return buf.Bytes(), nil
}
```

Почему не используется встроенная отрисовка библиотеки. Модульные тесты с независимым декодером (раздел 5.4) выявили два дефекта `rsc.io/qr`: встроенный PNG-кодировщик `Code.PNG()` аварийно завершается («invalid flate repeat») при масштабе 1, то есть для небольших картинок, а метод `Code.Image()` игнорирует масштаб и белую рамку, так как метод `At` не учитывает ни `Scale`, ни отступ. Поэтому от библиотеки используется только матрица (`qr.Encode` и `Code.Black`), а растеризация реализована в проекте и покрыта тестами.

Пул буферов. Кодировщик `image/png` при каждом вызове выделяет около 1 МБ под состояние сжатия. Поскольку запросы QR-кодов могут идти потоком (например, дашборд загружает коды для списка ссылок), кодировщик создан один раз на уровне пакета и использует `png.EncoderBufferPool` на базе `sync.Pool`: буферы переиспользуются между запросами. Бенчмарк `BenchmarkPNG` (картинка 256 пикселей) показал снижение выделяемой памяти с 1 247 111 до 86 742 байт на операцию (в 14 раз) и числа выделений с 61 до 34, время операции сократилось с 1,18 до 0,985 мс (подраздел 5.6).

Проверка ссылки. Метод `LinkService.CheckActive(ctx, code)` проверяет, что код допустим, а ссылка существует, не отключена и не истекла: сначала по кэшу, при промахе или ошибке кэша — по хранилищу. В отличие от `Resolve` метод не ставит событие в очередь аналитики и не увеличивает счётчик популярности: показ QR-кода не является переходом и не должен искажать статистику. Для несуществующего кода возвращается `ErrNotFound` (404), для отключённой или истёкшей ссылки — `ErrGone` (410).

Обработчик. Обработчик `handleQR` (листинг 4.9) зарегистрирован по шаблону `GET /qr/{file}`: в нём из последнего сегмента отделяется расширение `.png`, при его отсутствии возвращается 404. Параметр `size` проверяется до обращения к хранилищу (400 при нечисловом значении или

значении вне диапазона). В QR кодируется `BASE_URL + "/" + code`. Ответ содержит `Content-Type: image/png`, `Content-Length` и `Cache-Control: public, max-age=3600`: картинка зависит только от кода и размера, поэтому браузеры и промежуточные кэши могут её хранить. Адрес сделан публичным (без токена), потому что в код записана лишь короткая ссылка, которая и так открыта любому, кто её знает; зато код можно подставлять в тег `<img>` без передачи заголовков. Счётчик `shortener_qr_generated_total` растёт после успешной выдачи изображения.

Листинг 4.9 — Обработчик выдачи QR-кода (`internal/httpapi/handlers.go`)

```
// handleQR отдаёт PNG с QR-кодом короткой ссылки: GET
/qr/{code}.png?size=256.
// Адрес публичный: в QR зашита только короткая ссылка, которая и так
открыта.
func (s *server) handleQR(w http.ResponseWriter, r *http.Request) {
    code, ok := strings.CutSuffix(r.PathValue("file"), ".png")
    if !ok {
        writeError(w, http.StatusNotFound, "link not found")
        return
    }
    size := qrcode.DefaultSize
    if v := r.URL.Query().Get("size"); v != "" {
        parsed, err := strconv.Atoi(v)
        if err != nil || parsed < qrcode.MinSize || parsed > qrcode.MaxSize {
            writeError(w, http.StatusBadRequest,
                "size must be between "+strconv.Itoa(qrcode.MinSize)+" and
"+strconv.Itoa(qrcode.MaxSize))
            return
        }
        size = parsed
    }
    if err := s.Links.CheckActive(r.Context(), code); err != nil {
        s.writeServiceError(w, r, err)
        return
    }
    img, err := qrcode.PNG(s.BaseURL+"/"+code, size)
    if err != nil {
        s.writeServiceError(w, r, err)
        return
    }
    s.Metrics.QRGenerated.Add(1)
    w.Header().Set("Content-Type", "image/png")
    w.Header().Set("Content-Length", strconv.Itoa(len(img)))
    // Картинка зависит только от кода и размера, её можно кэшировать.
    w.Header().Set("Cache-Control", "public, max-age=3600")
    _, _ = w.Write(img)
}
```

Ограничение для телефона. QR-код содержит адрес, построенный из `BASE_URL`. При запуске с `BASE_URL=http://localhost` ссылка на телефоне не откроется, так как для телефона `localhost` — он сам. В Windows-поставке для этого предусмотрен скрипт `Запустить для телефона.bat`, который определяет IP-адрес ноутбука в локальной сети, запускает сервис на `0.0.0.0:80` с `BASE_URL=http://<IP>` и открывает дашборд по этому адресу.

4.7 Разбор User-Agent

Для определения браузера, ОС и типа устройства применяются эвристики на основе `strings.Contains` без внешних библиотек (`internal/analytics/useragent.go`). Порядок проверок существенен: строки User-Agent Edge, Opera, Яндекс.Браузера и Samsung Internet содержат токен `Chrome/`, а Chrome — токен `Safari/`, поэтому более специфичные браузеры проверяются раньше (листинг 4.10).

Листинг 4.10 — Определение браузера и типа устройства

```
func parseBrowser(ua string) string {
    switch {
    case strings.Contains(ua, "Edg/"), strings.Contains(ua, "EdgA/"),
strings.Contains(ua, "EdgiOS/"), strings.Contains(ua, "Edge/"):
        return "Edge"
    case strings.Contains(ua, "OPR/"), strings.Contains(ua, "Opera"),
strings.Contains(ua, "OPT/"):
        return "Opera"
    case strings.Contains(ua, "YaBrowser"):
        return "Yandex"
    case strings.Contains(ua, "SamsungBrowser"):
        return "Samsung Internet"
    case strings.Contains(ua, "Firefox/"), strings.Contains(ua, "FxiOS/"):
        return "Firefox"
    case strings.Contains(ua, "Chrome/"), strings.Contains(ua, "CriOS/"),
strings.Contains(ua, "Chromium/"):
        return "Chrome"
    case strings.Contains(ua, "Safari/"):
        return "Safari"
    }
    return "Other"
}

func parseDevice(ua string) string {
    if strings.TrimSpace(ua) == "" {
        return "bot"
    }
    l := strings.ToLower(ua)
    for _, k := range []string{"bot", "crawler", "spider", "curl", "wget",
"python-requests", "go-http-client"} {
```

```
        if strings.Contains(l, k) {
            return "bot"
        }
    }
    switch {
    case strings.Contains(ua, "iPad"):
        return "tablet"
    case strings.Contains(ua, "Android") && !strings.Contains(ua, "Mobile"):
        return "tablet"
    case strings.Contains(ua, "Mobile"), strings.Contains(ua, "iPhone"),
strings.Contains(ua, "iPod"):
        return "mobile"
    }
    return "desktop"
}
```

При определении типа устройства первой выполняется проверка на роботов (пустой User-Agent считается роботом), затем планшеты: Android без токена `Mobile` относится к планшетам, как принято в User-Agent браузеров для Android. Эвристики не претендуют на полноту: нестандартные и намеренно искажённые строки попадают в категорию `Other`.

4.8 Аутентификация

Токен формируется как `base64url(userID.exp).base64url(HMAC-SHA256)`. При проверке подпись сравнивается за постоянное время функцией `hmac.Equal` до разбора полезной нагрузки, затем проверяется срок действия (листинг 4.11).

Листинг 4.11 — Выпуск и проверка токена

```
func (s *Service) issue(userID int64) string {
    exp := s.now().Add(s.ttl).Unix()
    payload :=
base64.RawURLEncoding.EncodeToString([]byte(fmt.Sprintf("%d.%d", userID,
exp)))
    return payload + "." +
base64.RawURLEncoding.EncodeToString(s.sign(payload))
}

func (s *Service) ParseToken(token string) (int64, error) {
    payload, sig, ok := strings.Cut(token, ".")
    if !ok {
        return 0, ErrInvalidToken
    }
    got, err := base64.RawURLEncoding.DecodeString(sig)
    if err != nil || !hmac.Equal(got, s.sign(payload)) {
        return 0, ErrInvalidToken
    }
    // ... разбор "id.exp" ...
```

```
    if err1 != nil || err2 != nil || !s.now().Before(time.Unix(exp, 0)) {
        return 0, ErrInvalidToken
    }
    return id, nil
}
```

Пароли хэшируются bcrypt со стоимостью 12 (константа `bcryptCost` в `main.go`); допустимая длина пароля — от 8 до 72 байт (предел bcrypt). При входе несуществующего пользователя выполняется сравнение с заранее вычисленным фиктивным хэшем, а в обоих случаях отказа возвращается одна ошибка `ErrInvalidCredentials`, которая превращается в ответ `401 invalid email or password`. Тем самым и текст ответа, и время обработки не позволяют определить, зарегистрирован ли адрес.

4.9 HTTP-слой: маршрутизация и middleware

Маршруты регистрируются в `http.ServeMux` с использованием шаблонов Go 1.22: метод и путь задаются одной строкой, параметры пути извлекаются методом `r.PathValue` (листинг 4.12). Шаблон `GET /{code}` является самым общим; мультиплексор выбирает наиболее конкретный подходящий шаблон независимо от порядка регистрации, поэтому `/healthz`, `/metrics`, `/dashboard/` и `/api/...` не перекрываются маршрутом редиректа. Шаблон `GET /{$}` соответствует только корню.

Листинг 4.12 — Маршруты и цепочка middleware

```
mux.HandleFunc("GET /healthz", s.handleHealth)
mux.HandleFunc("GET /metrics", s.handleMetrics)

mux.HandleFunc("POST /api/v1/auth/register", s.limited(s.handleRegister))
mux.HandleFunc("POST /api/v1/auth/login", s.limited(s.handleLogin))
mux.HandleFunc("GET /api/v1/me", s.authed(s.handleMe))
mux.HandleFunc("POST /api/v1/links", s.limited(s.authed(s.handleCreateLink)))
mux.HandleFunc("GET /api/v1/links", s.authed(s.handleListLinks))
mux.HandleFunc("GET /api/v1/links/{code}", s.authed(s.handleGetLink))
mux.HandleFunc("DELETE /api/v1/links/{code}", s.authed(s.handleDeleteLink))
mux.HandleFunc("GET /api/v1/links/{code}/stats", s.authed(s.handleStats))
mux.HandleFunc("GET /api/v1/top", s.handleTop)
mux.HandleFunc("GET /qr/{file}", s.handleQR)
// ...
mux.HandleFunc("GET /{code}", s.handleRedirect)

var h http.Handler = mux
h = limitBody(h)
```

```
h = securityHeaders(h)
h = recoverer(s.log, h)
h = accessLog(s.log, s.Metrics, s.TrustProxy, h)
h = requestID(h)
return h
```

Middleware применяются как «луковица»: последний обёрнутый является внешним, поэтому запрос последовательно проходит `requestID` → `accessLog` → `recoverer` → `securityHeaders` → `limitBody` → маршрутизатор. Порядок значим: `recoverer` расположен внутри `accessLog`, чтобы паника обработчика превращалась в ответ 500 и попадала в журнал и метрики; `requestID` — снаружи, чтобы идентификатор был доступен всем остальным. Обёртки уровня маршрута `limited` (ограничение частоты) и `authed` (проверка токена) применяются только к тем конечным точкам, которым они нужны. Ограничение частоты установлено на `POST /auth/register`, `POST /auth/login` и `POST /links`; редиректы, `/healthz` и `/metrics` не лимитируются (тест `TestRateLimit`).

Заголовки безопасности задаются middleware `securityHeaders` для всех ответов: `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy: strict-origin-when-cross-origin` и политика `Content-Security-Policy`, разрешающая скрипты и стили только с собственного источника (`script-src 'self'; style-src 'self'; object-src 'none'; base-uri 'none'; frame-ancestors 'none'`). Поскольку дашборд не использует встроенных скриптов и стилей, политика не мешает его работе. Ответ редиректа дополнительно содержит `Cache-Control: private, max-age=0`.

4.10 Ограничение частоты запросов

Ограничитель (`internal/httpapi/ratelimit.go`) реализует алгоритм «token bucket» по ключу — IP-адресу клиента. Каждый ключ получает ведро ёмкостью `burst`, пополняемое со скоростью `rps` токенов в секунду; запрос списывает один токен. Состояние хранится в карте под мьютексом, а ведра,

которые успели бы наполниться полностью, периодически удаляются, чтобы память не росла при множестве разовых клиентов (листинг 4.13).

Листинг 4.13 — Token bucket

```
func (l *RateLimiter) Allow(key string) (bool, time.Duration) {
    if l == nil || l.rps <= 0 {
        return true, 0
    }
    l.mu.Lock()
    defer l.mu.Unlock()

    now := l.now()
    l.sweep(now)

    b, ok := l.buckets[key]
    if !ok {
        b = &bucket{tokens: l.burst, last: now}
        l.buckets[key] = b
    } else {
        elapsed := now.Sub(b.last).Seconds()
        if elapsed > 0 {
            b.tokens = math.Min(l.burst, b.tokens+elapsed*l.rps)
            b.last = now
        }
    }
    if b.tokens >= 1 {
        b.tokens--
        return true, 0
    }
    wait := time.Duration((1 - b.tokens) / l.rps * float64(time.Second))
    return false, wait
}
```

При отказе обработчик возвращает `429 Too Many Requests` с заголовком `Retry-After` (в секундах) и увеличивает метрику `shortener_rate_limited_total`. Значения по умолчанию — 5 запросов в секунду и всплеск до 20 — задаются переменными `RATE_LIMIT_RPS` и `RATE_LIMIT_BURST`. IP клиента определяет функция `clientIP`: заголовок `X-Forwarded-For` (первый адрес списка) учитывается только при `TRUST_PROXY=true` и только если он является корректным IP-адресом; иначе используется `RemoteAddr`. Состояние ограничителя хранится в памяти процесса, поэтому при нескольких экземплярах приложения лимит действует на каждый из них отдельно.

4.11 Служебные конечные точки

Обработчик `/healthz` последовательно опрашивает зависимости (`database` и `cache`, методом `Ping`) с общим таймаутом 2 с и возвращает `200` с телом `{"status":"ok","checks":{...}}` либо `503` со статусом `unavailable`. Конечная точка `/metrics` выводит счётчики в текстовом формате Prometheus: `shortener_http_requests_total`, `shortener_http_responses_5xx_total`, `shortener_redirects_total`, `shortener_redirects_not_found_total`, `shortener_redirects_gone_total`, `shortener_rate_limited_total`, `shortener_links_created_total`, `shortener_qr_generated_total`, `shortener_http_in_flight` (gauge), а также счётчики сборщика `shortener_analytics_accepted_total`, `shortener_analytics_dropped_total`, `shortener_analytics_written_total` и `shortener_analytics_failed_total`. Все счётчики реализованы атомарными типами стандартной библиотеки, без внешних библиотек.

4.12 Миграции и пакетная запись в PostgreSQL

Схема изменяется нумерованными SQL-файлами `internal/storage/postgres/migrations/NNN_*.sql`, встроенными в бинарный файл директивой `//go:embed`. Метод `Migrate` вызывается при создании хранилища и выполняет все ещё не применённые файлы в одной транзакции под рекомендательной блокировкой, чтобы одновременный запуск нескольких экземпляров не привёл к конкурирующему применению (листинг 4.14). Таблицу `schema_migrations` создаёт сам этот код, поэтому в `001_init.sql` её нет.

Листинг 4.14 — Применение миграций под advisory lock

```
tx, err := s.pool.Begin(ctx)
if err != nil {
    return fmt.Errorf("postgres: начало транзакции миграций: %w", err)
}
defer tx.Rollback(ctx) //nolint:errcheck // после Commit вернёт ErrTxClosed
```

```
if _, err := tx.Exec(ctx, `SELECT pg_advisory_xact_lock($1)`,
migrationLockID); err != nil {
    return fmt.Errorf("postgres: блокировка миграций: %w", err)
}
if _, err := tx.Exec(ctx, `CREATE TABLE IF NOT EXISTS schema_migrations (
    version INT PRIMARY KEY,
    applied_at TIMESTAMPTZ NOT NULL DEFAULT now())`); err != nil {
    return fmt.Errorf("postgres: schema_migrations: %w", err)
}
// ... чтение применённых версий ...
for _, m := range list {
    if applied[m.version] {
        continue
    }
    body, err := migrationsFS.ReadFile("migrations/" + m.name)
    // ...
    if _, err := tx.Exec(ctx, string(body)); err != nil {
        return fmt.Errorf("postgres: миграция %s: %w", m.name, err)
    }
    if _, err := tx.Exec(ctx, `INSERT INTO schema_migrations (version) VALUES
($1)`, m.version); err != nil {
        return fmt.Errorf("postgres: отметка миграции %s: %w", m.name, err)
    }
}
return tx.Commit(ctx)
```

Блокировка `pg_advisory_xact_lock` снимается автоматически при завершении транзакции. Поскольку DDL в PostgreSQL транзакционен, при ошибке в любой миграции все изменения откатываются.

Пакет переходов записывается протоколом `COPY` через `pgx.CopyFrom`, а счётчики ссылок увеличиваются одним запросом с разворачиванием массивов (листинг 4.15).

Листинг 4.15 — Пакетная вставка переходов и обновление счётчиков

```
func (s *Store) InsertBatch(ctx context.Context, clicks []model.Click) error
{
    if len(clicks) == 0 {
        return nil
    }
    _, err := s.pool.CopyFrom(ctx,
        pgx.Identifier{"clicks"},
        []string{"link_id", "clicked_at", "referrer", "country", "browser",
"os", "device", "visitor_id"},
        pgx.CopyFromSlice(len(clicks), func(i int) ([]any, error) {
            c := clicks[i]
            return []any{c.LinkID, c.ClickedAt, c.Referrer, c.Country,
c.Browser, c.OS, c.Device, c.VisitorID}, nil
        }))
    return mapErr(err)
}

// AddClicks (фрагмент)
_, err := s.pool.Exec(ctx,
```

```
    `UPDATE links SET click_count = click_count + d.delta
     FROM unnest($1::bigint[], $2::bigint[]) AS d(id, delta)
     WHERE links.id = d.id`, ids, vals)
```

4.13 Сохранение данных демо-режима

Без PostgreSQL сервис хранил данные только в памяти, и при закрытии окна консоли пропадали учётные записи, ссылки и статистика; кроме того, секрет подписи токенов генерировался заново при каждом запуске, поэтому «слетал» и вход. Для демонстрации это неудобно, а устанавливать PostgreSQL на ноутбук ради показа нецелесообразно. Поэтому добавлен третий режим хранения (раздел 2.8): при заданной переменной `DATA_FILE` и пустой `DATABASE_URL` используется тип `memory.Persistent` — обёртка над `memory.Store`, которая при запуске загружает снимок состояния из JSON-файла, а затем сохраняет его после изменений. Совместное задание `DATA_FILE` и `DATABASE_URL` отвергается как ошибка конфигурации.

Отслеживание изменений. В `memory.Store` введён счётчик версии `version`, который увеличивается в каждом изменяющем методе (`Create`, `Deactivate`, `AddClicks`, `CreateUser`, `InsertBatch`) вызовом `touch()` под уже захваченной блокировкой записи (листинг 4.16). Тот же вызов неблокирующе отправляет уведомление в канал `changed` ёмкостью 1: если уведомление уже ожидает обработки, новое не нужно, так как фоновая горутина всё равно сохранит актуальное состояние. Для обычного хранилища в памяти канал не создаётся (`nil`), и накладные расходы сводятся к инкременту счётчика.

Листинг 4.16 — Отметка об изменении данных (`internal/storage/memory/memory.go`)

```
// touch отмечает изменение данных. Вызывается под s.mu.Lock().
func (s *Store) touch() {
    s.version++
    if s.changed != nil {
        select {
        case s.changed <- struct{}{}:
        default: // уведомление уже ожидает обработки
        }
    }
}
```

Фоновое сохранение с задержкой. Горутина `loop()` ожидает уведомление, затем выдерживает паузу `debounce` (константа `dataSaveDebounce` в `main.go`, 500 мс) и только потом сохраняет снимок (листинг 4.17). Все изменения, произошедшие за паузу, попадают в одну запись, поэтому поток переходов по ссылкам не превращается в поток записей на диск: не более двух записей в секунду независимо от числа переходов. Если версия данных с момента прошлой записи не изменилась, метод `Save` возвращается сразу и файл не трогается. Ошибка сохранения журналируется, но не останавливает сервис: данные остаются в памяти и будут записаны при следующем изменении или при закрытии.

Листинг 4.17 — Фоновая горутина сохранения (`internal/storage/memory/persist.go`)

```
// loop ждёт уведомления об изменениях и сохраняет снимок не чаще раза в
debounce:
// поток переходов не превращается в поток записей на диск.
func (p *Persistent) loop() {
    defer close(p.done)
    for {
        select {
        case <-p.stop:
            return
        case <-p.changed:
        }
        select {
        case <-p.stop:
            return // финальное сохранение выполнит Close
        case <-time.After(p.debounce):
        }
        if err := p.Save(); err != nil {
            p.log.Error("сохранение данных в файл", "path", p.path, "err",
err)
        }
    }
}
```

Атомарная запись. Простая перезапись файла небезопасна: сбой питания или завершение процесса посреди записи оставляет обрезанный JSON, и все данные оказываются потеряны. Поэтому `Save` пишет снимок во временный файл в том же каталоге (`os.CreateTemp`; тот же каталог

гарантирует, что переименование не пересекает границы файловых систем), вызывает `Sync` для сброса данных на носитель, закрывает файл и только затем заменяет старый файл вызовом `os.Rename` (листинг 4.18). Переименование выполняется операционной системой атомарно, поэтому на диске в любой момент лежит либо прежняя, либо новая полная версия. Права временного файла — 0600 (так создаёт `CreateTemp`), потому что в снимке находятся хэши паролей. Блокировка `saveMu` сериализует записи из фоновой горутины и из `Close`, а снимок копируется под короткой блокировкой на чтение (`takeSnapshot`), так что сериализация JSON и ввод-вывод не задерживают обработку запросов.

Листинг 4.18 — Атомарная запись снимка (`internal/storage/memory/persist.go`)

```
func (p *Persistent) Save() error {
    p.saveMu.Lock()
    defer p.saveMu.Unlock()
    snap, ver := p.takeSnapshot()
    if ver == p.savedVer {
        return nil
    }
    snap.SavedAt = time.Now().UTC()
    data, err := json.Marshal(snap)
    if err != nil {
        return err
    }
    dir := filepath.Dir(p.path)
    tmp, err := os.CreateTemp(dir, filepath.Base(p.path)+".*.tmp")
    if err != nil {
        return err
    }
    tmpName := tmp.Name()
    defer os.Remove(tmpName) //nolint:errcheck // после успешного Rename
файла уже нет
    if _, err := tmp.Write(data); err != nil {
        _ = tmp.Close()
        return err
    }
    if err := tmp.Sync(); err != nil {
        _ = tmp.Close()
        return err
    }
    if err := tmp.Close(); err != nil {
        return err
    }
    if err := os.Rename(tmpName, p.path); err != nil {
        return err
    }
    p.savedVer = ver
    return nil
```

```
}
```

Формат снимка. Файл представляет собой JSON-объект с полем `format` (версия формата, сейчас 1), временем записи `saved_at`, счётчиками следующих идентификаторов и массивами `users`, `links` и `clicks`. Для снимка заведены отдельные структуры `userRec`, `linkRec` и `clickRec`, а не доменные модели: в моделях поля `PasswordHash` и `VisitorID` помечены `json:"-"`, чтобы не попадать в ответы API, а в файле они необходимы для восстановления входа и подсчёта уникальных посетителей. Версия формата позволяет при несовместимых изменениях распознавать устаревшие файлы вместо их ошибочной интерпретации.

Отказ запуска при повреждении. Если файл существует, но не разбирается как JSON или имеет неизвестную версию формата, `Open` возвращает ошибку, сервис не стартует, а файл остаётся нетронутым. Альтернатива — начать с пустого хранилища — привела бы к тому, что первая же фоновая запись молча затёрла бы повреждённый файл вместе с данными, которые ещё можно восстановить вручную или из резервной копии. Отсутствующий файл, напротив, не ошибка: это обычный первый запуск. При восстановлении счётчики идентификаторов принимаются не меньшими максимальных существующих, что защищает от повторной выдачи ID при ручной правке файла.

Остановка. Метод `Close` останавливает фоновую горутину, дожидается её завершения и выполняет итоговое сохранение; повторный вызов безопасен (`sync.Once`). В `main` хранилище закрывается последним: сначала останавливается HTTP-сервер, затем дренируется сборщик аналитики и лишь после этого вызывается `Close`, поэтому итоговый снимок включает все обработанные переходы (раздел 4.15). Если процесс завершён принудительно (закрытие окна консоли, выключение питания), теряются только изменения за последние `debounce`, то есть около 0,5 с; целостность файла при этом сохраняется благодаря атомарной записи.

Секрет подписи токенов. Чтобы токены оставались действительными после перезапуска, секрет не должен генерироваться заново. Если AUTH_SECRET не задан, а DATA_FILE задан, функция config.LoadOrCreateSecret читает секрет из файла <DATA_FILE>.secret, а при его отсутствии генерирует 48 случайных байт (кодировка base64url) и записывает в этот файл с правами 0600. Секрет короче 32 байт в существующем файле считается ошибкой. Явно заданный AUTH_SECRET имеет приоритет над файлом. Секрет вынесен в отдельный файл, а не в data.json, потому что это разные по смыслу данные: файл данных можно копировать и сбрасывать, не затрагивая ключ подписи, а ключ не попадает в копии данных. Пути, получаемые из DATA_FILE, нормализуются функцией filepath.Clean; этим устранены замечания анализатора gosec G304 о чтении файла по пути из переменной, без подавляющих комментариев #nosec (раздел 5.5).

4.14 Запросы статистики

Агрегаты по ссылке вычисляются средствами СУБД за полуинтервал [from, to); каждый запрос использует составной индекс idx_clicks_link_time (листинг 4.19). Запросы разрезов заданы константами, значения передаются параметрами ($1…$4), конкатенация строк SQL не применяется, что исключает SQL-инъекции.

Листинг 4.19 — Запросы статистики

```
const qByCountry = `SELECT trim(country) AS k, count(*) AS n FROM clicks
WHERE link_id = $1 AND clicked_at >= $2 AND clicked_at < $3 GROUP BY k ORDER
BY n DESC, k LIMIT $4`
// аналогично qByReferrer, qByBrowser, qByOS, qByDevice


`SELECT count(*), count(DISTINCT visitor_id) FROM clicks
 WHERE link_id = $1 AND clicked_at >= $2 AND clicked_at < $3`


`SELECT date_trunc('day', clicked_at AT TIME ZONE 'UTC') AS d, count(*)
 FROM clicks WHERE link_id = $1 AND clicked_at >= $2 AND clicked_at < $3
 GROUP BY d ORDER BY d`
```

Функция trim(country) нужна потому, что значение типа CHAR(2) может дополняться пробелами. Дни группируются по UTC, ограничение

`LIMIT $4` задаёт размер рейтингов (`StatsTopN`, по умолчанию 10). Сервисный слой дополнительно проверяет, что период не пуст и не превышает 366 суток (`ErrBadPeriod`).

4.15 Запуск и корректная остановка

Точка входа (`cmd/shortener/main.go`) загружает конфигурацию, выбирает реализации хранилища и кэша (PostgreSQL и Redis при заданных `DATABASE_URL` и `REDIS_URL`, иначе реализации в памяти; при заданном `DATA_FILE` — хранилище в памяти со снимком в файле, раздел 4.13), создаёт сборщик аналитики и запускает его в отдельной горутине, собирает сервисы и HTTP-сервер с таймаутами (`ReadHeaderTimeout` 5 с, `ReadTimeout` 15 с, `WriteTimeout` 30 с, `IdleTimeout` 90 с). Остановка по сигналу выполняется в порядке, показанном в листинге 4.20: сначала перестаёт принимать запросы HTTP-сервер, затем останавливается и дренируется сборщик, и лишь после этого (через `defer`) закрываются кэш и хранилище, которые сборщик использует при записи.

Листинг 4.20 — Корректная остановка

```
ctx, stop := signal.NotifyContext(context.Background(), os.Interrupt,
syscall.SIGTERM)
defer stop()
// ... сборка зависимостей ...
collectorCtx, cancelCollector := context.WithCancel(context.Background())
var collectorDone sync.WaitGroup
collectorDone.Add(1)
go func() {
    defer collectorDone.Done()
    collector.Run(collectorCtx)
}()
// ...
select {
case <-ctx.Done():
    log.Info("получен сигнал остановки")
case err := <-serveErr:
    if err != nil {
        runErr = fmt.Errorf("http-сервер: %w", err)
    }
}

// Корректная остановка: сервер -> сборщик (дренаж очереди) -> store/cache
(defer).
shutdownCtx, cancel := context.WithTimeout(context.Background(),
shutdownTimeout)
defer cancel()
```

```
if err := srv.Shutdown(shutdownCtx); err != nil {
    log.Error("остановка http-сервера", "err", err)
}
cancelCollector()
collectorDone.Wait()
```

Сборщик получает собственный контекст, не связанный с контекстом сигнала: иначе он начал бы останавливаться одновременно с сервером и мог бы потерять события запросов, ещё обрабатываемых при `Shutdown`. Тайм-аут остановки сервера — 15 с.

4.16 Дашборд

Интерфейс владельца реализован на чистых HTML, CSS и JavaScript без сборщиков и фреймворков (файлы `web/static/index.html`, `style.css`, `app.js`) и встраивается в бинарный файл директивой `//go:embed static`. Сервис раздаёт его по адресу `/dashboard/` с заголовком `Cache-Control: no-cache`. Токен доступа хранится в `localStorage`; при ответе `401` на защищённый запрос выполняется выход.

Все данные, введённые пользователями (названия, адреса), вставляются в DOM через `textContent` и `setAttribute`; `innerHTML` не используется, поэтому внедрение разметки невозможно. Вспомогательная функция `el` создаёт элементы, присваивая текст свойством `textContent`. Дополнительно действует строгая политика CSP, запрещающая встроенные скрипты. Хранение токена в `localStorage` доступно скриптам страницы, поэтому защита от XSS (отсутствие `innerHTML` и CSP) существенна и для сохранности токена.

В строку каждой ссылки добавлена кнопка «QR», открывающая модальное окно (атрибуты `role="dialog"` и `aria-modal`). Оно создаётся из JavaScript через `createElement` и `textContent`, поэтому по-прежнему совместимо со строгой политикой CSP. Окно закрывается кнопкой «Закрыть», щелчком по фону и клавишей `Esc`, после закрытия фокус возвращается на кнопку «QR». Изображение запрашивается размером 512 пикселей и

отображается как 256 × 256 (чёткость на экранах с высокой плотностью), а кнопка «Скачать PNG» сохраняет файл `qr-<код>.png` размером 1024 пикселя.

График переходов по дням построен вручную в формате SVG, без библиотек (листинг 4.21). Пропущенные дни заполняются нулями, верх шкалы выбирается функцией «красивого» шага, высота столбца пропорциональна числу переходов, подпись дня и значение выводятся во всплывающем элементе `<title>`.

Листинг 4.21 — Построение SVG-графика

```
function drawChart(host, byDay, from, to) {
  host.textContent = "";
  var counts = {};
  byDay.forEach(function (d) { counts[String(d.day).slice(0, 10)] = d.count;
});
  // ... список дней от from до to, пропуски — нули ...
  var W = 720, H = 220, L = 38, R = 8, T = 10, B = 28;
  var root = svg("svg", { viewBox: "0 0 " + W + " " + H, role: "img", "aria-
label": "График переходов по дням" });
  // ...
  var step = niceStep(max / 4);
  var top = Math.ceil(max / step) * step;
  var plotW = W - L - R, plotH = H - T - B;
  // ... линии сетки и подписи шкалы ...
  var slot = plotW / days.length;
  var bw = Math.max(1, Math.min(28, slot * 0.7));
  days.forEach(function (d, i) {
    var h = (d.count / top) * plotH;
    var x = L + i * slot + (slot - bw) / 2;
    var bar = svg("rect", {
      x: x, y: T + plotH - h, width: bw, height: Math.max(h, d.count > 0 ? 1
: 0), rx: 2, "class": "bar"
    });
    bar.appendChild(svg("title", { text: d.key + ": " + d.count }));
    root.appendChild(bar);
    // ... подпись по оси X через каждые labelEvery столбцов ...
  });
  host.appendChild(root);
}
```

Атрибут `viewBox` делает график масштабируемым: на узких экранах он сжимается вместе с контейнером без перерисовки. Разрезы по странам, источникам, браузерам, ОС и устройствам выводятся горизонтальными полосами функцией `drawBars` с подписью числа и доли.

4.17 Контейнеризация

Образ собирается в два этапа (`Dockerfile`). Первый этап (`golang:1.27-alpine`) копирует `go.mod` и `go.sum` и загружает зависимости отдельным слоем, который переиспользуется, пока манифесты не изменились, затем собирает статический бинарный файл. Второй этап использует образ `gcr.io/distroless/static-debian12:nonroot`, в котором нет оболочки и пакетного менеджера; процесс запускается от непривилегированного пользователя (листинг 4.22).

Листинг 4.22 — Dockerfile

```
FROM golang:1.27-alpine AS build
WORKDIR /src
COPY go.mod go.sum* ./
RUN go mod download
COPY . .
RUN mkdir -p /out \
 && CGO_ENABLED=0 go build -trimpath -ldflags="-s -w" -o /out/shortener
./cmd/shortener

FROM gcr.io/distroless/static-debian12:nonroot
COPY --from=build /out/shortener /shortener
USER nonroot
EXPOSE 8080
ENTRYPOINT ["/shortener"]
```

Файл `docker-compose.yml` описывает сервисы `app`, `postgres` (образ `postgres:17-alpine`, том `pgdata`) и `redis` (образ `redis:7-alpine`, параметры `--maxmemory 128mb --maxmemory-policy allkeys-lru`). Для баз заданы проверки работоспособности `pg_isready` и `redis-cli ping` (интервал 5 с, до 10 попыток), а приложение зависит от них с условием `service_healthy` и поэтому не стартует раньше готовности (листинг 4.23). Переменная `AUTH_SECRET` объявлена обязательной: при её отсутствии Compose прекращает запуск с сообщением об ошибке.

Листинг 4.23 — Фрагмент docker-compose.yml

```
services:
  app:
    build: .
    ports:
      - "8080:8080"
```

```
    environment:
      DATABASE_URL:
postgres://shortener:shortener@postgres:5432/shortener?sslmode=disable
      REDIS_URL: redis://redis:6379/0
      AUTH_SECRET: ${AUTH_SECRET:?Задайте AUTH_SECRET в файле .env (см.
.env.example)}
      BASE_URL: http://localhost:8080
    depends_on:
      postgres:
        condition: service_healthy
      redis:
        condition: service_healthy
    restart: unless-stopped
```

4.18 Непрерывная интеграция

Конвейер GitHub Actions (`.github/workflows/ci.yml`) запускается при каждом `push` и `pull request`, права токена ограничены чтением (`contents: read`). Состав заданий приведён в таблице 4.1.

Таблица 4.1 — Задания конвейера непрерывной интеграции

Задание	Содержание	Результат
`test`	`go vet ./...`; `go test -race -coverprofile=coverage.out ./...`	отчёт о покрытии как артефакт
`integration`	сервис `postgres:17-alpine` с проверкой готовности; `go test -tags integration ./...` с `TEST_DATABASE_URL`	проверка хранилища на реальной СУБД
`lint`	`golangci-lint` (действие `golangci-lint-action@v8`) по `.golangci.yml`	замечания линтеров
`security`	`gosec ./...` и `govulncheck ./...`	SAST и поиск известных уязвимостей
`docker`	сборка образа без публикации; запускается после успешного `test`	проверка воспроизводимости сборки

Версия Go во всех заданиях берётся из `go.mod` (`go-version-file`), поэтому закрепление версии инструментальной цепочки в модуле автоматически применяется и в конвейере. Дополнительно Dependabot еженедельно предлагает обновления модулей Go, действий GitHub Actions и образов Docker.

4.19 Пользовательский интерфейс

Экран входа и регистрации (рисунок 4.1) содержит две вкладки и форму из полей email и пароля; сообщения об ошибках выводятся под формой.

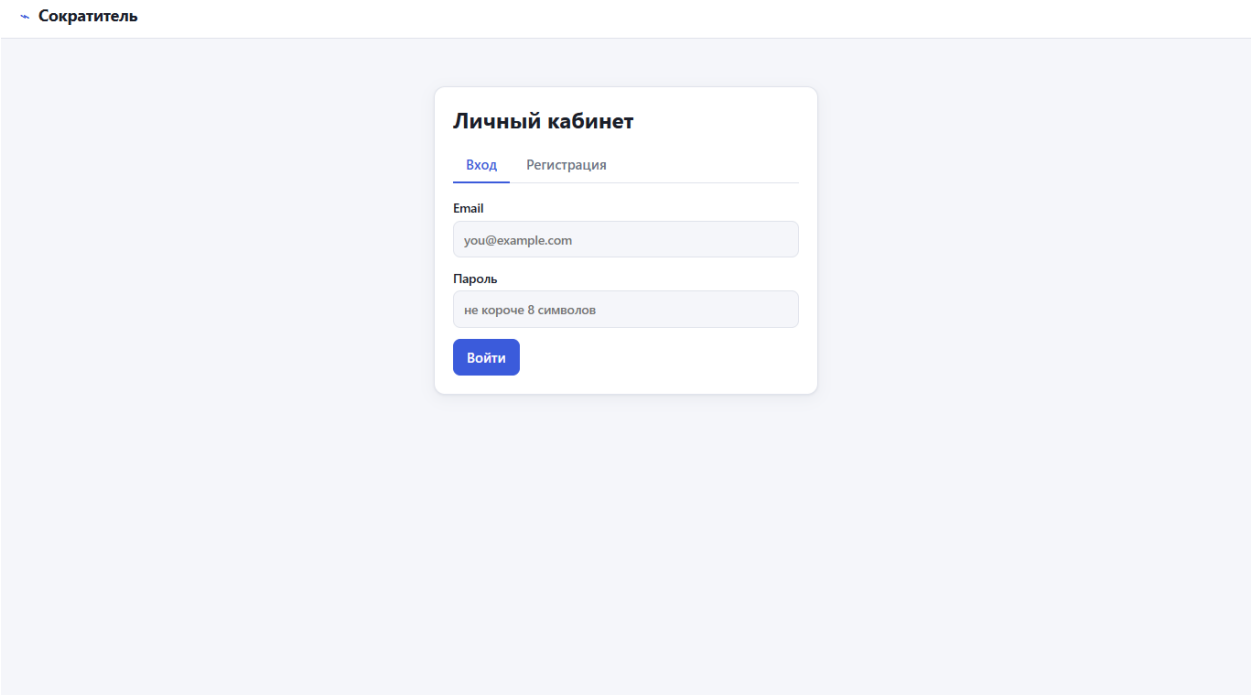


Рисунок 4.1 — Экран входа в личный кабинет

После входа открывается основной экран (рисунок 4.2): форма создания ссылки (адрес, необязательные алиас, заголовок и срок действия), таблица «Мои ссылки» с кнопками копирования, просмотра статистики и удаления и, при выборе ссылки, карточка статистики с суммарным числом переходов и уникальных посетителей, графиком по дням и рейтингами стран, источников, браузеров, ОС и устройств. Период задаётся датами или быстрыми кнопками на 7, 30 и 90 дней.

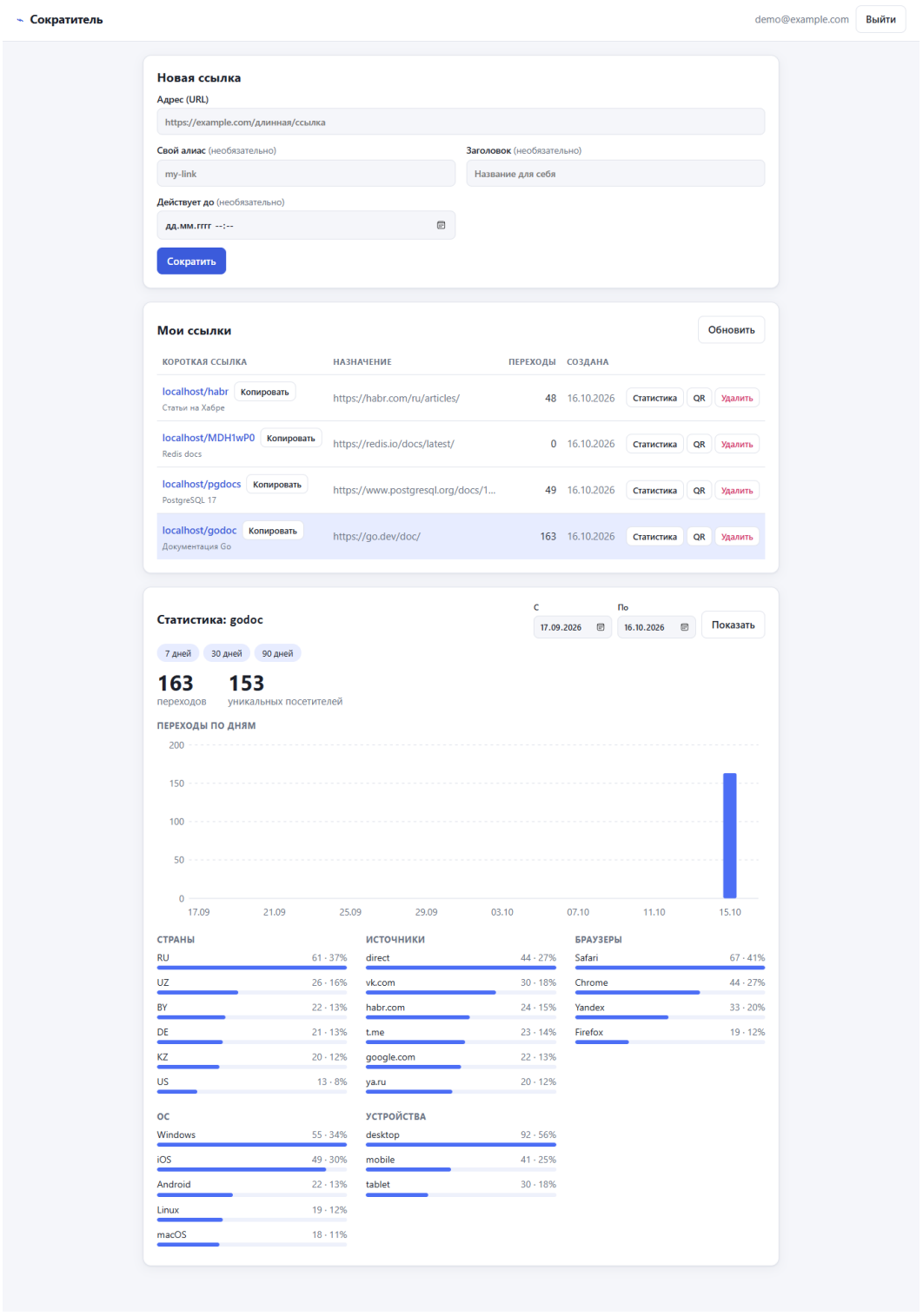


Рисунок 4.2 — Дашборд со статистикой по ссылке (светлая тема)

Тема оформления определяется настройкой системы пользователя (`prefers-color-scheme`): при тёмной теме применяется соответствующая палитра (рисунок 4.3).

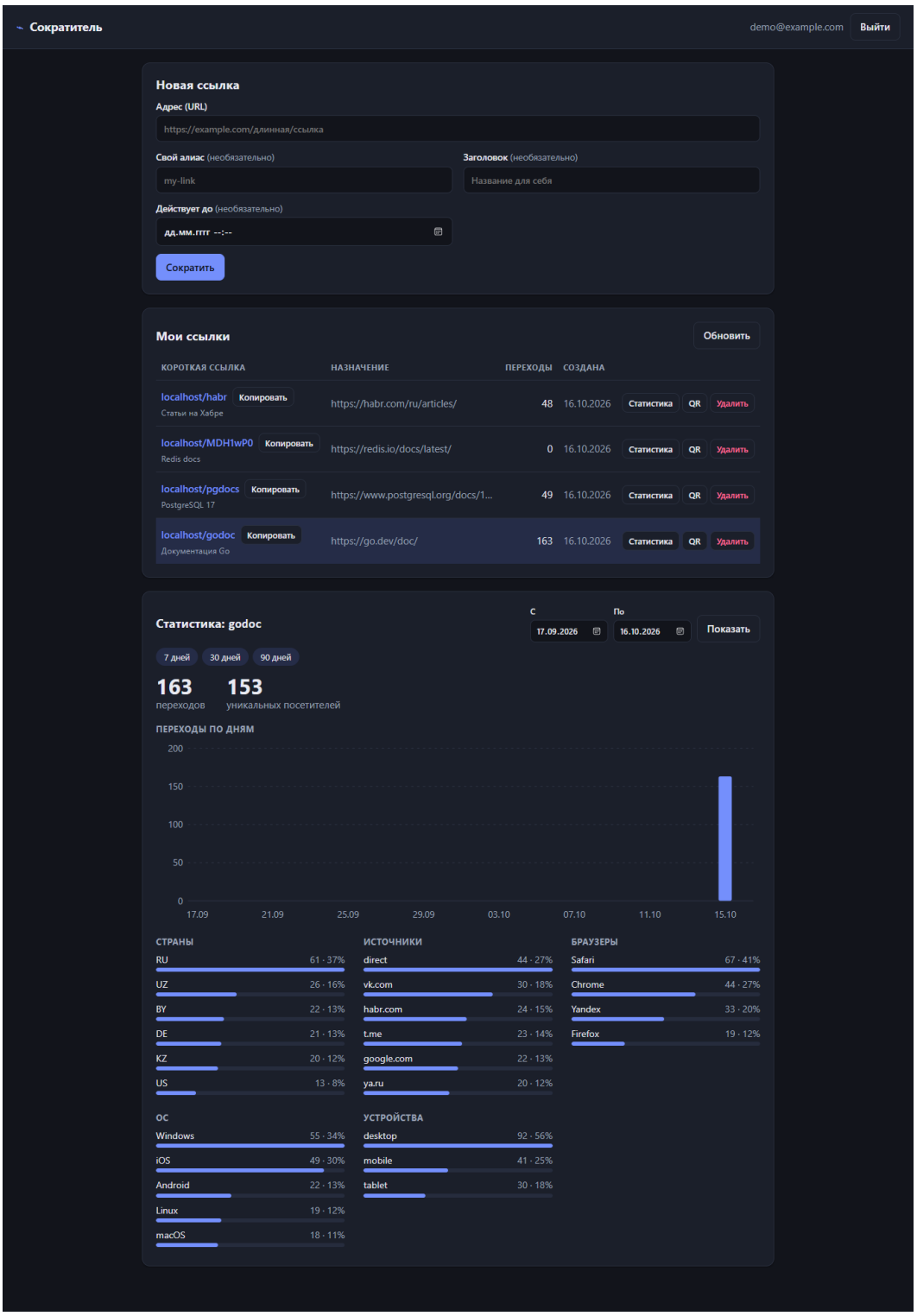


Рисунок 4.3 — Дашборд в тёмной теме

Вёрстка адаптивна: на узких экранах блоки располагаются в один столбец, график масштабируется, а таблица ссылок получает горизонтальную прокрутку (рисунок 4.4).

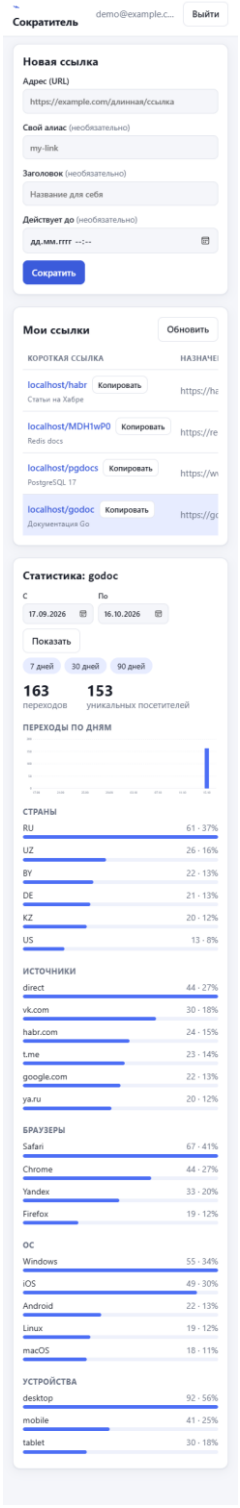


Рисунок 4.4 — Мобильная версия дашборда

Кнопка «QR» в строке ссылки открывает окно с кодом, который можно отсканировать камерой телефона или сохранить в файл (рисунок 4.5).

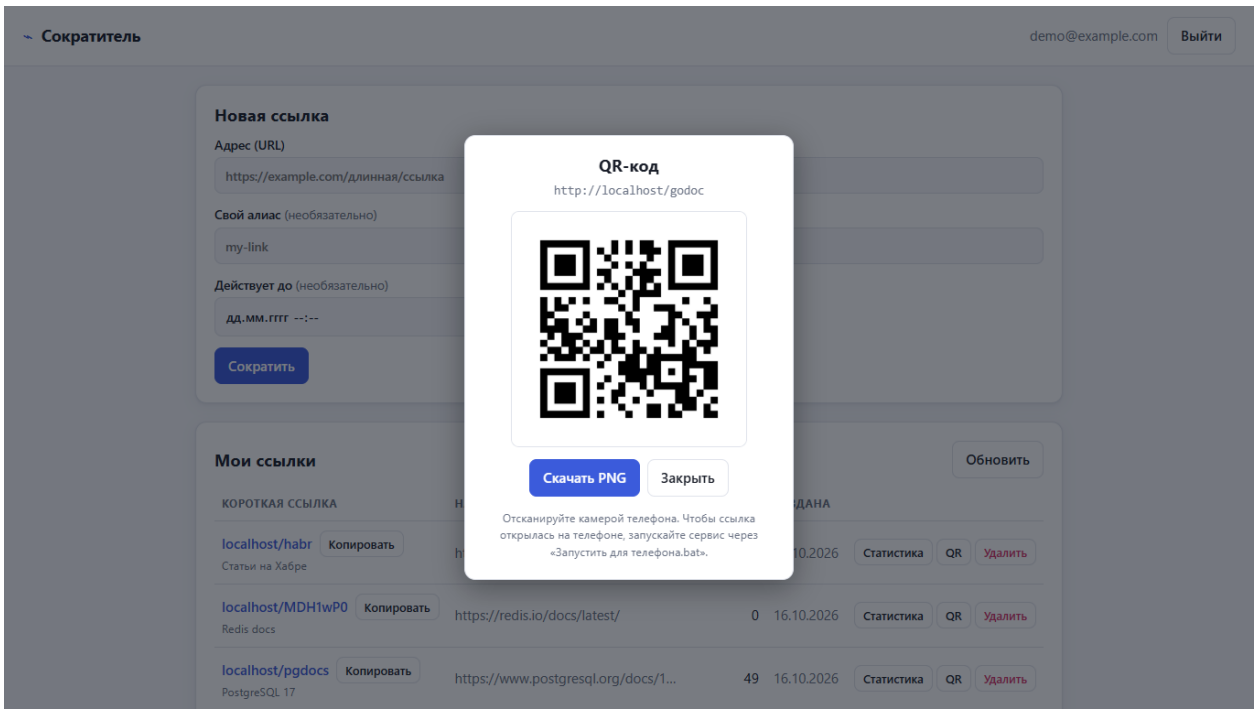


Рисунок 4.5 — Окно с QR-кодом ссылки

4.20 Выводы по главе 4

1. Реализация соответствует проекту: слои связаны интерфейсами, сборка конкретных реализаций выполнена в одной точке — `cmd/shortener/main.go`.

2. Критический путь редиректа не содержит записи в базу: запись статистики вынесена в пул воркеров с пакетной вставкой через `COPY` и дренажом при остановке.

3. Для коротких ссылок реализован QR-код: матрицу строит `rsc.io/qr`, отрисовка выполнена собственным кодом из-за двух найденных дефектов библиотеки, буферы PNG-кодировщика переиспользуются через пул (память на операцию меньше в 14 раз), а выдача QR не искажает статистику переходов.

4. Безопасность обеспечена на нескольких уровнях: ограничение частоты, подпись токенов с сравнением за постоянное время, bcrypt, параметризованные запросы, вывод через `textContent`, заголовки безопасности и строгая политика CSP.

5. Развёртывание сведено к одной команде благодаря многоэтапному образу `distroless`, Compose с проверками работоспособности и конвейеру CI из пяти заданий.

5 ТЕСТИРОВАНИЕ

5.1 Стратегия тестирования

Тестирование организовано по принципу пирамиды: основание составляет большое число быстрых модульных тестов, не требующих внешних сервисов, выше расположены интеграционные тесты хранилища на реальной СУБД, ещё выше — немногочисленные сквозные и нагрузочные проверки (рисунок 5.1).

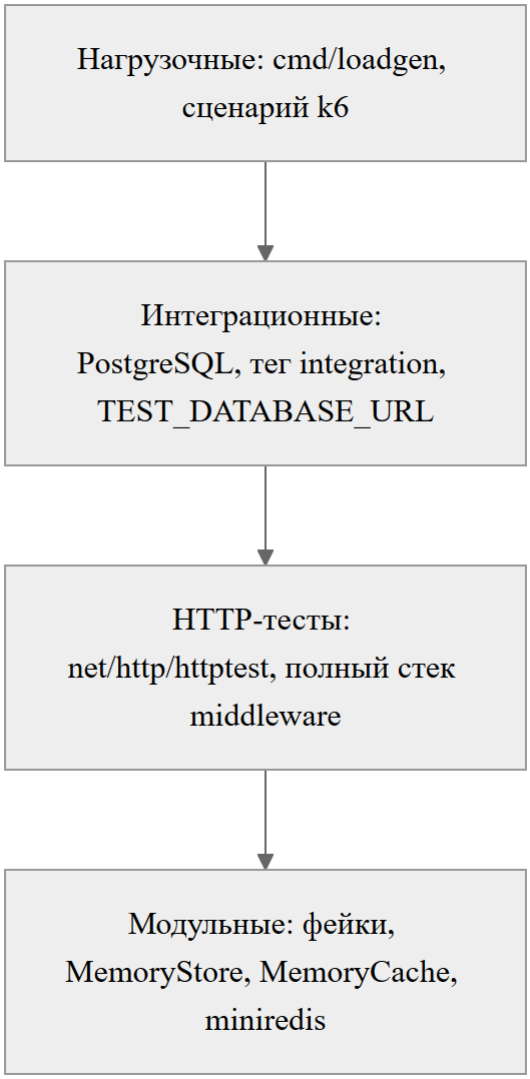


Рисунок 5.1 — Пирамида тестирования

Уровни имеют следующие особенности.

1. **Модульные тесты.** Каждый пакет тестируется изолированно. Зависимости заменяются фейками или лёгкими реализациями: хранилище в памяти (`storage/memory`), кэш в памяти, имитация Redis `miniredis` для `RedisCache`, подставные репозитории и счётчики для `Collector`. Время внедряется через поле `now`, что позволяет проверять истечение срока ссылок и токенов без пауз. Тесты на таблицах случаев (`table-driven`) охватывают граничные значения.

2. **Интеграционные тесты** хранилища PostgreSQL помечены тегом сборки `integration` и используют строку подключения из переменной `TEST_DATABASE_URL`; при её отсутствии пропускаются. В конвейере CI для них поднимается контейнер `postgres:17-alpine`. Они проверяют миграции, ограничения уникальности, `COPY`-вставку и SQL-запросы статистики, которые невозможно проверить на реализации в памяти.

3. **HTTP-тесты** используют `httptest`: поднимается полный обработчик со всеми middleware, а запросы проверяют коды ответов, заголовки и тела JSON.

4. **Бенчмарки** (`testing.B`) измеряют путь разрешения ссылки и обработчик редиректа.

5. **Нагрузочное тестирование** выполняется генератором `cmd/loadgen`; для запуска в окружении Compose подготовлен сценарий k6.

В конвейере CI модульные тесты выполняются с детектором гонок (`go test -race`), что важно для кода с каналами, блокировками и атомарными счётчиками.

5.2 Результаты модульного тестирования и покрытие

Тесты выполнены командой `go test -count=1 -cover ./...` на машине разработки (Go 1.27.1, Windows 11). Все 86 функций верхнего уровня (211 тестов с учётом подтестов) завершились успешно. Покрытие операторов по пакетам приведено в таблице 5.1.

Таблица 5.1 — Покрытие кода тестами по пакетам

Пакет	Назначение	Покрытие операторов, %
`internal/service`	бизнес-логика ссылок	100,0
`internal/storage/memory`	хранилище в памяти, снимок в файл (`persist.go`)	95,3
`internal/shortcode`	генерация кодов, алиасы	96,9
`internal/qrcode`	PNG с QR-кодом ссылки	95,8
`internal/config`	конфигурация, секрет в файле	90,2
`internal/auth`	регистрация, вход, токены	89,3
`internal/httpapi`	обработчики, middleware, rate limiting, выдача QR	88,4
`internal/analytics`	сборщик, User-Agent, реферер	85,6
`internal/cache`	`MemoryCache`, `RedisCache` (на miniredis)	81,5
`internal/storage/postgres`	хранилище PostgreSQL	0,0 при обычном запуске; тесты — в задании `integration`
`internal/model`, `cmd/shortener`, `cmd/loadgen`	структуры данных, точки входа	0,0 (тестов нет)

Суммарное покрытие по всем пакетам при обычном запуске составило 70,8 %. Если исключить пакеты без модульных тестов (`model`, `cmd/*`) и пакет `postgres`, проверяемый отдельно в интеграционном режиме, суммарное покрытие остальных пакетов равно 90,8 %. Нулевое покрытие `storage/postgres` в таблице — следствие того, что на машине разработки PostgreSQL не установлен и интеграционные тесты пропускаются (при запуске с тегом `integration` без `TEST_DATABASE_URL` они завершаются как пропущенные); покрытие этого пакета в настоящей записке не измерялось, а проверка его работоспособности выполняется заданием `integration` конвейера CI. Покрытие `storage/memory` снизилось с 99,2 до 95,3 %: добавлен код сохранения данных (`persist.go`), ветви которого, связанные с ошибками файловой системы (сбой `Sync`, `Close` и `Rename`), в тестах не воспроизводятся.

Покрытие `httpapi` снизилось с 91,0 до 88,4 % за счёт добавленного кода, часть ветвей которого (ошибки записи в соединение) не воспроизводится в тестах. Низкое покрытие `cmd/` типично для точек входа и компенсируется запуском сервиса при замерах производительности.

Следует отметить, что покрытие операторов показывает лишь, какие строки были выполнены, но не гарантирует качества проверок; поэтому в дополнение к нему тесты содержат проверки отрицательных сценариев и граничных значений (таблица 5.2).

5.3 Ключевые тест-кейсы

Перечень основных проверок по пакетам приведён в таблице 5.2.

Таблица 5.2 — Ключевые тест-кейсы

Пакет	Что проверяется	Ожидаемый результат
`shortcode`	длина и алфавит кодов (`TestRandomLengthAndAlphabet`)	код заданной длины из символо `0-9A-Za-z`
`shortcode`	отсутствие коллизий на большой выборке (`TestRandomNoCollisions`)	повторов нет
`shortcode`	равномерность символов, 50 000 кодов (`TestRandomUniform`)	частота каждого символа в пределах ±10 % от ожидаемой
`shortcode`	алиасы: длина, допустимые символы, начало с `-`/`_`, зарезервированные (`TestValidateAlias`)	соответствующие ошибки, допустимые алиасы принимаю
`service`	создание ссылки: нормализация схемы и хоста, пустой и слишком длинный адрес, `ftp`, `javascript:`, адрес без хоста, адрес на собственный хост (в том числе с портом), `expires_at` в прошлом (`TestCreate`)	`ErrInvalidURL` либо нормализованный адрес
`service`	коллизии кода при генерации (`TestCreate_Collisions`)	повторная генерация; после пят неудач — ошибка
`service`	занятый алиас (`TestCreate_AliasTaken`)	`ErrAliasTaken`
`service`	кэш только для популярных ссылок (`TestResolve_CacheOnlyWhenPopular`)	запись попадает в кэш после по обращений
`service`	TTL кэша не превышает оставшийся срок ссылки (`TestResolve_CacheTTL`)	TTL = min(`CacheTTL`, остаток срока)
`service`	отказ кэша (`TestResolve_DegradedCache`)	редирект работает из хранилищ
`service`	просроченная запись в кэше (`TestResolve_CachedEntryExpired`)	`ErrGone`

Пакет	Что проверяется	Ожидаемый результат
service	повторное удаление (TestDelete_Twice), ошибка кэша при удалении	ErrNotFound; ошибка кэша не ломает удаление
service	недопустимый код не доходит до хранилища (TestResolve_InvalidCodeSkipsStore)	ErrNotFound без обращения к хранилищу
service	hostOnly для example.com:8080, [::1]:8080, [::1] (TestHostOnly)	хост без порта и скобок
auth	регистрация, вход, дубликат, слабый пароль, неверные учётные данные	успех / ErrEmailTaken / ErrWeakPassword / ErrInvalidCredentials
auth	токен: подмена полезной нагрузки, чужой ключ, пустой, без разделителя, мусор, срок действия (TestToken)	ErrInvalidToken; токен действителен до момента истечения и недействителен по
analytics	разбор User-Agent и реферера, порядок проверок браузеров	правильные браузер, ОС, устройство, домен источника
analytics	обогащение события (TestEnrich)	страна из подсказки, из GeoIP и ZZ; VisitorID из 16 символов зависит от IP
analytics	пакетирование по размеру и по таймеру	пакеты записываются при достижении BatchSize и по FlushInterval
analytics	дренаж при остановке (TestCollectorDrainOnStop)	записаны все 100 событий; пос остановки Track отбрасывает событие
analytics	переполнение очереди (TestCollectorDropsOnOverflow)	Accepted = 2, Dropped = 3 пр буфере 2 и пяти событиях
analytics	сбой записи (TestCollectorWriteFailure)	растёт Failed, Written = 0
cache	MemoryCache и RedisCache (miniredis): Get/Set/Delete, TTL, окно счётчика, топ	идентичное поведение двух реализаций
config	значения по умолчанию, обязательность AUTH_SECRET при DATABASE_URL, некорректные значения	корректная конфигурация либо агрегированная ошибка
storage/memory	создание, копии, список, деактивация, пользователи, счётчики, статистика, конкурентный доступ (TestConcurrent)	корректные данные; отсутствие гонок
storage/memory	круговая проверка снимка (TestPersistentRoundTrip): пользователь с хэшем пароля, ссылки с expires_at и названием, отключённая ссылка, переходы и уникальные посетители	после закрытия и повторного открытия данные совпадают; х пароля и visitor_id сохране нумерация ID продолжается; ко email остаются занятыми
storage/memory	фоновое сохранение (TestPersistentAutosave)	файл появляется без вызова Cl временные файлы не остаются каталоге
storage/memory	отсутствие изменений (TestPersistentNoChangesNoWrite)	при неизменной версии данных файл не перезаписывается

Пакет	Что проверяется	Ожидаемый результат
`storage/memory`	повреждённый файл (`TestPersistentCorruptedFile`, 2 случая: некорректный JSON и неизвестная версия формата)	`Open` возвращает ошибку; содержимое файла не изменяет
`storage/memory`	конкурентные изменения при параллельных `Save` (`TestPersistentConcurrent`, 8 горутин по 50 изменений)	после загрузки ровно 400 переходов; отсутствие гонок
`storage/memory`	ошибка записи (`TestPersistentSaveError`)	`Close` возвращает ошибку при несуществующем каталоге
`config`	секрет в файле (`TestDataFileSecretPersists`)	секрет создаётся при первом запуске и тот же читается при втором; явный `AUTH_SECRET` приоритетнее файла
`config`	ошибки `DATA_FILE` (`TestDataFileErrors`, 3 случая)	ошибка при `DATA_FILE` вместе `DATABASE_URL`; ошибка при секрете короче 32 байт; ошибк записи секрета в несуществую каталог
`httpapi`	регистрация и вход, создание ссылки, слишком большое тело (`TestBodyTooLarge`)	201 / 200 / 413
`httpapi`	редирект, 404 и 410 (`TestRedirect*`)	`302` с `Location` и `Cache-Control`; 404; 410
`httpapi`	чужие ссылки (`TestListGetDeleteOwnership`)	`404` на чужую ссылку
`httpapi`	запрос без токена (`TestUnauthorized`)	`401` с `WWW-Authenticate`
`httpapi`	rate limiting (`TestRateLimit`)	при burst = 3 коды 401, 401, 40 429, 429; заголовок `Retry-Aft` `/healthz` не лимитируется
`service`	проверка активности ссылки без учёта перехода (`TestCheckActive`, 9 случаев, и `TestCheckActive_StoreError`)	активная — без ошибки; несуществующая и недопустим код — `ErrNotFound`; удалённа истёкшая (до/после срока, из к — `ErrGone`; при отказе кэша проверка выполняется по хранилищу; ошибка хранилищ возвращается как есть
`qrcode`	круговая проверка: PNG расшифровывается независимым декодером gozxing (`TestPNGRoundTrip`, 3 URL, включая 32-символьный алиас)	декодированный текст совпада исходным
`qrcode`	размеры и квадратность (`TestPNGSize`); приведение размера к диапазону 64–1024; малые размеры (масштаб модуля 1)	картинка квадратная, сторона н больше запрошенной; для мал размеров паники нет

Пакет	Что проверяется	Ожидаемый результат
qrcode	слишком длинный текст (TestPNGTooLong)	возвращается ошибка
httpapi	выдача QR-кода (TestQRCode): статус, тип, расшифровка	200, image/png, QR расшифровывается в http://sho.rt/promo
httpapi	QR не считается переходом	событие аналитики не поставле очередь; счётчик shortener_qr_generated_t вырос на 1
httpapi	ошибки QR (TestQRCodeErrors, таблица случаев)	404 — нет кода, недопустимый код, нет расширения .png; 410 ссылка удалена; 400 — size н число, 63 и 1025; size=64 принимается
httpapi	заголовки безопасности и дашборд (TestDashboardAndHeaders)	nosniff, DENY, CSP с default src 'self', X-Request-Id; корень перенаправляет на /dashboard/
httpapi	/healthz и /metrics	200 / 503 при отказе зависимос счётчики в формате Prometheus
httpapi	определение IP (TestClientIP)	X-Forwarded-For учитывается только при доверии; мусор в заголовке игнорируется
storage/postgres (интеграционные)	идемпотентность миграций, пользователи и ссылки, ограничения уникальности, статистика	результаты совпадают с реализацией в памяти

5.4 Дефекты, найденные тестами

В ходе тестирования выявлены и исправлены следующие дефекты.

1. **Повторное удаление ссылки возвращало успех вместо 404.** Метод Deactivate в хранилищах (в памяти и PostgreSQL) не учитывал флаг is_active: для уже деактивированной ссылки операция повторно «успешно» обновляла строку. Исправление: в PostgreSQL условие WHERE дополнено AND is_active (операция затрагивает ноль строк и возвращает ErrNotFound), в хранилище в памяти добавлена аналогичная проверка. Дефект обнаружен тестом TestDelete_Twice, а в слое хранилища — тестами TestDeactivate.

2. **Проверка ссылки на собственный хост не работала для IPv6-адресов.** Функция hostOnly, выделяющая имя хоста из BASE_URL, не снимала

квадратные скобки, поэтому для `[::1]:8080` возвращалось `[::1]`, тогда как `url.URL.Hostname()` для проверяемого адреса возвращает `::1`; сравнение не срабатывало, и можно было создать ссылку на сам сервис. Исправление: реализация заменена вызовом `(&url.URL{Host: h}).Hostname()`; случаи `[::1]:8080` и `[::1]` добавлены в `TestHostOnly`.

3. **Конфликт имён методов `Create` во встроенных интерфейсах.** Интерфейс `Store` встраивает `LinkRepository` и `UserRepository`; оба объявляли метод `Create` с разными сигнатурами, из-за чего контракт не компилировался. Метод создания пользователя переименован в `CreateUser`. Дефект выявлен на этапе проверки контрактов интерфейсов при сборке пакетов, разработанных независимо, и подтверждает ценность фиксации интерфейсов до реализации (раздел 3.7).

4. **Аварийное завершение PNG-кодировщика `rsc.io/qr` при масштабе 1.** Встроенный метод `Code.PNG()` паникует с сообщением «invalid flate repeat», если масштаб модуля равен 1, то есть для небольших картинок. Дефект выявлен модульным тестом `TestPNGSize` (подтест «меньше минимума приводится к минимуму»): при размере 64 пикселя и коде из 29 модулей с рамкой масштаб равен 1. Решение: от библиотеки используется только матрица модулей (`qr.Encode`, `Code.Black`), PNG формируется собственной растеризацией через стандартный `image/png`.

5. **Метод `Code.Image()` библиотеки `rsc.io/qr` игнорирует масштаб и белую рамку.** Границы изображения вычисляются верно (`(Size + 8) · Scale`), но метод `At` обращается к модулю `(x, y)` напрямую, не деля координаты на `Scale` и не вычитая рамку. В результате при масштабе больше 1 QR-код рисуется в левом верхнем углу в натуральную величину, а остальная часть картинки остаётся белой. Дефект обнаружен при анализе исходного кода библиотеки, когда искался обход дефекта 4. Решение то же: собственная отрисовка с целым масштабом и рамкой; корректность подтверждена расшифровкой результата независимым декодером.

Оба дефекта библиотеки показывают ценность проверки результата независимым средством: тест, который лишь сравнивал бы размеры или сигнатуру PNG, не обнаружил бы нечитаемый код.

5.5 Статический анализ и безопасность

go vet. Команда `go vet ./...` не выдала замечаний.

govulncheck. Проверка выявила 13 уязвимостей стандартной библиотеки версии go1.27.1. Десять из них (GO-2026-6603, GO-2026-6605, GO-2026-6607, GO-2026-6608, GO-2026-6609, GO-2026-6610, GO-2026-6611, GO-2026-6612, GO-2026-6613, GO-2026-6617) достижимы из кода проекта через пакеты `net/http`, `net/http/internal/http2`, `crypto/tls` и `net/textproto`. Ещё одна (GO-2026-6604, пакет `os`) находится в импортируемом пакете, а две (GO-2026-6599, GO-2026-6600) — в модуле стандартной библиотеки, но уязвимые функции проектом не вызываются. Все 13 исправлены в версии go1.27.2. Уязвимости устранены фиксацией директивы `toolchain go1.27.2` в `go.mod`: при установленном go1.27.1 команда `go` автоматически переключается на исправленную инструментальную цепочку. Дополнительно конвейер CI устанавливает последний патч ветки 1.27 (`go-version: "1.27.x"`, `check-latest: true`). Повторный запуск `govulncheck` на go1.27.2 показал 0 уязвимостей, достижимых из кода проекта, и 0 — в импортируемых пакетах. Это показывает практическую ценность проверки: уязвимости находились не в зависимостях проекта, а в самой стандартной библиотеке, и закрываются обновлением инструментальной цепочки.

Дополнительно обнаружена одна уязвимость в модуле `golang.org/x/crypto` (GO-2026-5932). Исправленной версии на момент проверки не существует, при этом анализ достижимости показал, что уязвимый код проектом не вызывается. Риск принят; обновления модуля отслеживаются Dependabot, а проверка `govulncheck` выполняется в каждом прогоне конвейера CI.

gosec. Анализатор безопасности `gosec` проверил 28 файлов (3858 строк, включая пакеты `qrcode` и сохранение данных) и не выявил ни одной проблемы; подавлений через `#nosec` в коде нет. После добавления сохранения данных первый прогон выдал два замечания G304 («чтение файла по пути из переменной») для чтения файла данных и файла секрета, путь к которым задаётся через `DATA_FILE`. Они устранены нормализацией пути функцией `filepath.Clean`, а не подавлением правила: путь задаёт администратор через окружение, и его очистка убирает неоднозначные элементы (`..`, повторные разделители) до обращения к файловой системе. Первый запуск показал, что опубликованная сборка `gosec` использует библиотеку `golang.org/x/tools` v0.49.0, не поддерживающую формат экспорта типов Go 1.27 («export data version 5 is greater than maximum supported version 4»). Из-за этого анализатор пропускал построение SSA-представления пакетов и формально сообщал об отсутствии проблем, фактически не выполнив межпроцедурные проверки. Такой «ложно-зелёный» результат опасен, поэтому `gosec` был пересобран с `golang.org/x/tools` v0.52.0, после чего анализ прошёл без ошибок построения типов. Тот же способ сборки закреплён в задании `security` конвейера CI.

golangci-lint. Набор линтеров (таблица 3.4) выполняется в задании `lint` конвейера CI; локально — командой `make lint`. Первый прогон в CI выявил 16 замечаний, исправленных затем в коде:

– `errcheck` — не проверялись ошибки записи метрик и закрытия тела ответа в тестах;

– `bodyclose` — тестовая функция возвращала `*http.Response` с уже закрытым телом, что анализатор принимал за утечку; она заменена на структуру со статусом и заголовками;

– `staticcheck` (SA1019) — использовался устаревший метод `ZRevRange` клиента Redis, заменён на `ZRangeArgs` с параметром `Rev`;

– `revive` (`unused-parameter`) — 10 замечаний о неиспользуемых параметрах `ctx` и `*http.Request`. Эти параметры обязательны: методы

реализуют интерфейсы `storage.Store` и `http.HandlerFunc`. Поэтому правило отключено в `.golangci.yml` с поясняющим комментарием.

Ещё один прогон после добавления сохранения данных выявил три замечания `errcheck` в тестах (необработанные ошибки закрытия и записи) и одно замечание `gocritic` (`ifElseChain`) в `main.go`: цепочка `if-else` при выборе хранилища заменена на `switch`. Все исправлены; итоговый результат — 0 замечаний.

После добавления функции QR-кодов повторные запуски `golangci-lint`, `gosec` и `govulncheck` дали 0 замечаний, 0 проблем и 0 достижимых уязвимостей соответственно. После исправлений все задания конвейера (`test`, `integration`, `lint`, `security`, `docker`) завершаются успешно. Задание `integration` выполняет интеграционные тесты хранилища на PostgreSQL 17, а `docker` собирает образ приложения.

5.5.1 Угрозы и меры

Соответствие угроз по OWASP [17], принятых в разделе 1.4.3, реализованным мерам и проверяющим их тестам показано в таблице 5.3.

Таблица 5.3 — Угрозы, меры защиты и их проверка

Угроза	Мера в реализации	Проверка
Открытое перенаправление [18]	допускаются только схемы `http` и `https`; запрещены ссылки на собственный хост (с учётом порта и IPv6); адрес обязательно создаёт аутентифицированный владелец; ссылку можно деактивировать; rate limiting на создание	`TestCreate` (схемы `ftp`, `javascript:`, собственный хост), `TestHostOnly`
Межсайтовый скриптинг (XSS) в дашборде	вывод пользовательских данных только через `textContent`, `innerHTML` не применяется; CSP `script-src 'self'`; `X-Content-Type-Options: nosniff`	`TestDashboardAndHeaders`; просмотр исходного кода `app.js`

Угроза	Мера в реализации	Проверка
Перебор паролей	rate limiting по IP на `/auth/login` и `/auth/register`; bcrypt со стоимостью 12; единое сообщение об ошибке и выравнивание времени ответа при неизвестном email	`TestRateLimit`, `TestLoginFailures`
SQL-инъекции	параметризованные запросы pgx; запросы разрезов — константы	просмотр кода, интеграционные тесты
Подделка токена	подпись HMAC-SHA256, сравнение `hmac.Equal`, срок действия; подпись проверяется до разбора полезной нагрузки	`TestToken` (подмена, чужой ключ, срок)
Доступ к чужим ссылкам	проверка `owner_id`; чужая ссылка неотличима от отсутствующей	`TestListGetDeleteOwnership`
Подмена IP и страны	`X-Forwarded-For` и `CF-IPCountry` учитываются только при `TRUST_PROXY=true`	`TestClientIP`
Большие тела запросов	ограничение 64 КБ (`http.MaxBytesReader`)	`TestBodyTooLarge`
Перегрузка очереди аналитики	неблокирующая постановка, счётчик потерь	`TestCollectorDropsOnOverflow`
Раскрытие информации через публичный адрес QR	в QR записана только короткая ссылка, которая и так открыта; код и размер проверяются до обращения к генератору, размер ограничен 64–1024 пикселями (защита от запросов огромных изображений); удалённые и истёкшие ссылки получают `410`	`TestQRCodeErrors`
Уязвимые зависимости	`govulncheck`, `gosec`, Dependabot в CI	раздел 5.5

Файл данных демо-режима содержит хэши паролей, а файл секрета — ключ подписи токенов, поэтому оба создаются с правами 0600 (запись во временный файл, затем атомарное переименование), добавлены в `.gitignore`

и исключены из репозитория; повреждённый файл не перезаписывается (раздел 4.13).

Остаточные риски: ограничение частоты хранится в памяти процесса и не действует на редиректы; токен не отзывается до истечения срока действия и хранится в `localStorage`; при регистрации ответ `409` раскрывает факт существования адреса; сервис не проверяет адреса по базам вредоносных ресурсов. Эти ограничения перечислены в заключении как направления развития.

5.5.2 Сквозная проверка сохранения данных при жёстком завершении

Модульные тесты проверяют сохранение при корректном закрытии хранилища, но главный сценарий — закрытие окна консоли, при котором `Close` не вызывается. Он проверен на собранном `shortener.exe` в Windows. Порядок проверки:

1. Запуск с `DATA_FILE=data.json`; регистрация пользователя, создание ссылки с кодом `persist`, пять переходов по ней.

2. Жёсткое завершение процесса командой `Stop-Process -Force`, эквивалентное закрытию окна: обработчики остановки не выполняются, итоговое сохранение не происходит.

3. Повторный запуск с теми же переменными окружения.

Результат: токен, выданный до завершения, принят без повторного входа (секрет сохранён в `data.json.secret`); вход по паролю работает; статистика ссылки показывает 5 переходов, а счётчик `click_count` равен 5. Это подтверждает, что фоновое сохранение с задержкой 0,5 с записало данные на диск до завершения процесса, а файл остался целым несмотря на отсутствие корректной остановки.

5.6 Производительность

5.6.1 Стенд

Замеры выполнены реальными запусками на стенде, описанном в таблице 5.4. Каждый замер выполнен один раз, без усреднения по нескольким запускам; значения зависят от загрузки машины.

Таблица 5.4 — Стенд для замеров производительности

Параметр	Значение
Процессор	AMD Ryzen 5 5600H with Radeon Graphics, 6 ядер / 12 потоков, максимальная частота 3301 МГц
Оперативная память	15,4 ГБ
Операционная система	Microsoft Windows 11 Pro 10.0.26300
Go	go1.27.1 windows/amd64
Режим сервиса	демонстрационный: хранилище и кэш в памяти, без PostgreSQL и Redis
Нагрузочный клиент	`cmd/loadgen` на той же машине, через loopback (127.0.0.1)

Сервис и генератор нагрузки работают на одной машине и делят ядра процессора; журнал доступа (JSON, по строке на запрос) писался в файл. Поэтому абсолютные значения RPS следует рассматривать как оценку снизу для данной конфигурации, а не как предел самого приложения.

5.6.2 Микробенчмарки

Команда запуска: `go test -run='^$' -bench=. -benchmem ./internal/service/ ./internal/httpapi/`. Суффикс `-12` в именах означает `GOMAXPROCS=12`; варианты `Parallel` используют `b.RunParallel`. Результаты для сервисного слоя (хранилище — `memory.New()`, кэш — `cache.NewMemory()`, генератор — `shortcode.NewRandom(7)`, трекер — заглушка) приведены в таблице 5.5.

Таблица 5.5 — Бенчмарки сервисного слоя

Бенчмарк	Итераций	нс/оп	Б/оп	аллокаций/оп
`ResolveCacheHit` (попадание в кэш)	21 291 582	57,74	32	1
`ResolveCacheHitParallel`	9 499 104	135,1	32	1
`ResolveCacheMiss` (промах, чтение из хранилища в памяти)	8 512 551	135,4	144	2
`ResolveCacheMissParallel`	13 014 986	99,52	144	2
`CreateRandomCode`	1 000 000	2927	610	5
`CreateRandomCodeParallel`	1 000 000	1957	611	5

Разрешение закэшированной ссылки в сервисном слое занимает около 58 нс и одну аллокацию. У варианта `ResolveCacheHitParallel` время на операцию выше, чем у последовательного (135 против 58 нс): все горутины конкурируют за общую блокировку кэша в памяти; для Redis конкуренция происходит на уровне сети и сервера Redis, поэтому этот результат относится только к реализации в памяти. Создание ссылки со случайным кодом (около 3 мкс) определяется обращением к `crypto/rand` и аллокациями.

Бенчмарки HTTP-слоя измеряют полный стек: middleware (идентификатор запроса, восстановление после паники, журнал доступа, заголовки безопасности), маршрутизатор, `Resolve` и ответ `302`; хранилище и аутентификатор — фейки из тестов (таблица 5.6).

Таблица 5.6 — Бенчмарки HTTP-слоя

Бенчмарк	Итераций	нс/оп	Б/оп	аллокаций/оп
`RedirectHandlerCacheHit` (`ResponseRecorder`, без сети)	132 022	10 255	7202	33
`RedirectHandlerCacheHitParallel`	217 141	6330	7229	33
`RedirectHandlerCacheMiss` (кэш всегда промахивается)	133 950	9134	7314	34
`RedirectHTTPLoopback` (`httptest.NewServer`, параллельный клиент)	39 334	33 408	8410	95

Сопоставление таблиц показывает, что работа самой бизнес-логики (около 58 нс при попадании в кэш) составляет доли процента от стоимости обработки запроса в HTTP-слое (около 10 мкс без сети и около 33 мкс через

loopback). Накладные расходы стека `net/http`, middleware, формирования записи журнала доступа и аллокаций (около 33 на запрос) доминируют, поэтому оптимизация алгоритма кэширования сама по себе почти не влияет на пропускную способность. Различие между попаданием и промахом на уровне HTTP (10,3 против 9,1 мкс) лежит в пределах погрешности однократного замера.

Для пакета `qrcode` выполнен бенчмарк `BenchmarkPNG` (картинка 256 пикселей, на том же стенде). Первая версия создавала кодировщик `image/png` на каждый вызов, и каждый вызов заново выделял около 1 МБ буферов сжатия; после перехода на пул буферов (`png.EncoderBufferPool` на основе `sync.Pool`) результаты изменились так, как показано в таблице 5.7.

Таблица 5.7 — Бенчмарк генерации QR-кода (256 px) до и после пула буферов

Версия	мс/оп	Б/оп	аллокаций/оп
без пула буферов	1,18	1 247 111	61
с пулом буферов	0,985	86 742	34

Выделяемая память на операцию уменьшилась в 14 раз, число выделений — почти вдвое, время — примерно на 17 %. Для сервиса это означает меньшую нагрузку на сборщик мусора при потоке запросов QR; абсолютная стоимость (менее 1 мс) пренебрежимо мала по сравнению с сетевой задержкой, к тому же ответ допускает кэширование (`Cache-Control: public, max-age=3600`).

5.6.3 Нагрузочный тест

Сервис собран и запущен в демонстрационном режиме (`HTTP_ADDR=127.0.0.1:8091`), генератор — командой `loadgen -url http://127.0.0.1:8091/<код> -c <C> -d 10s`. Перед замерами зарегистрирован пользователь, создана ссылка (`POST /api/v1/links`) и

выполнен прогрев — 10 запросов `GET /<код>`, все ответы `302`. Результаты для четырёх уровней конкурентности приведены в таблице 5.8.

Таблица 5.8 — Результаты нагрузочного теста редиректа (по 10 с на уровень)

c	Запросов	RPS	p50	p90	p95	p99	max	Ошибки транспорта
1	29 372	2937,1	520,9 мкс	536,1 мкс	555 мкс	651,7 мкс	29,92 мс	0
16	56 103	5609,0	2,690 мс	3,334 мс	3,766 мс	4,808 мс	10,26 мс	0
64	56 712	5665,4	11,17 мс	12,82 мс	13,50 мс	14,89 мс	28,46 мс	0
256	56 181	5595,1	45,18 мс	50,99 мс	52,68 мс	55,72 мс	103,6 мс	0

Все запросы получили ответ `302` (статусов, отличных от 302, не зафиксировано). По счётчикам `/metrics` после всех прогонов: редиректов 198 378, принято событий аналитики 198 378, отброшено 0; записано 196 863 на момент снятия метрик, остаток находился в очереди сборщика. Отсутствие отброшенных событий при пропускной способности около 5,6 тыс. запросов в секунду подтверждает, что буфер на 10 000 событий и пакетная запись справляются с потоком переходов.

Анализ. Пропускная способность перестаёт расти уже при c = 16 и остаётся около 5,6 тыс. запросов в секунду при дальнейшем увеличении конкурентности в 16 раз. При этом задержка растёт пропорционально числу одновременных запросов, что описывается законом Литтла: среднее число запросов в системе $N = \lambda \cdot W$. Для c = 256 и $\lambda \approx 5595$ запросов в секунду среднее время ответа $W \approx 256 / 5595 \approx 45{,}8$ мс, что согласуется с измеренным p50 = 45,18 мс; аналогично для c = 64 получаем 11,3 мс против 11,17 мс, для c = 16 — 2,85 мс против 2,69 мс. Следовательно, система перешла в режим насыщения: дополнительные запросы лишь ожидают в очереди, не увеличивая пропускной способности. Рост p99 при c = 256 (55,7 мс) также отражает очередь, а не деградацию обработки отдельного запроса.

Вероятными ограничителями являются следующие. Во-первых, синхронная запись JSON-строки журнала доступа на каждый запрос (в файл), выполняемая в потоке обработки; для служебного журнала в продуктивном окружении её целесообразно буферизовать или отключать для редиректов. Во-вторых, генератор нагрузки работает на той же машине и делит с сервисом ядра процессора и сетевой стек loopback; на Windows накладные расходы TCP-цикла существенны, чем объясняется задержка около 0,5 мс на запрос при c = 1. Микробенчмарки (таблицы 5.5 и 5.6) показывают, что чистая работа приложения на один запрос занимает единицы–десятки микросекунд, то есть на порядки меньше наблюдаемых задержек. Какой именно из факторов является основным, данными замерами не установлено, профилирование (`pprof`) не выполнялось. Тестирование с PostgreSQL и Redis, а также сценарием k6 (`docs/loadtest/k6-redirect.js`, порог p95 < 50 мс при 200 виртуальных пользователях) в рамках этих замеров не проводилось; результаты для реальных зависимостей будут отличаться: попадание в Redis добавляет сетевое обращение, а промах — запрос к PostgreSQL по уникальному индексу.

Требование НФТ-1 (разрешение популярной ссылки без обращения к основной СУБД, обработка на стороне сервиса за единицы миллисекунд) подтверждается в части, касающейся сервиса: при попадании в кэш обработка занимает десятки микросекунд, а при $c \leq 64$ задержки по p99 не превышают 15 мс при клиенте на той же машине.

5.7 Выводы по главе 5

1. Применена многоуровневая стратегия: модульные тесты с фейками и `miniredis`, интеграционные тесты PostgreSQL под тегом `integration`, HTTP-тесты через `httptest`, бенчмарки и нагрузочный генератор. Все 86 функций тестов (211 с подтестами) проходят, в том числе круговая проверка QR-кодов независимым декодером.

2. Покрытие операторов составляет от 81,5 до 100 % по пакетам с модульными тестами (90,8 % суммарно по ним; 70,8 % по всем пакетам); пакет PostgreSQL проверяется интеграционными тестами в конвейере CI. Сохранение данных демо-режима проверено модульными тестами (круговая проверка, повреждённый файл, конкурентные записи) и сквозным сценарием с жёстким завершением процесса.

3. Тестами выявлены и исправлены три дефекта собственного кода (повторная деактивация ссылки, проверка собственного хоста для IPv6, конфликт имён методов интерфейсов) и обойдены два дефекта сторонней библиотеки `rsc.io/qr` (авария PNG-кодировщика при масштабе 1 и игнорирование масштаба и рамки в `Image()`).

4. `go vet` и `gosec` (28 файлов, 3858 строк) не выдали замечаний, а замечания G304 при добавлении сохранения данных устранены нормализацией путей без подавлений; `govulncheck` выявил 13 уязвимостей стандартной библиотеки go1.27.1, устранённых переходом на go1.27.2, и одну уязвимость `golang.org/x/crypto`, не затрагивающую проект (риск принят и отслеживается).

5. Редирект на стенде при клиенте на той же машине достигает около 5,6 тыс. запросов в секунду, задержка растёт согласно закону Литтла; попадание в кэш в сервисном слое занимает около 58 нс, и в общих затратах доминирует стек HTTP.

ЗАКЛЮЧЕНИЕ

В ходе курсовой работы разработан и реализован веб-сервис сокращения ссылок с аналитикой переходов на стеке Go, PostgreSQL и Redis. Сервис развёртывается одной командой Docker Compose, не зависит от сторонних облачных платформ и хранит данные на инфраструктуре владельца. Результаты по задачам, поставленным во введении, состоят в следующем.

1. **Предметная область и аналоги.** Исследован принцип работы сервисов сокращения ссылок, выбрано временное перенаправление 302 с запретом кэширования (`Cache-Control: private, max-age=0`), которое обеспечивает учёт каждого перехода и возможность отключения ссылки. Рассмотрены семь аналогов; установлено, что коммерческие решения закрыты и хранят данные у третьей стороны, а открытые используют иные стеки (глава 1).

2. **Генерация кодов.** Проанализированы способы генерации и оценена ёмкость пространства кодов. Выбран случайный код base62 длиной 7 символов на `crypto/rand` с отсечением смещённых значений: ёмкость составляет около $3{,}5 \cdot 10^{12}$, вероятность коллизии при вставке в таблицу из 10^7 записей — около $2{,}8 \cdot 10^{-6}$, уникальность гарантируется ограничением `UNIQUE` и повторной генерацией (до пяти попыток).

3. **Требования.** Сформулированы 12 функциональных и 8 нефункциональных требований, выполнен анализ угроз по классификации OWASP. В реализации обеспечены ограничение частоты регистрации, входа и создания ссылок, подпись токенов HMAC-SHA256 с постоянным временем сравнения, bcrypt со стоимостью 12, параметризованные SQL-запросы, вывод в дашборде средствами DOM и строгая политика CSP.

4. **Архитектура и проектирование.** Спроектирована слоистая архитектура с интерфейсами хранилища, кэша и сборщика аналитики, построены диаграммы контейнеров, вариантов использования,

последовательности, классов, ER-диаграмма и диаграмма развёртывания; определён REST API (глава 2).

5. **Аналитика без замедления редиректа и обезличивание.** Реализован асинхронный конвейер: обработчик помещает событие в буферизованный канал неблокирующей отправкой, пул воркеров пакетно записывает переходы командой `COPY`, при остановке очередь дренируется. Реализовано обезличивание: IP-адрес не сохраняется, уникальные посетители определяются по усечённому SHA-256 от соли, IP и User-Agent, реферер сводится к домену.

6. **Технологии, контейнеризация и CI.** Обоснован выбор Go, PostgreSQL, Redis, pgx и go-redis, собственных HMAC-токенов. Подготовлены многоэтапный образ на основе `distroless`, Compose с проверками работоспособности и конвейер GitHub Actions из пяти заданий (тесты с детектором гонок, интеграционные тесты, линтер, `gosec` и `govulncheck`, сборка образа).

7. **Реализация и проверка.** Сервис реализован и покрыт тестами: 86 функций тестов (211 с подтестами) проходят успешно, покрытие пакетов с модульными тестами — от 81,5 до 100 % (90,8 % суммарно; 70,8 % по всем пакетам), хранилище PostgreSQL проверяется интеграционными тестами в CI. Тесты выявили три дефекта собственного кода, все исправлены, и два дефекта сторонней библиотеки QR-кодов, обойдённых собственной отрисовкой. Статический анализ (`go vet`, `golangci-lint`, `gosec` — 28 файлов, 3858 строк) замечаний не дал; `govulncheck` нашёл 13 уязвимостей стандартной библиотеки go1.27.1, устранённых фиксацией инструментальной цепочки go1.27.2.

8. **QR-коды ссылок.** Реализована выдача QR-кода короткой ссылки (`GET /qr/{code}.png?size=N`) и окно просмотра и скачивания в дашборде. Кодирование выполняет библиотека `rsc.io/qr`, отрисовка в 1-битный PNG написана самостоятельно из-за двух дефектов библиотеки, найденных тестом с независимым декодером gozxing; буферы PNG-кодировщика

переиспользуются через пул (память на операцию снижена в 14 раз). Показ QR-кода не искажает статистику переходов. Для проверки с телефона подготовлен запуск `Запустить для телефона.bat`, определяющий IP-адрес ноутбука в локальной сети.

9. **Сохранение данных демонстрационного режима.** Без PostgreSQL данные существовали только в памяти, а секрет подписи токенов создавался заново при каждом запуске, поэтому после закрытия программы пропадали и данные, и вход. Добавлена переменная `DATA_FILE`: тип `memory.Persistent` загружает снимок состояния из JSON-файла и сохраняет его после изменений с задержкой 0,5 с (счётчик версии исключает пустые записи, поток переходов не превращается в поток записей) и при остановке. Запись атомарна (временный файл, `Sync`, `Rename`), повреждённый файл не перезаписывается, а секрет хранится в отдельном файле с правами 0600. Работоспособность подтверждена шестью модульными тестами хранилища, тестами конфигурации и сквозной проверкой с жёстким завершением процесса: после перезапуска токен принят, вход и статистика на месте.

Достигнутые показатели. Разрешение закэшированной ссылки в сервисном слое занимает около 58 нс (одна аллокация); обработка редиректа в HTTP-слое — около 10 мкс без сети. В демонстрационном режиме при нагрузке с той же машины сервис обработал около 5,6 тыс. редиректов в секунду без ошибок транспорта; при 64 одновременных запросах 99-й процентиль задержки составил 14,9 мс, при 256 — 55,7 мс, потерянных событий аналитики не зафиксировано. Эти значения получены в одной конфигурации (клиент и сервер на одной машине, хранилище и кэш в памяти) и являются оценкой снизу; замеры с PostgreSQL и Redis, а также сценарием k6 не проводились.

Ограничения. Файл данных демонстрационного режима рассчитан на один процесс и небольшой объём данных и не заменяет PostgreSQL; последние ~0,5 с изменений при аварийном завершении могут быть потеряны. Состояние ограничителя частоты хранится в памяти процесса; редиректы не

лимитируются на уровне приложения; токен не отзывается до истечения срока действия; рейтинг популярных ссылок приближённый; при сбое записи пакета аналитики его данные теряются; сервис не проверяет адреса по базам вредоносных ресурсов.

Перспективы развития.

– Пользовательские домены для брендированных ссылок.

– Кластер Redis (Redis Cluster или Sentinel) для отказоустойчивости кэша и общего состояния ограничителя частоты.

– Перенос аналитики в колоночное хранилище (ClickHouse) и предварительная агрегация; секционирование таблицы `clicks` по времени в PostgreSQL.

– Аутентификация через OAuth/OpenID Connect и механизм отзыва токенов.

– Экспорт метрик в Prometheus и построение панелей в Grafana; трассировка запросов.

– Ограничение частоты редиректов по IP и по коду ссылки.

– CAPTCHA при регистрации и интеграция проверки адресов по базам фишинговых ресурсов.

– Профилирование и тестирование производительности с реальными PostgreSQL и Redis, буферизация журнала доступа.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Донован А. А., Керниган Б. У. Язык программирования Go. — М.: Вильямс, 2018. — 432 с.

2. Клеппман М. Высоконагруженные приложения. Программирование, масштабирование, поддержка. — СПб.: Питер, 2018. — 640 с.

3. Макконнелл С. Совершенный код. Мастер-класс. — 2-е изд. — М.: Русская редакция, 2010. — 896 с.

4. Фаулер М. Архитектура корпоративных программных приложений. — М.: Вильямс, 2006. — 544 с.

5. Ричардсон К. Микросервисы. Паттерны разработки и рефакторинга. — СПб.: Питер, 2019. — 544 с.

6. Мартин Р. Чистая архитектура. Искусство разработки программного обеспечения. — СПб.: Питер, 2018. — 352 с.

7. Карлсон Дж. Redis в действии. — М.: ДМК Пресс, 2016. — 450 с.

8. RFC 9110. HTTP Semantics / R. Fielding, M. Nottingham, J. Reschke. — Текст: электронный // RFC Editor: [сайт]. — 2022. — URL: https://www.rfc-editor.org/rfc/rfc9110 (дата обращения: 15.10.2026).

9. RFC 2104. HMAC: Keyed-Hashing for Message Authentication / H. Krawczyk, M. Bellare, R. Canetti. — Текст: электронный // RFC Editor: [сайт]. — 1997. — URL: https://www.rfc-editor.org/rfc/rfc2104 (дата обращения: 15.10.2026).

10. The Go Programming Language. Documentation. — Текст: электронный // go.dev: [сайт]. — URL: https://go.dev/doc/ (дата обращения: 15.10.2026).

11. PostgreSQL Documentation. — Текст: электронный // PostgreSQL: [сайт]. — URL: https://www.postgresql.org/docs/ (дата обращения: 15.10.2026).

12. Redis Documentation. — Текст: электронный // Redis: [сайт]. — URL: https://redis.io/docs/ (дата обращения: 15.10.2026).

13. pgx — PostgreSQL Driver and Toolkit for Go. — Текст: электронный // GitHub: [сайт]. — URL: https://github.com/jackc/pgx (дата обращения: 15.10.2026).

14. go-redis: Redis client for Go. — Текст: электронный // GitHub: [сайт]. — URL: https://github.com/redis/go-redis (дата обращения: 15.10.2026).

15. Docker Documentation. — Текст: электронный // Docker Docs: [сайт]. — URL: https://docs.docker.com/ (дата обращения: 15.10.2026).

16. GitHub Actions Documentation. — Текст: электронный // GitHub Docs: [сайт]. — URL: https://docs.github.com/actions (дата обращения: 15.10.2026).

17. OWASP Top 10. — Текст: электронный // OWASP Foundation: [сайт]. — URL: https://owasp.org/www-project-top-ten/ (дата обращения: 15.10.2026).

18. Unvalidated Redirects and Forwards Cheat Sheet. — Текст: электронный // OWASP Cheat Sheet Series: [сайт]. — URL: https://cheatsheetseries.owasp.org/cheatsheets/Unvalidated_Redirects_and_Forwards_Cheat_Sheet.html (дата обращения: 15.10.2026).

19. О персональных данных: Федеральный закон от 27.07.2006 № 152-ФЗ. — Текст: электронный // КонсультантПлюс: [сайт]. — URL: https://www.consultant.ru/document/cons_doc_LAW_61801/ (дата обращения: 15.10.2026).

20. GeoLite2 Free Geolocation Data. — Текст: электронный // MaxMind: [сайт]. — URL: https://dev.maxmind.com/geoip/geolite2-free-geolocation-data (дата обращения: 15.10.2026).

ПРИЛОЖЕНИЯ

Приложение А. Инструкция по запуску

А.1 Демонстрационный режим (без внешних сервисов)

Для запуска требуется Go версии 1.27 или новее. Если `DATABASE_URL` и `REDIS_URL` не заданы, сервис использует хранилище и кэш в памяти; без дополнительных настроек данные не сохраняются между перезапусками. Если не задан и `AUTH_SECRET`, при каждом запуске генерируется случайный секрет (токены перестают действовать после перезапуска).

```
git clone https://github.com/Skufik2007/url-shortener.git
cd url-shortener
go run ./cmd/shortener
```

После запуска дашборд доступен по адресу `http://localhost:8080/` (перенаправление на `/dashboard/`). Для запуска с другим адресом задаются переменные окружения `HTTP_ADDR` и `BASE_URL`, например `HTTP_ADDR=127.0.0.1:8091 BASE_URL=http://localhost:8091`. Сборка бинарного файла: `make build` (результат — `bin/shortener`).

Чтобы данные демонстрационного режима переживали перезапуск, задаётся переменная `DATA_FILE` с путём к JSON-файлу (при пустом `DATABASE_URL`):

```
DATA_FILE=data.json go run ./cmd/shortener
```

При запуске данные загружаются из файла, после изменений (с задержкой около 0,5 с) и при остановке сохраняются обратно. Если `AUTH_SECRET` не задан, секрет подписи токенов хранится в файле `data.json.secret` (создаётся автоматически с правами 0600), поэтому вход тоже сохраняется. Для сброса данных достаточно остановить сервис и удалить оба файла; для резервной копии — скопировать их. `DATA_FILE` вместе с `DATABASE_URL` — ошибка конфигурации; режим предназначен только для демонстрации, для продакшена применяется PostgreSQL (раздел А.2). В

Windows-поставке `Запустить.bat` и `Запустить для телефона.bat` задают `DATA_FILE=data.json` и хранят файлы рядом с `shortener.exe`.

A.2 Docker Compose (PostgreSQL и Redis)

Требуются Docker и Docker Compose. Секрет подписи токенов обязателен.

```
cp .env.example .env
# в файле .env задать AUTH_SECRET: строка не короче 32 байт, например,
openssl rand -hex 32
docker compose up -d --build
```

Будут запущены три контейнера: `app` (порт 8080), `postgres` (PostgreSQL 17, том `pgdata`) и `redis` (Redis 7, `maxmemory 128mb`, `allkeys-lru`). Приложение стартует после успешных проверок работоспособности баз и самостоятельно применяет миграции. Проверка состояния: `curl http://localhost:8080/healthz`. Остановка: `docker compose down` (флаг `-v` дополнительно удаляет данные PostgreSQL).

A.3 Основные параметры конфигурации

Параметры задаются переменными окружения (таблица 2.6): `HTTP_ADDR`, `BASE_URL`, `DATABASE_URL`, `DATA_FILE`, `REDIS_URL`, `AUTH_SECRET`, `TOKEN_TTL`, `GEOIP_DB`, `TRUST_PROXY`, `CODE_LENGTH`, `ANALYTICS_WORKERS`, `ANALYTICS_BUFFER`, `ANALYTICS_BATCH`, `ANALYTICS_FLUSH`, `RATE_LIMIT_RPS`, `RATE_LIMIT_BURST`, `LOG_LEVEL`. Если сервис работает за обратным прокси, следует установить `TRUST_PROXY=true`.

A.4 Тесты и проверки

```
go test ./...                                    # модульные тесты
go test -race ./...                              # с детектором гонок
go test -tags integration ./...                  # интеграционные (нужен
TEST_DATABASE_URL)
go test -run='^$' -bench=. -benchmem ./internal/service/ ./internal/httpapi/
go run ./cmd/loadgen -url http://127.0.0.1:8080/<код> -c 64 -d 10s
```

Приложение Б. Спецификация REST API

Базовый путь — `/api/v1`. Тела запросов и ответов — JSON в кодировке UTF-8. Защищённые конечные точки требуют заголовок `Authorization: Bearer <token>`. Ошибки возвращаются в формате `{"error": "описание"}`. Время — в формате RFC 3339. Примеры значений условные.

Б.1 Регистрация

`POST /api/v1/auth/register`

```
{"email": "demo@example.com", "password": "correct-horse"}
```

Ответ `201 Created`:

```
{"token": "MS4xNzkyMDAwMDAw.q3h7...", "user": {"id": 1, "email":
"demo@example.com", "created_at": "2026-10-15T10:00:00Z"}}
```

Ошибки: `400` (`invalid email`, `password must be 8 to 72 bytes long`, `invalid JSON body`), `409` (`email already registered`), `413`, `429`.

Б.2 Вход

`POST /api/v1/auth/login` — тело как при регистрации. Ответ `200 OK` в том же формате. Ошибка `401`: `{"error": "invalid email or password"}` (одинаково для неверного email и неверного пароля), `429`.

Б.3 Текущий пользователь

`GET /api/v1/me` → `200 OK`: `{"id": 1}`. Без действительного токена — `401` с заголовком `WWW-Authenticate: Bearer realm="shortener"`.

Б.4 Создание ссылки

`POST /api/v1/links`

```
{"url": "https://go.dev/doc/", "alias": "godoc", "title": "Документация Go",
"expires_at": "2027-01-01T00:00:00Z"}
```

Обязательно только поле `url`; `alias` (3–32 символа `A-Za-z0-9_-`, не начинается с `-` и `_`, не зарезервирован), `title` (до 200 символов) и `expires_at`

(в будущем) необязательны. Ответ `201 Created`, заголовок `Location: <BASE_URL>/api/v1/links/godoc`:

```
{
  "id": 4,
  "code": "godoc",
  "original_url": "https://go.dev/doc/",
  "owner_id": 1,
  "title": "Документация Go",
  "created_at": "2026-10-15T10:05:00Z",
  "expires_at": "2027-01-01T00:00:00Z",
  "is_active": true,
  "click_count": 0,
  "short_url": "http://localhost:8080/godoc"
}
```

Ошибки: `400` (недопустимый адрес или схема, ссылка на сам сервис, некорректный алиас, срок в прошлом), `401`, `409` (`alias already taken`), `413`, `429`. Поля `title` и `expires_at` в ответе опускаются, если не заданы.

Б.5 Список ссылок

`GET /api/v1/links?limit=50&offset=0` — активные ссылки владельца, новые первыми; `limit` от 1 до 100 (по умолчанию 50), `offset` не меньше 0. Ответ `200 OK`:

```
{"links": [ { ...объект ссылки, как в Б.4... } ]}
```

Ошибки: `400` (`invalid limit`, `invalid offset`), `401`.

Б.6 Получение и удаление ссылки

`GET /api/v1/links/{code}` → `200 OK` с объектом ссылки (как в Б.4); `404` — ссылка не найдена, отключена или принадлежит другому пользователю.

`DELETE /api/v1/links/{code}` → `204 No Content` (ссылка деактивируется, запись удаляется из кэша); повторное удаление и чужая ссылка — `404`.

Б.7 Статистика

`GET /api/v1/links/{code}/stats?from=2026-09-16&to=2026-10-15` — даты в формате `YYYY-MM-DD`, граница `to` включается; по умолчанию — последние 30 суток; максимальный период — 366 суток. Ответ `200 OK`:

```
{
  "code": "godoc",
  "from": "2026-09-16T00:00:00Z",
  "to": "2026-10-16T00:00:00Z",
  "total_clicks": 163,
  "unique_visitors": 153,
  "by_day": [{"day": "2026-10-14T00:00:00Z", "count": 163}],
  "by_country": [{"key": "RU", "count": 61}, {"key": "UZ", "count": 26}],
  "by_referrer": [{"key": "direct", "count": 44}, {"key": "vk.com", "count":
30}],
  "by_browser": [{"key": "Safari", "count": 67}],
  "by_os": [{"key": "Windows", "count": 55}],
  "by_device": [{"key": "desktop", "count": 92}]
}
```

Списки `by_*` (кроме `by_day`) отсортированы по убыванию числа переходов и ограничены десятью элементами. Ошибки: `400` (`invalid 'from' date...`, `invalid 'to' date...`, `invalid period`), `401`, `404`.

Б.8 Популярные коды

`GET /api/v1/top?n=10` (без аутентификации; `n` от 1 до 100, по умолчанию 10) → `200 OK`: `{"codes": ["godoc", "abc1234"]}`; при неверном `n` — `400`.

Б.9 Редирект

`GET /{code}` → `302 Found` с заголовками `Location: <исходный адрес>` и `Cache-Control: private, max-age=0`. Если код не найден или недопустим — `404` (`{"error": "link not found"}`); если ссылка отключена или истекла — `410` (`{"error": "link expired or disabled"}`).

Б.10 QR-код ссылки

`GET /qr/{code}.png?size=256` (без аутентификации; `size` — сторона изображения в пикселях от 64 до 1024, по умолчанию 256). В QR-коде

закодирована короткая ссылка `BASE_URL/{code}`; выдача кода не считается переходом.

```
curl -o qr.png 'http://localhost:8080/qr/my-link.png?size=512'
```

Код	Условие	Тело
`200 OK`	ссылка существует, активна и не истекла	PNG-изображение (`Content-Type: image/png`, `Cache-Control: public, max-age=3600`)
`400 Bad Request`	`size` не число или вне диапазона 64–1024	`{"error": "size must be between 64 and 1024"}`
`404 Not Found`	код не найден или недопустим, либо в адресе нет расширения `.png`	`{"error": "link not found"}`
`410 Gone`	ссылка отключена или истекла	`{"error": "link expired or disabled"}`

Сторона изображения кратна размеру модуля и не превышает запрошенную; вокруг кода оставлена белая рамка шириной четыре модуля. Число выданных изображений учитывается метрикой `shortener_qr_generated_total`.

Б.11 Служебные конечные точки

`GET /healthz` → `200 OK`: `{"status": "ok", "checks": {"database": "ok", "cache": "ok"}}`; при отказе зависимости — `503`: `{"status": "unavailable", "checks": {"database": "error", "cache": "ok"}}`.

`GET /metrics` → `200 OK`, `Content-Type: text/plain; version=0.0.4`:

```
# HELP shortener_redirects_total Успешные редиректы.
# TYPE shortener_redirects_total counter
shortener_redirects_total 198378
```

Полный перечень метрик приведён в разделе 4.11. Ответ `429` на лимитируемых конечных точках содержит заголовок `Retry-After` (секунды).

Приложение В. SQL-миграция `001_init.sql`

Таблица `schema_migrations` создаётся кодом применения миграций и в файле отсутствует.

```
-- Начальная схема сервиса сокращения ссылок.

CREATE TABLE users (
    id            BIGSERIAL PRIMARY KEY,
    email         TEXT NOT NULL UNIQUE,
    password_hash TEXT NOT NULL,
    created_at    TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE links (
    id           BIGSERIAL PRIMARY KEY,
    code         VARCHAR(32) NOT NULL UNIQUE,
    original_url TEXT NOT NULL,
    owner_id     BIGINT REFERENCES users(id) ON DELETE CASCADE,
    title        TEXT NOT NULL DEFAULT '',
    created_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
    expires_at   TIMESTAMPTZ,
    is_active    BOOLEAN NOT NULL DEFAULT true,
    click_count  BIGINT NOT NULL DEFAULT 0
);

CREATE INDEX idx_links_owner_created ON links (owner_id, created_at DESC);

CREATE TABLE clicks (
    id         BIGSERIAL PRIMARY KEY,
    link_id    BIGINT NOT NULL REFERENCES links(id) ON DELETE CASCADE,
    clicked_at TIMESTAMPTZ NOT NULL,
    referrer   TEXT NOT NULL DEFAULT '',
    country    CHAR(2) NOT NULL DEFAULT 'ZZ',
    browser    TEXT NOT NULL DEFAULT '',
    os         TEXT NOT NULL DEFAULT '',
    device     TEXT NOT NULL DEFAULT '',
    visitor_id TEXT NOT NULL DEFAULT ''
);

CREATE INDEX idx_clicks_link_time ON clicks (link_id, clicked_at);
```

Приложение Г. История коммитов

Вывод команды `git log --oneline --graph develop` (ветка `develop`, на момент подготовки записки). Разработка велась в тематических ветках `feature/*`, влитых в `develop` слиянием.

```
*   edf68cd Merge branch 'feature/docs' into develop
|\
| * e98fde8 docs: README, лицензия MIT и скриншоты дашборда
|/
*   a55b9e3 Merge branch 'feature/perf' into develop
|\
```

```
| * a0e3de2 feat(loadgen): генератор нагрузки с перцентилями задержки
| * 74fb7b5 test: бенчмарки разрешения ссылок и HTTP-редиректа
|/
* 000e950 chore: единые окончания строк LF через .gitattributes
*   6cc2ef0 Merge branch 'feature/infra' into develop
|\
| * 9d15dbc test: сценарий нагрузочного тестирования k6
| * 261fc36 ci: GitHub Actions (тесты, интеграция, линтер, gosec,
govulncheck, сборка образа) и dependabot
| * 9bafeb8 build: Docker, Docker Compose, Makefile и конфигурация golangci-
lint
|/
*   19c15e2 Merge branch 'feature/http-api' into develop
|\
| * 6ff1478 feat(cmd): точка входа с graceful shutdown
| * 88b60d0 feat(web): дашборд владельца ссылок со статистикой
| * 0e44e8f feat(api): REST API, редиректы, middleware, rate limiting и
метрики Prometheus
| * c008e1b feat(config): загрузка конфигурации из переменных окружения
|/
*   f78613a Merge branch 'feature/service' into develop
|\
| * 800fe90 feat(service): бизнес-логика ссылок с кэшированием популярных и
деградацией при отказе Redis
|/
*   a49150f Merge branch 'feature/auth' into develop
|\
| * b060d9d feat(auth): регистрация и вход с bcrypt, HMAC-подписанные токены
|/
*   6a67ccd Merge branch 'feature/analytics' into develop
|\
| * 6d89eda feat(analytics): асинхронный сборщик переходов с пулом воркеров и
пакетной записью
| * d9f385d feat(analytics): разбор User-Agent, домена реферера и GeoIP
|/
*   654be95 Merge branch 'feature/shortcode-cache' into develop
|\
| * 1fbca16 feat(cache): кэш популярных ссылок на Redis и в памяти, счётчики
обращений
| * b7608d2 feat(shortcode): генератор base62 на crypto/rand и валидация
алиасов
|/
*   8e17674 Merge branch 'feature/storage' into develop
|\
| * 635a62f feat(storage): хранилище PostgreSQL на pgx с встроенными
миграциями и COPY-вставкой
| * a35b80c feat(storage): потокобезопасное хранилище в памяти для тестов и
демо-режима
|/
*   8ed907e Merge branch 'feature/domain' into develop
|\
| * 0385c07 feat(domain): доменные модели и интерфейсы хранилища, кэша и
аналитики
|/
* e0f0532 chore: инициализация модуля Go и проектный документ архитектуры
```